cDFT_solver

Grand User, Input/Output, Theory, Example Catalogue, Workflow, and Developer Manual

Source-audited, theory-enriched grand edition based on the updated package snapshot and two supplied research papers

Primary emphasis: exact input files, numerical-data contracts, all bundled example workflows, supplied-data combinations, extensibility, coexistence constraints, structural-correlation theory, and end-to-end physical calculations.

Scope of this edition — This grand manual was prepared by reading the current Python modules and the complete supplied Examples catalogue in the supplied archive. The snapshot contains 20 executable example workflow scripts, 21 input files, and 19 unique input definitions. The manual distinguishes code that is directly reusable from example-specific scan values and current implementation caveats.

Package author identified in the repository README/setup metadata: Vikki Anand Varma.

Manual date: 16 August 2026. Package metadata remains internally inconsistent: setup.py reports version 0.2.0, while pyproject.toml reports 0.1.1. This manual therefore identifies the audited source snapshot rather than asserting a single release version.

Table of Contents

Part- and chapter-level navigation for this source-audited edition

Page numbers below correspond to the final merged-example render.

PART I — Orientation, Installation, and Package Architecture

1. How to Read This Manual

The cDFT_solver package is easiest to understand as a sequence of data transformations. A human-readable .in file is parsed into a nested Python dictionary. Potential modules turn the interaction description into hard-core, mean-field, raw, and combined numerical potential dictionaries. Calculator modules then consume those dictionaries to evaluate correlation functions, free-energy expressions, integrated strengths, thermodynamic states, phase coexistence, or effective potentials.

The key rule — Do not think of the .in file as a loose configuration file. Its hierarchy determines the exact nested dictionary keys later functions search for. A line that is visually similar but produces a different nesting can make a later module silently miss a parameter or fall back to a default.

1.1 The core data pipeline

Stage	Primary object	Produced by	Consumed by
1. Human input	.in text file	User	super_dictionary_creator
2. Parsed configuration	{"system": {...}} dictionary	super_dictionary_creator	all generators/calculators
3. External numerical data	two-column r–value text files	simulation / user	process_supplied_data or potential splitters
4. Potential preprocessing	hc_data, mf_data, raw_data, total_data	potential_splitter modules	free energy, RDF, kernels
5. Symbolic thermodynamics	free_energy result with SymPy variables/expression	total_free_energy	phase_finder / coexistence
6. Structural calculation	g(r), c(r), gamma(r), G_u(r), delta-c, B2	RDF / c_2 / g_alpha modules	analysis and integrated strength
7. State calculation	mu, pressure, free-energy density, vij	phase_finder	EOS tables
8. Coexistence	densities per phase, mu, pressure, vij	coexistence_densities	binodal workflow
9. Inversion	u_ij(r), sigma_ij plus diagnostics	rdf_inversion_advance	effective-interaction analysis

1.2 What is generalized and what is preserved

> Theory-enriched edition note — Chapters 5, 8, 13, 15, 18, and 29 now distinguish software contracts from the underlying liquid-state and phase-equilibrium theory. The two supplied research papers are cited where they provide the theoretical basis; implementation observations continue to come from the audited package source. This separation is intentional: an equation may be thermodynamically general while a configuration key is specific to the current code snapshot.

The worked examples in Chapters 11–19 preserve the API and physics of the bundled examples but remove dependence on repository folder names. The portable pattern resolves an input path, makes its parent the run directory, and writes generated data to a scratch directory. The exact active .in definitions and cleaned reachable driver cores are reproduced together in merged Chapter 21–22 so that users can compare their own inputs against a known source snapshot.

2. Installation and Runtime Requirements

The repository README recommends installation directly from GitHub, while setup.py/pyproject.toml declare Python >= 3.8 and NumPy, SciPy, and Matplotlib. The audited source imports additional packages in active workflows, most importantly SymPy, tqdm, and Numba. The older engine modules also import pyFFTW.

Recommended environment for the demonstrated workflows

```
python -m venv cdft_env
source cdft_env/bin/activate
pip install --upgrade pip
pip install numpy scipy matplotlib sympy tqdm numba
pip install git+https://github.com/vikkivarma16/cDFT_solver.git
```

Compiled OZ library — Several radial/OZ modules load liboz_radial.so, .dylib, or .dll using ctypes. The archive includes Linux .so files in the relevant subpackages. On another platform or incompatible architecture, the C library may need to be compiled for that system. The C source includes a Linux build hint using LAPACKE, OpenBLAS, and FFTW3.

2.1 Minimal import smoke test

```
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
print('cDFT_solver import successful')
```

2.2 Package metadata note

File	Version / requirement observed	Manual interpretation
setup.py	version 0.2.0; Python >=3.8; numpy/scipy/matplotlib	Likely newer setup metadata
pyproject.toml	version 0.1.1; Python >=3.8; numpy/scipy/matplotlib	Version should be synchronized before release
active calculator imports	sympy, numba; examples use tqdm	Install explicitly for documented workflows
legacy engines	pyfftw plus imports of modules absent from current tree	Do not use as primary workflow in this snapshot

3. Package Architecture

The current package is modular. The runnable examples are distributed throughout the Examples tree as files named example_workflow_*.py. These direct drivers call package modules explicitly and are the most reproducible interface represented by the supplied example snapshot.

3.1 Directory roles

Directory	Role	Representative modules
cdft_solver/generators/parameters	Parse human input into hierarchical dictionaries	advance_dictionary.py
cdft_solver/generators/potential	Registry and analytic isotropic pair potentials	pair_potential_isotropic_default.py, registry.py
cdft_solver/generators/potential_splitter	Construct hard-core, mean-field, raw, and total potential representations	hc.py, mf.py, raw.py, total.py
cdft_solver/generators/supplied_data	Load file-backed x-y data into numerical arrays	process_supplied_data.py
cdft_solver/calculators/ radial_distribution_function	OZ + closure structural calculation	rdf_radial.py, rdf_two_d.py, rdf_planer.py
cdft_solver/calculators/rdf_inversion	Multistate Boltzmann/IBI effective-potential inversion	rdf_inversion_advance.py
cdft_solver/calculators/c_2_analysis	Reference-system, direct-correlation, ΔB2	c_analysis.py

	and G(r) analysis	
cdft_solver/calculators/g_alpha_r	Coupling-parameter kernels for integrated interaction strength	rdf_alpha_r.py, delta_c.py
cdft_solver/calculators/free_energy_*	Symbolic ideal, mean-field, hard-core and hybrid free energies	ideal.py, mean_field.py, hard_core_bulk_*.py
cdft_solver/calculators/total_free_energy	Merge symbolic components and export them	total_free_energy.py, free_energy_exporter.py
cdft_solver/calculators/ integrated_strength	Convert radial kernels to pair strengths v_ij	integrated_strength_radial_kernal.py
cdft_solver/calculators/phase_finder	Evaluate one thermodynamic state from density or chemical-potential constraints	thermodynamic_potentials_isocore.py / isochem.py
cdft_solver/calculators/ coexistence_densities	Solve multiphase coexistence	calculator_coexistence_densities_isochem.py / isocore.py
cdft_solver/calculators/virial_analysis	Second-virial evaluation and scaling/calibration	second_virial.py, second_virial_scale_calibration.py
cdft_solver/calculators/ one_d_profile_iterator	Planar profile iteration support	one_d_profile_iterator_box.py

3.2 Recommended entry points

- For a new calculation, create an ExecutionContext and parse the .in file with super_dictionary_creator.
- For potential/free-energy/EOS work, call hard_core_potentials, meanfield_potentials, raw_potentials, total_potentials, total_free_energy, then a phase-finder routine.
- For structural correlation work, call c_analysis or rdf_radial after parsing the same system dictionary.
- For inversion, first materialize supplied RDF data with process_supplied_data, then call boltzmann_inversion_advance.
- For phase coexistence, build the free-energy result once and then vary constraints on deep copies of the parsed configuration.

Legacy executor status — executor_dft_main.py imports get_unique_dir, which is not present in the current utils.py, and the older engine modules import several generator/calculator modules not present under those names. The direct example_workflow_*.py drivers bundled in the distributed Examples folders are therefore documented as the primary interface.

PART II — Input Language, Configuration, and Data Contracts

4. The .in Input Language: Exact Parsing Rules

The parser in generators/parameters/advance_dictionary.py reads each nonempty, noncomment line independently. It uses commas to identify multiple attributes or continuation values, then uses the text before the first equals sign to construct hierarchy keys.

4.1 Comments and blank lines

- A line is ignored when it is empty after stripping whitespace.
- A line is ignored when its first non-whitespace character is #.
- A line without = is ignored.
- Inline comments are not stripped by super_dictionary_creator. Therefore place comments on separate lines rather than after a value.

4.2 Numeric conversion

Every value is stripped and passed through float(). If conversion succeeds, an integral floating value is stored as int; otherwise it is stored as float. If float conversion fails, the value remains a string. Therefore yes, no, None, py, hnc, standard, and filenames are strings; 1.0 becomes integer 1; 1e-5 becomes floating 1e-05.

4.3 Hierarchy construction

```
rdf closure aa = py
solution extrinsic_constraints: density: a = 0.15
interactions primary: aa: type = mie, sigma = 1.0, cutoff = 3.5, epsilon = 0.5, m = 24, n = 12
```

These lines become nested keys by splitting the hierarchy using either colons or whitespace. The final attribute token immediately to the left of the first equals sign is removed from the hierarchy and inserted as an attribute/value at the deepest level.

Resulting nested structure

```
{
  "rdf": {
    "closure": {
      "aa": "py"
    }
  },
  "solution": {
    "extrinsic_constraints": {
      "density": {
        "a": 0.15
      }
    }
  },
  "interactions": {
    "primary": {
      "aa": {
        "type": "mie",
        "sigma": 1,
        "cutoff": 3.5,
        "epsilon": 0.5,
        "m": 24,
        "n": 12
      }
    }
  }
}
```

4.4 Commas: attributes versus list continuation

A comma-separated segment containing = starts a new attribute. A comma-separated segment without = is appended to the previous attribute as another value. This is why species = a, b produces ["a", "b"], while a one-species line species = a produces the string "a".

Recommended species syntax — Use single-character species labels such as a, b, c in the present package and explicitly list all species with commas. Multiple modules create pair keys by concatenating species names and

some hard-core code interprets pair keys with key[0] and key[1]. Multi-character species names are therefore not safe in the current implementation.

4.5 Repeated keys

Repeated dictionary branches are recursively merged. This makes separate lines for aa, ab, and bb interactions or closures natural. Conflicting non-dictionary values can be converted into lists by the generic merge routine, so a user should avoid defining the same scalar key more than once unless a list is intentionally desired.

4.6 base_dict override behavior

When base_dict is passed, the parser recursively searches the parsed result for keys present in base_dict and replaces the entire matched value. This is powerful but broad: a generic key can match deeper than expected. For reproducible public workflows, explicit edits to the parsed system dictionary are usually clearer than a large base_dict override.

5. Master Input-File Reference

This chapter defines the data blocks used by the bundled workflows and the active modules they feed. Not every workflow needs every block.

5.1 species

Single-species parser detail — In the current parser, “species = a” is returned as the scalar string "a", while “species = a, b” is returned as the list ["a", "b"]. Many present examples use one-character labels, so downstream code that applies list(species) still behaves as intended. For multi-character single-species labels, explicitly normalize the parsed value to a one-element list before using modules that assume an iterable of species names.

Input form	Parsed value	Meaning	Downstream assumption
species = a	"a"	one species	Most current modules convert/iterate it successfully because the label has one character.
species = a, b	["a","b"]	two species	Pair order is aa, ab, bb with symmetric matrix storage.
species = a, b, c	["a","b","c"]	three species	Pairs: aa, ab, ac, bb, bc, cc; every required closure must be provided.

5.2 interactions: primary / secondary / tertiary

Interactions are grouped by level and pair. The raw potential sums all levels for a pair. The mean-field splitter first applies a conversion registry to each level; the hard-core detector inspects every defined level. The current code explicitly recognizes the three level names primary, secondary, and tertiary.

```
interactions primary: aa: type = gs, sigma = 1.414, cutoff = 3.5, epsilon = 2.0
interactions secondary: aa: type = mie, sigma = 1.0, cutoff = 3.5, epsilon = 0.2, m = 12, n = 6
```

A pair may instead be file-backed:

```
interactions primary: aa: filename = pair_aa.txt
```

5.3 Interaction parameter reference

Key	Type	Required?	Meaning in current implementation
type	string	for analytic interaction	Registry key such as gs, lj, mie, hc.
filename	string	alternative to type	Tabulated potential; first column r, second column U(r).
sigma	float	potential dependent	Characteristic length / hard-core diameter; default generally 1.0.
epsilon	float	potential dependent	Energy amplitude; default generally 1.0.
cutoff	float	potential dependent	Finite evaluation range for LJ/Mie/etc.; also influences numerical grid extent.
m	number	Mie/MA/NVRN dependent	In bundled Mie use, larger repulsive exponent, e.g. m=24.
n	number	Mie/GEM/NVRN dependent	Smaller attractive exponent for Mie, or power for generalized Gaussian.
lambda	float	MA	Multiplier of repulsive power-law term.
r_cutoff	float	custom_1	Inner switching radius; defaults to 2^(1/6)sigma.
m_i, m_j	float	NVRN	Multiplicative species factors in inverse-power potential.

5.4 Current analytic potential registry

Registry names	Current mathematical behavior	Parameters	Notes
zero / zero_potential	0	none	Identically zero.
hc / ghc / hard_core / hard_sphere	U=2e8 for r<=sigma; 0 outside	sigma	Step-like hard core. Only hc/hardcore set explicit flag=1 in hard-core splitter; ghc provides sigma but does not directly set that flag.
lj / lennard-jones	shifted 12–6 LJ to U(cutoff)=0	sigma, epsilon, cutoff	At r≈0 code uses a large protection value.
gs / gaussian	epsilon exp[-(r/sigma)^2]	sigma, epsilon	The callable itself does not truncate at cutoff; grid construction can still use cutoff.
gem / gem_n / ggs / generalized_gaussian	epsilon exp[-(r/sigma)^n]	sigma, epsilon, n	Generalized Gaussian.

mie	shifted Mie power law	sigma, epsilon, cutoff, m, n	Bundled examples use m>n, e.g. 24/12.
salj	shifted LJ, flattened below numerical minimum	sigma, epsilon, cutoff	Used automatically as mean-field representation of lj.
ma	Mie-like attraction, flattened below numerical minimum	sigma, epsilon, cutoff, m, n, lambda	Used automatically as mean-field representation of mie.
custom_1	piecewise constant/12–6 form	sigma, epsilon, cutoff, r_cutoff	Implementation-defined custom potential.
custom_3	9–3-like potential	sigma, epsilon, cutoff	Implementation-defined wall-like radial form.
wca	piecewise function in source	sigma, epsilon, cutoff	Current code should be interpreted by its implementation; it is not the conventional textbook WCA expression over the full core.
nvrn	shifted inverse-power Gamma*m_i*m_j*sigma^n/r^n	epsilon, m_i, m_j, sigma, n, cutoff	Potential capped at 1e8.

5.5 Tabulated potential file contract

```
# r    u(r)
0.01   18.4206807
0.02   18.4206807
...
19.99  -1.74e-12
```

Property	Contract / behavior
Columns	At least two numeric columns. Potential splitters use column 1 as r and column 2 as U; extra columns are not used.
Header	Comment lines beginning with # are accepted by numpy.loadtxt.
Ordering	Use strictly increasing r for interpolation and reliable grid construction.
Left extrapolation	Tabulated potential interpolation uses the first U value below the file domain.
Right extrapolation	Uses U=0 above the file domain.
Path base	Current splitters resolve filename relative to the process current working directory, not automatically relative to ctx.input_file.
Units	No unit conversion is enforced. Use a consistent reduced or physical unit system across r, sigma, density, beta and potential.

Portable path rule — In a general workflow, resolve the input path and change the working directory to input_path.parent before parsing/generating. Then relative filenames in the .in file are naturally relative to the input file location, without depending on any repository folder name.

5.6 rdf block

Input key	Typical value	Consumed by	Meaning / current behavior
rdf closure aa	py / hnc / hybrid	RDF, inversion, c_analysis	Closure for pair aa. Every pair needed by the calculation must be present.
rdf alpha_max	0.01–0.4	OZ iterative solvers	Maximum/adaptive mixing parameter for RDF/OZ iteration.
rdf max_iteration	40000–600000	OZ solvers	Upper iteration limit.
rdf tolerance	1e-5 / 1e-6	many structural modules	OZ convergence tolerance.
rdf tolerence	1e-5 / 1e-6	some supplied examples and some current code paths	Misspelled legacy/example key; see compatibility note below.
rdf beta	1.0	potential scaling, B2, state calculation	Inverse thermal energy beta. No unit conversion.
rdf r_max	5–20	RDF radial grid	Maximum r of structural grid.
rdf n_points	2000	RDF radial grid	Number of interior radial points.
rdf max_iteration_ibi	10000	rdf_inversion_advance	Maximum IBI updates.
rdf alpha_ibi_max	0.1	rdf_inversion_advance	Upper adaptive IBI step factor.
rdf ibi_tolerance	1e-4	rdf_inversion_advance	Convergence threshold on maximum absolute g difference.

Spelling compatibility — The archive contains both tolerance and tolerence. The advanced inversion and c_analysis explicitly read rdf_block.get("tolerance", 1e-6), while the EOS example inputs use tolerence. For new inputs, use tolerance. If a particular function still searches tolerence, its default may be used when tolerance is absent; verify the target module when reproducing an old example.

5.7 Closure names

The built-in registry defines PY, HNC, and hybrid closures. Pair closure matrices are filled symmetrically: if aa, ab, bb are specified for species [a,b], the solver uses ab also for ba.

Name	Role
py	Percus–Yevick closure; especially used for hard/repulsive pairs in the examples.
hnc	Hypernetted-chain closure; used for soft polymer-like pairs in the binary examples.
hybrid	Package-defined hybrid closure available in the registry.

5.8 supplied_data: multistate RDF input

```
supplied_data states 1 rdf aa: filename = g_aa.txt
supplied_data states 1 rdf ab: filename = g_ab.txt
supplied_data states 1 rdf bb: filename = g_bb.txt
supplied_data states 1 densities a = 0.10
supplied_data states 1 densities b = 0.02
supplied_data states 1 beta = 1.0
```

process_supplied_data recursively replaces each {"filename": ...} leaf with {"x": array, "y": array}. In rdf_inversion_advance, each state is then converted to a dense target matrix g_target with shape (N,N,Nr) and a Boolean fixed_mask with shape (N,N). Missing RDF pairs are left unconstrained rather than automatically invented.

5.9 RDF text-file contract

```
# r    g(r)
0.0175  0.0000000000
0.0525  0.0000000000
...
6.9825  0.9998770921
```

Property	Current behavior
Minimum columns	At least 2. Column 1 becomes x/r; column 2 becomes y/g for the usual inversion case.
Extra columns	process_supplied_data stores all remaining columns as a 2D y array; inversion workflows are safest with exactly one g column per file.
Interpolation below first r	Uses the first supplied g value.
Interpolation above last r	Uses g=1.0.
Tail cleaning	Advanced inversion cleans the interpolated RDF tail before using it as target data.
Density metadata	Each state must provide densities. Missing species in a density dictionary are assigned 0.0 by inversion processing.
Temperature metadata	beta is used if present; otherwise temperature is converted as beta=1/temperature; otherwise beta defaults to 1.0.

5.10 free_energy block

Key	Accepted/current values	Meaning
free_energy mode	standard, hybrid, hybrid_lattice	total_free_energy dispatch. Comments mentioning advanced do not match the current dispatcher.
free_energy method	smf, emf, void	Mean-field symbolic model selected by mean_field.py.
free_energy hs_functional	whitebeer, rosenfeld	Hard-core bulk functional for standard mode when sigma is nonzero.
free_energy integrated_strength_kernel	uniform, meanfield, rdf, delta_c	Kernel dispatcher/current v_ij pathway. uniform and meanfield initially create K=1; rdf uses rdf_alpha_r; delta_c uses delta_c_alpha.

free_energy supplied_data	yes/no string in examples	Present in input, but the demonstrated EOS calls pass supplied_data=None to state evaluation.
free_energy ensemble	isochem in binodal example	Used by coexistence configuration.

Important semantic point — In the current EOS source path, integrated_strength_kernel = rdf means "build a coupling-parameter RDF-based kernel internally using rdf_alpha_r," not "read an RDF text file." The bundled EOS workflow calls evaluate_canonical_state(... supplied_data=None).

5.11 solution constraints

```
solution extrinsic_constraints: density: a = 0.15
solution extrinsic_constraints: density: b = 0.05
```

Canonical state evaluation requires a density entry for every species. The result uses the species order to build the density vector.

```
solution extrinsic_constraints: number_of_phases = 2
solution extrinsic_constraints: heterogeneous_pair = None
solution extrinsic_constraints: total_density_bound = 10
solution intrinsic_constraints: chemical_potential: a = 14
```

The isochemical coexistence solver treats intrinsic constraints as fixed intensive quantities and solves for phase compositions/densities subject to equality of unconstrained chemical potentials and pressure. In the parsed input, None is the string "None", not Python None; the solver consequently normalizes it as a one-item heterogeneous-pair list, but because its length is not 2 it is not converted to a species pair.

5.11.1 Thermodynamic meaning of coexistence constraints versus package key names

The supplied phase-separation paper formulates coexistence through two logically different classes of conditions. First are the Gibbs equilibrium equalities: the chemical potential of every freely exchanging species must be the same in each coexisting phase, and the pressure must be common to all phases. These conditions are intrinsic to equilibrium and do not by themselves select a unique canonical state. Second are ensemble or preparation constraints, such as fixed total particle numbers (equivalently average densities) and the phase-volume fractions required by material balance. Those additional constraints select the particular coexistence point realized by the imposed global composition. The paper gives these equations explicitly, although it does not use the literal labels "intrinsic" and "extrinsic"; this manual uses those words as a conceptual classification. [Theory: Varma & Scacchi, Phys. Rev. E 113, 065414 (2026).]

Important package terminology warning: the configuration namespaces `solution intrinsic_constraints` and `solution extrinsic_constraints` are implementation names and do not map one-to-one onto the thermodynamic classification above. For example, in the current isocore solver, `intrinsic_constraints: species_fraction` supplies the fixed overall species densities used in the canonical material-balance equations; thermodynamically these are external/ensemble constraints. Conversely, the isochemical solver uses `intrinsic_constraints: chemical_potential` to pin selected intensive variables, while equality of the remaining chemical potentials and pressure is generated internally by the residual equations.

Concept	**Thermodynamic role**	**Current package representation**
Gibbs chemical-potential equality	Intrinsic equilibrium condition	Generated automatically between phases for unconstrained species; selected

		species may instead be pinned in isochem
Mechanical equilibrium (equal pressure)	Intrinsic equilibrium condition	Generated automatically unless a target pressure is explicitly supplied
Overall species amount / average density	Extrinsic ensemble or preparation constraint	Stored as `intrinsic_constraints: species_fraction` in the isocore implementation
Phase fraction(s)	Extrinsic variable required by canonical material balance	Solved explicitly by the isocore coexistence residual
Number of phases	Problem definition / algorithmic constraint	`extrinsic_constraints: number_of_phases`
Density search bound	Numerical search control, not a thermodynamic coexistence law	`extrinsic_constraints: total_density_bound`
Heterogeneous pair	Solution-selection / phase-labeling aid	`extrinsic_constraints: heterogeneous_pair`

6. Execution Context, Paths, and Output Locations

```
from dataclasses import dataclass
from pathlib import Path

@dataclass
class ExecutionContext:
    input_file: Path | None = None
    input_data: str | None = None
    scratch_dir: Path | None = None
    plots_dir: Path | None = None
```

The context is deliberately small. It carries the human input location and the two main output directories. It does not currently contain work_dir, although process_supplied_data checks for that attribute and otherwise falls back to Path.cwd().

6.1 Portable run-directory pattern

```
from pathlib import Path
import os
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir

input_path = Path('system.in').expanduser().resolve()
os.chdir(input_path.parent)            # makes relative data filenames portable
scratch = create_unique_scratch_dir()  # created beside the input
plots = scratch / 'plots'
plots.mkdir(exist_ok=True)

ctx = ExecutionContext(
    input_file=input_path,
    scratch_dir=scratch,
    plots_dir=plots,
)
```

6.2 Typical files written to scratch

Producer	Typical files

super_dictionary_creator	input_system.json
hard_core_potentials	hc.json or supplied_data_potential_hc.json
meanfield_potentials	mf.json or supplied_data_potential_mf.json
raw_potentials	raw.json or supplied_data_potential_raw.json
total_potentials	total.json or supplied_data_potential_total.json
total_free_energy components	fe_id.json, fe_mf.json, fe_hc.json, fe_hybrid.json
free_energy_exporter	fe_total.json
process_supplied_data	supplied_data_materialized.json
c_analysis	result_sigma_analysis.json, delta_B2_results.json, result_G_of_r.json, delta_c_results.json, <prefix>_potential.json
boltzmann_inversion_advance	reference/diagnostic JSON plus <prefix>_potential.json and RDF/potential plots
evaluate_canonical_state	state_from_density.json if export=True

7. Core Output Data Contracts

7.1 Parsed system dictionary

Example parsed output: two-component binodal


```
{
  "system": {
    "species": [
      "a",
      "b"
    ],
    "interactions": {
      "primary": {
        "aa": {
          "type": "gs",
          "sigma": 1,
          "cutoff": 3.5,
          "epsilon": 2
        },
        "ab": {
          "type": "lj",
          "sigma": 1,
          "cutoff": 1.1224,
          "epsilon": 1
        },
        "bb": {
          "type": "mie",
          "sigma": 1,
          "m": 24,
          "n": 12,
          "cutoff": 2.5,
          "epsilon": 0.1
        }
      }
    },
    "rdf": {
      "closure": {
        "aa": "py",
        "ab": "py",
```

```
        "bb": "py"
      },
      "alpha_max": 0.1,
      "max_iteration": 600000,
      "tolerence": 1e-05,
      "beta": 1,
      "r_max": 10,
      "n_points": 2000
    },
    "free_energy": {
      "ensemble": "isochem",
      "mode": "standard",
      "method": "smf",
      "hs_functional": "whitebeer",
      "integrated_strength_kernel": "uniform",
      "supplied_data": "no"
    },
    "solution": {
      "extrinsic_constraints": {
        "number_of_phases": 2,
        "heterogeneous_pair": "None",
        "total_density_bound": 10
      },
      "intrinsic_constraints": {
        "chemical_potential": {
          "a": 14
        }
      }
    }
  }
}
```

All later modules use recursive lookup or explicit paths into this structure. A JSON export from super_dictionary_creator is therefore an excellent debugging artifact: inspect it before running expensive calculations.

7.2 hard_core_potentials output

```
{
  "species": ["a", "b"],
  "sigma": [[sigma_aa, sigma_ab], [sigma_ba, sigma_bb]],
  "flag": [[flag_aa, flag_ab], [flag_ba, flag_bb]],
  "potentials": {
    "aa": {"r": [...], "U": [...]},
    "ab": {"r": [...], "U": [...]},
    "bb": {"r": [...], "U": [...]}
  }
}
```

sigma and flag are N×N symmetric matrices. The generated hard-core potential for each pair is 1e12 below the effective sigma and zero above it. For tabulated/strongly repulsive interactions, sigma may be estimated using a Barker–Henderson integral after a WCA-like reference split. Off-diagonal diameters are then made additive from self diameters when sufficient information exists.

7.3 meanfield_potentials output

```
{
  "species": ["a", "b"],
  "mf_interactions": {
    "primary": {
      "aa": {...converted interaction...},
      "ab": {...converted interaction...},
      "bb": {...converted interaction...}
    }
  },
  "potentials": {"aa": {"r": [...], "U": [...]}, ...}
```

```
}
```

The conversion registry maps hc/ghc/hard_core to zero_potential, lj to salj, and mie to ma before mean-field potential construction. Other registered types are left unchanged.

7.4 raw_potentials output

```
{
  "species": ["a", "b"],
  "potentials": {
    "aa": {"r": [...], "U": [...]},
    "ab": {"r": [...], "U": [...]},
    "bb": {"r": [...], "U": [...]}
  }
}
```

Raw potentials sum all primary/secondary/tertiary contributions without splitting. This is the physical interaction representation used for B2 comparisons and hard-core detection in several workflows.

7.5 total_potentials output

```
{
  "species": ["a", "b"],
  "hc_flag_map": {"aa": 0, "ab": 1, "ba": 1, "bb": 1},
  "total_potentials": {
    "aa": {"r": [...], "U": [...]},
    "ab": {"r": [...], "U": [...]},
    "bb": {"r": [...], "U": [...]}
  }
}
```

The total representation includes the HC potential only for pairs whose hard-core flag is 1; it then adds/interpolates the mean-field representation. Note that many structural-analysis routines deliberately use raw_potentials instead when they need the original full interaction.

7.6 total_free_energy result

```
{
  "variables": (rho_a, rho_b, v_a_a, v_a_b, v_b_a, v_b_b),
  "expression": <SymPy expression>,
  "function": <SymPy Lambda>,
  "components": [...],
  "selected_model": "ideal + mean_field + whitebeer"
}
```

The in-memory result contains SymPy objects. free_energy_exporter serializes symbols and expressions into JSON-safe representations. The phase-finder functions reconstruct symbolic thermodynamics and differentiate the free-energy density with respect to rho_i.

7.7 Canonical state result

```
{
  "ensemble": "canonical",
  "species": ["a", "b"],
  "densities": [rho_a, rho_b],
  "vij": [[v_aa, v_ab], [v_ba, v_bb]],
  "chemical_potentials": [mu_a, mu_b],
  "pressure": P,
  "free_energy_density": f
}
```

The thermodynamic relations implemented are mu_i = partial f / partial rho_i and P = -f + sum_i rho_i mu_i. The pair strengths v_ij are recalculated for the requested state through the selected integrated-strength kernel.

7.8 Coexistence result

```
{
  "ensemble": "isochem",
  "species": ["a", "b"],
  "n_phases": 2,
  "rhos_per_phase": [[rho_a_1, rho_b_1], [rho_a_2, rho_b_2]],
  "mu_per_phase": [[...], [...]],
  "pressure_per_phase": [P1, P2],
  "vij_per_phase": [[[...]], [[...]]]
}
```

The solver uses an outer loop because v_ij may itself depend on density through the kernel. Within each outer step a nonlinear root solve enforces the specified intrinsic chemical potentials and equality conditions across phases.

7.9 process_supplied_data output

```
{
  "states": {
    "1": {
      "rdf": {
        "aa": {"x": array([...]), "y": array([...])},
        "ab": {"x": array([...]), "y": array([...])}
      },
      "densities": {"a": 0.10, "b": 0.02},
      "beta": 1
    }
  }
}
```

The returned in-memory object contains NumPy arrays; the optional supplied_data_materialized.json export converts those arrays to lists.

7.10 c_analysis principal outputs

Output	Important keys	Physical content
result_sigma_analysis.json	sigma_opt, sigma_bh, bh_meta, r, u_ref, u_real, g/c/gamma for real/reference representations	Reference hard-core mapping and structural comparisons.
delta_B2_results.json	B2_real, B2_ref, delta_B2, 2_delta_B2	Second-virial difference between real and reference potentials.
result_G_of_r.json	r, G_r_sigma_opt, G_r_real, G_u_r_sigma_opt, G_u_r_real, u_attractive_real	Coupling-parameter / thermodynamic-integration kernel diagnostics.
delta_c_results.json	c_real, c_ref_hard, c_sigma_opt, c_rep_sigma_opt, four delta-c arrays	Direct-correlation differences used to construct interaction corrections.
<prefix>_potential.json	pairs -> r, u_r, sigma	Pair potential representation used/exported by the analysis.

8. Physical Quantities and Their Data Flow

8.1 Pair potential U_ij(r)

The starting interaction is either analytic or tabulated. For each pair, the package may maintain several representations: raw/full U, hard-core reference, and mean-field/soft contribution. Keeping these

representations distinct is essential because different physical routes deliberately integrate different forms.

8.2 Hard-core diameter sigma_ij

The hard-core splitter recognizes explicit hc-like definitions and can infer effective diameters for strongly repulsive analytic or tabulated interactions. The structural/inversion analysis additionally estimates core locations from g(r), performs collective sigma optimization, and evaluates Barker–Henderson diameters. These are different definitions and are exported separately rather than silently conflated.

8.3 Radial distribution function g_ij(r)

The RDF/OZ routines build an N×N matrix of pair correlation functions on a radial grid r_k = k*r_max/(n_points+1), k=1,...,n_points. Closures are supplied pairwise and symmetrized. In inversion, supplied target RDFs are interpolated to this common grid.

Physical interpretation: g_ij(r) measures the probability of finding species j at separation r from species i relative to an uncorrelated fluid at the same bulk densities. The total correlation h_ij(r)=g_ij(r)-1 therefore separates the correlated part from the ideal-background value. In reciprocal space, the same information is encoded by partial static structure factors S_ij(k), which are especially useful for identifying collective fluctuations and are often the experimentally accessible observables. [Theory: Varma, Archer & Scacchi, “A route to the thermodynamics of colloid-polymer mixtures from structural information,” manuscript dated 16 Aug 2026.]

8.4 Direct correlation function c_ij(r) and gamma_ij(r)

The structural solvers maintain c(r) and gamma(r) alongside g(r). The c_analysis workflow compares the direct correlations of real and reference representations and exports negative differences such as -(c_real-c_ref_hard).

The direct correlation function has a special status in cDFT: it is not simply another transform of the RDF. For a homogeneous one-component system, the Ornstein-Zernike (OZ) relation separates the total correlation into a direct part and correlations propagated indirectly through surrounding particles. In Fourier space this yields S(k)=1/[1-rho*c_hat(k)], so an experimentally measured S(k), once density and transform conventions are fixed, can be inverted to obtain c_hat(k). More fundamentally, c^(2) is proportional to the second functional derivative of the excess free energy with respect to density. This is why the DCF is the natural bridge between microscopic structure and the free-energy functional used by cDFT. [Theory: Varma, Archer & Scacchi, 2026 manuscript.]

8.5 Second virial coefficient B2

The implementation uses B2 = -2*pi * integral [exp(-beta U(r)) - 1] r^2 dr. c_analysis exports B2_real, B2_ref, Delta B2, and 2 Delta B2 pairwise. The integrated-strength routine also uses 2 Delta B2 for a uniform kernel when a hard core is detected, unless the kernel label requests the direct meanfield route.

8.6 Integrated pair strength v_ij

The nominal radial expression is v_ij = integral 4*pi*r^2*K_ij(r)*U_ij(r) dr. Current code contains route-dependent behavior: uniform + hard core can switch to 2 Delta B2; a pure meanfield label directly integrates K*U_MF; an RDF/coupling kernel supplies a nonuniform radial quantity; delta_c supplies a direct-correlation-based kernel.

Source-level caution — In vij_radial_kernel, the nonuniform else branch integrates 4*pi*r^2*K(r) without

multiplying by U(r). This matches the current kernel generators, whose returned “value” can already contain the interaction weighting. Do not reinterpret K as a dimensionless g(r) unless you have checked the generator used.

8.7 Free-energy density

Ideal contribution: f_id = sum_i rho_i [ln(rho_i)-1]. Standard mean field: f_mf = (1/2) sum_ij v_ij rho_i rho_j. When hard cores are detected and mode=standard, a White Bear or Rosenfeld hard-core bulk contribution is added. The symbolic expressions are merged after unifying variables by name.

8.8 Chemical potentials and pressure

From the merged symbolic free-energy density f, the canonical evaluator computes mu_i = partial f / partial rho_i and P = -f + sum_i rho_i mu_i. These are evaluated numerically after v_ij is calculated for the current density vector.

8.9 Packing fraction

The binodal example converts density to packing fraction as phi_i = rho_i*(pi/6)*d_i^3 using user-selected effective diameters d_i. These d values are not automatically read from hc_data in the supplied example; they are hard-coded post-processing inputs. A general workflow should decide explicitly whether d_i is the hard-core diagonal sigma, a Barker–Henderson diameter, an optimized reference diameter, or another physically defined size.

8.10 Effective potential inversion

The advanced inversion uses one or more target RDF states. If a pair already has a nonzero model potential, that pair is initially protected from inversion; supplied RDF pairs are initialized from -ln g and marked for inversion. The multistate IBI loop solves OZ for each state, accumulates weighted logarithmic g_pred/g_target corrections, adapts the IBI step factor, and updates the shared pair potential until the maximum RDF deviation meets ibi_tolerance or the iteration limit is reached.

9. Input Preflight Checklist

1. Choose single-character species labels and list all species in a deterministic order.
2. For every physical pair required by the solver, provide either an analytic interaction or a tabulated potential file.
3. Provide a closure for every unique pair used by the RDF/OZ calculation.
4. Use rdf tolerance for new structural/inversion inputs; document any legacy use of tolerence.
5. Ensure every tabulated potential/RDF file has an increasing first column and at least two numeric columns.
6. Run from the input-file directory or use absolute filenames so current Path.cwd()-based loaders resolve files correctly.
7. Parse the input and inspect input_system.json before expensive calculation.
8. Check the hard-core sigma and flag matrices before accepting a free-energy model.
9. For canonical calculations, provide density for every species.
10. For inversion, provide density and beta/temperature metadata for every supplied state.
11. For coexistence, confirm the intrinsic-constraint count does not exceed N_species-1 and set a physically meaningful total_density_bound.

12. Keep beta, length, potential and density units mutually consistent; the package does not perform dimensional-unit conversion.

PART III — Complete Example Catalogue and Reusable Physical Workflows

10. Complete Catalogue of Bundled Examples and How to Use Them

The supplied Examples archive contains 20 executable example_workflow_*.py drivers distributed across 18 workflow folders, together with 21 .in files representing 19 unique input definitions. The catalogue below is organized by the literal folder structure so that a reader can open any example directory and immediately distinguish the active input, the driver, external numerical data, helper/plotting scripts, and shipped reference outputs. The workflow column follows reachable top-level execution rather than merely listing imports.

Folder-role convention — A file named example_input_*.in is the active configuration for the associated driver unless stated otherwise. Files named example_workflow_*.py are executable drivers. Tabulated .txt files are numerical inputs only when referenced by the .in file or driver; plotter_*.py, converter.py, and potential_extractor.py are helper/post-processing scripts and are not automatically executed by the driver. Shipped .png files and output_*.txt files are reference/generated artifacts, not configuration inputs. Folder names are documentation labels rather than API requirements.

10.1 Audited catalogue

Example folder (literal path)	Driver(s)	Input / local support files	Reachable driver workflow	Audit / cleanup note
Radial_distrubtion_function	example_workflow_rdf_radial.py; example_workflow_rdf_planer.py	example_input_rdf_radial.in; example_input_rdf_planer.in; no external data files	radial: parse -> rdf_radial; planar: parse -> construct (Ns,Nz) density field -> rdf_planer	Literal archive spelling is “distrubtion”. The radial driver imports rdf_planer and NumPy without using them; the planar driver has an unused Path import.
Potential_inversion/ One_component_colloid_inversion	example_workflow_potential_invertor.py	example_input_inversion.in; rdf_colloid.txt; helper: potential_extractor.py	parse -> process_supplied_data -> boltzmann_inversion_advance	NumPy, Path, rdf_planer and rdf_radial are imported but unused by the driver.
Potential_inversion/ Binary_colloid_polymer_inversion	example_workflow_potential_invertor.py	example_input_inversion.in; average_g11.txt; average_g12.txt;	parse -> process_supplied_data -> boltzmann_inversi	All three unique pair RDFs are supplied at one state. NumPy,

Example folder (literal path)	Driver(s)	Input / local support files	Reachable driver workflow	Audit / cleanup note
		average_g22.txt; helper: potential_extractor.py	on_advance	Path, rdf_planer and rdf_radial imports are unused.
Structural_correlation_analysis/One_component	example_workflow_c_2_analysis.py	example_input_structural_analysis.in; aa_colloids.txt; plotter_correlation_function.py; plotter_integrated_strength_kernel.py	parse -> process_supplied_data (no-op for this input) -> c_analysis with densities=[0.4]	The tabulated potential is loaded through interactions/filename, not supplied_data. The psd result is not passed to c_analysis; rdf/inversion imports are unused.
Structural_correlation_analysis/Binary_colloid_polymer_mixture	example_workflow_c_2_analysis.py	example_input_structural_analysis.in; aa.txt; ab.txt; bb.txt; two plotters; three shipped PNG reference images	parse -> process_supplied_data (no-op for this input) -> c_analysis with densities=[0.6,0.096]	The three tables are interaction potentials. psd is redundant here and the rdf/inversion imports are unused.
Equation_of_state/One_component_colloid_system	example_workflow_phase_finder_thermodynamic_quantities_automation.py	example_input_phase_finder_isocore.in; aa_colloids_12_6.txt; aa_colloids_24_12.txt	parse -> potential exports -> total_free_energy -> repeated evaluate_canonical_state over rho grid -> eos_rho_data.json	Input currently names aa_colloids.txt, which is not shipped. Select one table by editing the filename or creating that alias. Path import is unused.
Equation_of_state/Binary_colloid_polymer_mixture	example_workflow_phase_finder_thermodynamic_quantities_automation.py	example_input_phase_finder_isocore.in; aa.txt; ab.txt; bb.txt	parse -> potential exports -> total_free_energy -> fixed-x density scan with evaluate_canonical_state -> eos_rho_data_binary.json	The scan uses x_b=0.167. Path import is unused.

Example folder (literal path)	Driver(s)	Input / local support files	Reachable driver workflow	Audit / cleanup note
Phase_coexistence/LJ_fluid	example_workflow_isocore_bulk.py; example_workflow_isochem_bulk.py	example_input_isocore.in; example_input_isochem.in; plotter.py; fig_4.png	parse -> potential exports -> total_free_energy -> coexistence_densities_isocore OR coexistence_densities_isochem	Both drivers import rdf_radial without needing it for the coexistence result. In the isochem driver, the rdf_radial call after exit(0) is unreachable and is excluded from the documented workflow.
Phase_coexistence/Binary_colloid_polymer_mixture	example_workflow_isochem_bulk_automation.py	example_input_isochem.in	parse -> potential exports -> total_free_energy -> mu_a / density-bound scan -> coexistence_densities_isochem -> binodal_data.txt + binodal_plot.png	rdf_radial is imported but unused. The saved rows contain 8 values although the shipped text header names only 4 packing-fraction columns.
Phase_coexistence/Ternary_colloid_polymer_mixture/Colloids_phase_response/EMF	example_workflow_automation_phase_response_colloids.py	example_input_isocore.in; colloids_delta_rho_vs_density_12_24_simulation.txt	parse -> potential exports -> total_free_energy -> scan global rho_c -> coexistence_densities_isocore -> coexistence_density_scan.txt	The simulation text file is shipped as comparison/reference data and is not read by the main driver. Path import is unused.
Phase_coexistence/Ternary_colloid_polymer_mixture/Colloids_phase_response/SMF	example_workflow_automation_phase_response_colloids.py	example_input_isocore.in; colloids_delta_rho_vs_density_12_24_simulation.txt	parse -> potential exports -> total_free_energy -> scan global rho_c -> coexistence_densities_isocore -> coexistence_density_scan.txt	Same driver logic as EMF; the free_energy method and ac secondary strength differ in the input. Reference simulation data are not consumed by the driver.
Phase_coexistence/Ternary_colloid_p	example_workflow_automation_phase_response_poly	example_input_isocore.in; delta_rho_vs_dens	parse -> potential exports -> total_free_energy	The shipped comparison table is not read by the

Example folder (literal path)	Driver(s)	Input / local support files	Reachable driver workflow	Audit / cleanup note
olymer_mixture/ Polymers_phase_response/EMF	mers.py	ity_12_24.txt	-> scan total polymer density, split equally into a/b -> coexistence_densities_isocore -> polymer_coexistence_scan.txt	driver. Path import is unused.
Phase_coexistence / Ternary_colloid_polymer_mixture/ Polymers_phase_response/SMF	example_workflow_automation_phase_response_polymers.py	example_input_isocore.in; delta_rho_vs_density_12_24.txt	parse -> potential exports -> total_free_energy -> scan total polymer density, split equally into a/b -> coexistence_densities_isocore -> polymer_coexistence_scan.txt	Same scan logic as EMF with the SMF input model. Comparison data are shipped but not consumed by the driver.
Inhomogeneous_profile/ Ternary_colloid_polymer_mixture	example_workflow_isocore_planer.py	example_input_isocore.in	parse -> one_d_profile_iterator_box(export_json=True, export_plots=True)	The driver is intentionally short. Path is imported but unused; exit(0) is the final executable statement and no computational code follows it.
Phase_finder	example_workflow_phase_finder_thermodynamic_quantities_automation.py	example_input_phase_finder_isocore.in; three plotter_*.py helpers	parse -> potential exports -> total_free_energy -> valid one-component state evaluation -> attempted 2D phi_a/phi_b grid -> thermo_surface_phi_data.json	The input defines only species a, but the later grid assumes a and b and reads two chemical potentials. That grid is a binary template and is not runnable unchanged with the supplied input. Duplicate NumPy/JSON imports and Path

Example folder (literal path)	Driver(s)	Input / local support files	Reachable driver workflow	Audit / cleanup note
				are unnecessary.
Energy_landscape	example_workflow_automation_free_energy_landscape.py	example_input_isocore.in; converter.py; shipped output_XY_data.txt and output_YZ_data.txt	parse -> potential exports -> total_free_energy -> isocore coexistence -> evaluate_canonical_state on constrained XY/YZ phase pairs -> text surfaces + phase_diagram_combined_new.png	converter.py is not called by the driver. The shipped output_*.txt files are reference/generated artifacts. Path import is unused.
Dispersion_relation	example_workflow_dispersion_relation.py	example_input_dispersion.in; plotter_dispersion_relation.py; plotter_rdf_relation.py	parse -> process_supplied_data (no supplied_data block; result unused) -> dispersion_relation(densities=[1,1], diffusivity=[1,1], supplied_data=None)	Minimal workflow is parse -> dispersion_relation. Path, rdf_planer, rdf_radial, boltzmann_inversion_advance and c_analysis are imported but unused.
Second_virial_coefficient_analysis	example_workflow_virial_analysis.py	example_input_virial_analysis.in; example_input_virial_analysis_calibration.in	parse reference -> second_virial -> parse calibration model -> choose B2_target -> second_virial_scale_calibration	process_supplied_data is a no-op because neither input has supplied_data. The computed B2 is printed and then overwritten by [[-0.125]] before calibration; document this as an explicit target choice. NumPy and Path imports are unused.

The package directory currently spells the RDF example folder as Radial_distrubtion_function. The manual uses the grammatically correct term “Radial Distribution Function” in headings while preserving the literal package path when identifying files.

10.2 Workflow-audit rule: executed, optional, redundant, and unreachable

The rectified example descriptions follow control flow rather than the import list. A package module is included in an "executed workflow" only when the driver actually reaches the corresponding call. This prevents helper imports, experimental remnants, or statements placed after an unconditional exit(0) from being presented as required stages.

- Computational dependency — the returned object is consumed by a later calculation or is the requested final physical result.
- Optional export/diagnostic branch — the call is executed and may write useful files, but its returned object is not consumed by the later state/coexistence calculation in that driver.
- Redundant/no-op stage — the call has no effect for the supplied input or its result is immediately ignored. Such a step is reported for source fidelity but omitted from the cleaned reference workflow.
- Unreachable code — statements after an unconditional top-level exit(0) are not part of the executed example, even when the relevant module is imported near the top of the file.

10.3 Common execution preamble

```
from pathlib import Path
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator

input_file = Path("path/to/system.in").resolve()
scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)

ctx = ExecutionContext(
    input_file=str(input_file),
    scratch_dir=scratch,
    plots_dir=plots,
)

system = super_dictionary_creator(
    ctx,
    export_json=True,
    filename="input_system.json",
    super_key_name="system",
)
```

After this preamble, the workflow branches according to the desired physical quantity. The important reusable object is system: the nested dictionary produced from the input file. In the shipped automation examples, selected density or constraint entries are modified inside a scan while the same symbolic free-energy object is reused. Optional raw/total potential exports should be kept only when those files are needed for inspection or later analysis.

10.4 What is example-specific and what is reusable

Keep as reusable pattern	Replace for your system
ExecutionContext + super_dictionary_creator	literal input filename and local directory
potential preprocessing sequence	potential parameters or tabulated files
free-energy construction before a scan	density/composition scan points
evaluate_canonical_state / coexistence call	which density or chemical potential is varied
supplied-data materialization	simulation filenames and state metadata
export structure	output filenames, plotting choices, resolution

11. Radial Distribution Function Examples — 3D Radial and 2D Planar

The folder Radial_distrubtion_function contains both direct RDF drivers and both input files. No separate numerical data file is required. The two drivers demonstrate different density contracts: a one-dimensional species-density vector for isotropic radial RDFs and a species-by-plane density field for planar correlations. The literal folder spelling is preserved here because it is the actual archive path.

11.1 Three-dimensional/isotropic radial RDF

Bundled source: Radial_distrubtion_function/example_workflow_rdf_radial.py. The example defines one species and one Gaussian-soft interaction, then evaluates the isotropic RDF at density 0.3.

```
task = profile_planer
ensemble = isocore
species = a
interactions primary: aa: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.01
rdf closure aa = hnc
rdf alpha_max =  0.001
rdf max_iteration  = 100000
rdf tolerance = 0.00001
rdf beta = 1.0
rdf r_max  =  10.0
rdf n_points =  400
```

The example passes densities = [0.3]. For Ns species, rdf_radial expects a density vector with one value per species in the same order as the species definition.

```
densities = [0.3]
result = rdf_radial(
    ctx=ctx,
    rdf_config=system,
    densities=densities,
    supplied_data=None,
    export=True,
    plot=True,
    filename_prefix="rdf",
)
```

11.2 Radial RDF return contract

rdf_radial returns a dictionary keyed by ordered species-pair tuples, for example ("a", "a") or ("a", "b"). Each pair contains a common radial grid and correlation arrays.

```
result[("a", "a")] = {
    "r":       ndarray,   # shape (Nr,)
    "g_r":     ndarray,   # shape (Nr,)
    "c_r":     ndarray,   # shape (Nr,)
    "gamma_r": ndarray,   # shape (Nr,)
}
```

When export=True, the current implementation writes <prefix>_rdf.json to the scratch directory. The JSON contains metadata and pair entries with r, g_r, h_r, c_r, and gamma_r. When plot=True it also exports matrix plots for g(r), c(r), and gamma(r), with the densities embedded in the filenames.

11.3 Two-dimensional/planar RDF

Bundled source: Radial_distrubtion_function/example_workflow_rdf_planer.py. This example uses three species and constructs a homogeneous density field over Nz=50 planes.

```
task = rdf_planer
species = a, b, c
interactions primary: aa: type = gs,   sigma = 1.414,  cutoff = 3.5,     epsilon = 2.0
interactions primary: ab: type = gs,   sigma = 1.414,  cutoff = 3.5,     epsilon = 2.5
interactions primary: ac: type = ma,  sigma = 1.0,    cutoff = 5.0,    epsilon = 0.3104, m = 24, n = 12
interactions primary: bb: type = gs,   sigma = 1.414,  cutoff = 3.5,     epsilon = 2.0
```

```
interactions primary: cc: type = lj,   sigma = 1.0,    cutoff = 1.1224,  epsilon = 1.0
interactions secondary: aa: type = ghc,  sigma = 1.02, cutoff = 3.2, epsilon = 1.0
interactions secondary: bb: type = ghc,  sigma = 1.0, cutoff = 3.2, epsilon = 1.0
rdf closure aa = hnc
rdf closure ab = hnc
rdf closure ac = py
rdf closure bb = hnc
rdf closure bc = py
rdf closure cc = py
rdf alpha_max =  0.1
rdf max_iteration  = 100000
rdf tolerance = 0.01
rdf beta = 1.0
planer_rdf space_properties: dimension = 2
planer_rdf space_properties: space_confinement = abox
planer_rdf box_properties: box_length = 5, 5, 0
planer_rdf box_properties: box_points = 50, 50, 0
```

The crucial difference is the density array. The workflow starts from one bulk density per species and replicates it over the planar coordinate:

```
Nz = 50
rho_bulk = np.array([0.4, 0.0001, 0.09])      # shape (Ns,)
rho_z = np.repeat(rho_bulk[:, None], Nz, axis=1)  # shape (Ns, Nz)

result = rdf_planer(
    ctx,
    rdf_config=system,
    densities=rho_z,
    supplied_data=None,
    export=True,
    plot=True,
    filename_prefix="rdf_2d",
)
```

11.4 Planar RDF array shapes

Quantity	Shape	Interpretation
densities	(Ns, Nz)	density of species i on plane z
u_r	(Ns, Ns, Nz, Nz, Nr)	pair potential resolved by two planes and radial separation
g_r	(Ns, Ns, Nz, Nz, Nr)	planar pair distribution
h_r	(Ns, Ns, Nz, Nz, Nr)	total correlation
c_r	(Ns, Ns, Nz, Nz, Nr)	direct correlation
gamma_r	(Ns, Ns, Nz, Nz, Nr)	indirect correlation

The planar solver exports <prefix>.json with metadata (species, Nz, Nr, beta) and pair-resolved arrays. It also calls its correlation plotting routine, which produces diagonal radial correlations and plane-plane diagnostics.

11.5 Choosing between radial and planar RDF examples

Question	Use
Homogeneous isotropic bulk fluid?	rdf_radial
Need g_ij(r) for inversion or c-analysis?	rdf_radial
Density varies across planes or confinement is planar?	rdf_planer
Need a 2D correlation field between z_i and z_j planes?	rdf_planer

12. Effective-Potential Inversion Examples — One Component, Binary, and Multistate

The inversion examples start from supplied RDF files rather than from a complete interaction block. process_supplied_data materializes the file-backed RDF leaves, and boltzmann_inversion_advance reconstructs effective pair potentials using Boltzmann/IBI updates coupled to the OZ/closure solver.

12.1 One-component inversion input

```
species = a
rdf closure aa = py
rdf alpha_max =  0.1
rdf max_iteration  = 40000
rdf max_iteration_ibi = 10000
rdf alpha_ibi_max  = 0.1
rdf tolerance = 0.00001
rdf ibi_tolerance  = 0.0001
rdf beta = 1.0
rdf r_max  =  20.0
rdf n_points =  2000
supplied_data states 1 rdf aa: filename = rdf_colloid.txt
supplied_data states 1 densities a = 0.4
supplied_data states 1 beta = 1.0
```

The supplied RDF file is a two-column numerical table. The first column is r and the second is g_aa(r). Comment lines are accepted by NumPy loadtxt.

```
# Averaged RDF
# r    g(r)
0.017500 0.0000000000
0.052500 0.0000000000
0.087500 0.0000000000
0.122500 0.0000000000
0.157500 0.0000000000
0.192500 0.0000000000
```

12.2 Portable one-component inversion workflow

```
supplied_data = process_supplied_data(
    ctx=ctx,
    config=system,
    export_json=True,
    export_plot=True,
)

result = boltzmann_inversion_advance(
    ctx=ctx,
    rdf_config=system,
    supplied_data=supplied_data,
    filename_prefix="effective",
    export_plot=True,
    export_json=True,
)
```

The state metadata in the input couples the supplied RDF to density and beta. The inversion routine maps the supplied RDF onto its internal radial grid, constructs the initial effective potential, solves OZ with the requested closure, compares the calculated and target RDFs, and iteratively corrects the potential until the IBI tolerance or iteration limit is reached.

Driver cleanup note — Both inversion drivers import NumPy, Path, rdf_planer, and rdf_radial even though those names are never used. The effective computational workflow is therefore exactly parse input -> materialize supplied RDF files -> boltzmann_inversion_advance. potential_extractor.py is a separate helper and is not called by the inversion driver.

12.3 Binary inversion input

```
species = a, b
rdf closure aa = hnc
rdf closure ab = py
rdf closure bb = py
rdf alpha_max =  0.1
rdf max_iteration  = 40000
rdf max_iteration_ibi = 10000
rdf alpha_ibi_max  = 0.1
rdf tolerance = 0.00001
rdf ibi_tolerance  = 0.0001
rdf beta = 1.0
rdf r_max  =  20.0
rdf n_points =  2000
supplied_data states 1 rdf aa: filename = average_g11.txt
supplied_data states 1 rdf ab: filename = average_g12.txt
supplied_data states 1 rdf bb: filename = average_g22.txt
supplied_data states 1 densities a = 0.109860935
supplied_data states 1 densities b = 0.022041065
supplied_data states 1 beta = 1.0
```

The binary example supplies all three unique pair RDFs (aa, ab, bb) at one thermodynamic state. The same machinery also permits partial inversion: known interactions may remain model-defined while only selected pairs are constrained by supplied RDFs. The detailed mixed-data rules and inversion masks are documented in Part VI.

12.4 Turning the bundled single-state input into a multistate inversion

```
supplied_data states 1 rdf aa: filename = g_aa_state1.txt
supplied_data states 1 densities a = 0.20
supplied_data states 1 beta = 1.0

supplied_data states 2 rdf aa: filename = g_aa_state2.txt
supplied_data states 2 densities a = 0.40
supplied_data states 2 beta = 1.0
```

Multiple states constrain one shared effective interaction. This is not a loop around independent inversions: the advanced inverter keeps a common potential and uses structural information from all supplied state points during the update process.

12.5 Data-contract checkpoints before inversion

- Every supplied state must provide the densities required to interpret that RDF state.
- RDF leaves materialized by process_supplied_data contain x/y arrays; the inversion routine explicitly accepts these generic arrays as well as r/g leaves.
- Pair names must match the species labels and the unique pair ordering used by the solver.
- r_max and n_points define the inversion grid and therefore control interpolation of supplied data.
- alpha_ibi_max, max_iteration_ibi, and ibi_tolerance control the outer inversion update, while alpha_max, max_iteration, and tolerance control each underlying OZ solve.

13. Structural-Correlation Analysis Examples — g(r), c(r), gamma(r), and Pair Diagnostics

The structural-correlation examples use c_analysis on model-defined or tabulated pair potentials. Unlike inversion, c_analysis does not require a target RDF when the interaction is already known; it computes the structural correlations at the supplied densities and exports pair-resolved diagnostics.

13.1 One-component structural analysis

```
species = a
```

```
interactions primary: aa: filename = aa_colloids.txt
rdf closure aa = py
rdf alpha_max =  0.4
rdf max_iteration  = 400000
rdf tolerance = 0.000001
rdf beta = 1.0
rdf r_max  =  20.0
rdf n_points =  2000
```

The interaction is supplied through a two-column potential file rather than through an analytic type. The example data begin as:

```
# r    u(r)
  9.99500250e-03   1.84206807e+01
  1.99900050e-02   1.84206807e+01
  2.99850075e-02   1.84206807e+01
  3.99800100e-02   1.84206807e+01
  4.99750125e-02   1.84206807e+01
  5.99700150e-02   1.84206807e+01
densities = np.array([0.4])
c_analysis(
    ctx=ctx,
    rdf_config=system,
    supplied_data=None,
    densities=densities,
    filename_prefix="structure",
    export_plot=True,
    export_json=True,
)
```

13.2 Binary structural analysis

```
species = a, b
interactions primary: aa: filename = aa.txt
interactions primary: ab: filename = ab.txt
interactions primary: bb: filename = bb.txt
rdf closure aa = hnc
rdf closure bb = py
rdf closure ab = py
rdf alpha_max =  0.1
rdf max_iteration  = 400000
rdf tolerance = 0.00001
rdf beta = 1.0
rdf r_max  =  10.0
rdf n_points =  2000
```

The binary case reads aa.txt, ab.txt, and bb.txt as three tabulated pair interactions, then calls the same c_analysis entry point with a two-element density vector. This illustrates a useful principle: the same structural-analysis API can operate on analytical interactions, file-backed interactions, or mixtures of the two.

Driver cleanup note — In both structural-correlation folders, process_supplied_data is called although the input contains no supplied_data block; the returned value is not passed to c_analysis, which is called with supplied_data=None. The tabulated aa/ab/bb files are loaded through interactions ... filename instead. The cleaned workflow is therefore parse input -> c_analysis(densities=...). The imported rdf_planer, rdf_radial, and boltzmann_inversion_advance modules are not part of the executed analysis.

13.3 What c_analysis adds beyond a plain RDF calculation

- Constructs and compares structural/correlation quantities for each species pair.
- Provides direct-correlation c_ij(r) and indirect-correlation gamma_ij(r) information needed by DCF-based integrated-strength routes.
- Performs hard-core/reference diagnostics used by later analysis of effective interactions.

- Exports numerical JSON and plots under the requested filename prefix when enabled.

13.4 Structural hierarchy: g(r), h(r), S(k), and c^(2)(r)

For a homogeneous one-component fluid, the structural hierarchy begins with g(r). Defining h(r)=g(r)-1 removes the uncorrelated background. The static structure factor then obeys S(k)=1+rho*h_hat(k), where the hat denotes the Fourier transform. The Ornstein-Zernike equation relates h to the pair direct correlation c^(2), and in Fourier space gives h_hat(k)=c_hat(k)/[1-rho*c_hat(k)]. Combining the two equations gives S(k)=1/[1-rho*c_hat(k)]. Thus, in the ideal mathematical limit of complete, noise-free data, g(r), S(k), and c^(2)(r) contain equivalent pair-structure information, but each representation emphasizes different physics and has different numerical sensitivities. [Theory: Varma, Archer & Scacchi, 2026 manuscript.]

h(r) = g(r) - 1; S(k) = 1 + rho h_hat(k); S(k) = 1 / [1 - rho c_hat(k)]

Practical implication for c_analysis: do not judge the quality of a reference system from g(r) alone. Two representations can look very similar in real space while differing more clearly in S(k), particularly at small k, or in c^(2)(r), where the non-reference tail can be isolated directly.

13.5 Multicomponent OZ coupling: why cross correlations matter

For an n-component mixture the correlation functions are matrices. Each h_ij(r) satisfies an OZ equation that contains a sum over every intermediate species k, so the aa, ab, bb, … correlations are not independent one-component problems. The partial structure factors satisfy S_ij(k)=delta_ij+sqrt(rho_i*rho_j) h_hat_ij(k). Consequently, a cross term such as S_ab(k) is a genuine collective observable: it measures correlated concentration/density fluctuations between different species and contributes to the thermodynamic response of the mixture. This is the theoretical reason the package stores the full N×N correlation matrix and why a binary structural workflow must preserve pair ordering and density metadata. [Theory: Varma, Archer & Scacchi, 2026 manuscript.]

h_ij(r) = c_ij(r) + Σ_k rho_k ∫ c_ik(|r-r′|) h_kj(r′) dr′

13.6 Closure relations are physical approximations, not only numerical switches

The OZ equations do not determine h and c uniquely; a closure relation is required. The supplied structural paper uses HNC for soft interactions and PY for hard-core or strongly excluded-volume interactions in the inversion/reference workflow. The HNC closure retains the exponential coupling between the potential and the indirect correlation gamma=h-c and is commonly suited to soft interactions. The PY closure treats the core condition efficiently and is used here for hard-core reference systems. Therefore a pairwise closure matrix in the input should be interpreted as a model choice tied to the physical character of each interaction, not merely as a convergence option. A custom closure added through Part VI must satisfy the same c/h/g conventions as the OZ iteration. [Theory: Varma, Archer & Scacchi, 2026 manuscript.]

13.7 Reference-system subtraction and the thermodynamic content of Delta c^(2)

When an interaction contains a repulsive core plus a softer tail, the structural route separates a reference system from the full system. The reference DCF c_ref^(2)(r) encodes the excluded-volume/reference contribution. The difference Delta c_ij^(2)(r)=c_ij^(2)(r)-c_ij,ref^(2)(r) isolates the correlation contribution not already represented by the reference free energy. In the DCF route used by the package, integrating this difference produces the pair coefficient A_ij that enters the quadratic density correction to the excess free energy. For purely soft polymer-polymer interactions no hard reference is required, so the full soft

correlation can be treated directly instead of subtracting a reference DCF. [Theory: Varma, Archer & Scacchi, 2026 manuscript.]

A_ij^(DCF) = -k_B T ∫ Delta c_ij^(2)(r) dr (with the appropriate radial measure in the implemented geometry)

13.8 Long-wavelength structure and proximity to phase separation

The small-k region of S_ij(k) corresponds to long-wavelength collective fluctuations. In the binary colloid-polymer analysis of the supplied structural paper, S_cp(k) and S_cc(k) are more sensitive to the reference-system choice than S_pp(k), especially at small k; this sensitivity reflects how colloid-colloid attraction modifies collective fluctuations and therefore phase behavior. A structural plot that agrees around the first real-space peak but disagrees at small k can therefore still imply a significant thermodynamic difference. For phase-stability work, the low-k region should be inspected explicitly rather than treating S(k) only as a transformed visualization of g(r). [Theory: Varma, Archer & Scacchi, 2026 manuscript.]

13.9 Experimental, simulation, and inversion routes to structural input

The theory accommodates two common data routes. Scattering experiments often provide S(k) directly; simulations and particle-resolved imaging more naturally provide g(r). The OZ relation links these representations, but finite windows, noise, discretization, and transform truncation make the numerical conversion nontrivial. The supplied structural paper therefore treats inverse Boltzmann iteration primarily as an intermediate tool for constructing the repulsive reference and estimating an effective particle diameter, rather than regarding the reconstructed finite-density potential as the final thermodynamic object. This distinction is important because effective inverted potentials can retain state-point dependence at finite density. [Theory: Varma, Archer & Scacchi, 2026 manuscript.]

14. Equation-of-State Examples — One Component and Binary Tabulated Interactions

The two Equation_of_state folders demonstrate the same reusable thermodynamic pattern: parse the input, optionally export the hard-core/mean-field/raw/total potential representations, construct the symbolic free energy, then update only the density constraints inside a scan and call evaluate_canonical_state. In the shipped drivers, raw_data and total_data are produced for export/inspection but are not passed into the later state loop; the state calculation consumes system and free_energy.

14.1 One-component EOS input

```
species = a
interactions primary: aa: filename = aa_colloids.txt
rdf closure aa = py
rdf alpha_max =  0.01
rdf r_max = 5.0
rdf max_iteration  = 200000
rdf tolerence = 0.000001
rdf n_points =  2000
rdf beta = 1.0
free_energy mode = standard
free_energy hs_functional = whitebeer
free_energy method = smf
free_energy integrated_strength_kernel = rdf
free_energy supplied_data = no
solution extrinsic_constraints: density: a = 0.15
```

The bundled folder contains aa_colloids_12_6.txt and aa_colloids_24_12.txt. The input file refers generically to aa_colloids.txt, so a portable run should explicitly point to the chosen table or copy/rename the desired dataset. A tabulated potential has columns r and u(r), for example:

```
# r    u(r)
  9.99500250e-03   1.84206807e+06
  1.99900050e-02   1.84206807e+06
  2.99850075e-02   1.84206807e+06
  3.99800100e-02   1.84206807e+06
  4.99750125e-02   1.84206807e+06
  5.99700150e-02   1.84206807e+06
```

14.2 EOS preprocessing sequence

```
hc_data  = hard_core_potentials(ctx, system, 5000, "hc.json", True)
mf_data  = meanfield_potentials(ctx, system, 5000, "mf.json", True)
raw_data = raw_potentials(ctx, system, 5000, "raw.json", True)
total_data = total_potentials(
    ctx, hc_source=hc_data, mf_source=mf_data,
    file_name_prefix="total.json", export_files=True,
)

free_energy = total_free_energy(
    ctx=ctx,
    hc_data=hc_data,
    system_config=system,
    export_json=True,
    filenames={
        "hard_core": "fe_hc.json",
        "mean_field": "fe_mf.json",
        "ideal": "fe_id.json",
        "hybrid": "fe_hybrid.json",
    },
)
```

14.3 One-component density scan

```
results = []
for rho in rho_values:
    system["system"]["solution"]["extrinsic_constraints"]["density"]["a"] = float(rho)
    state = evaluate_canonical_state(
        ctx=ctx,
        config_dict=system,
        fe_res=free_energy,
        supplied_data=None,
        export=False,
        verbose=False,
    )
    results.append({
        "rho": float(rho),
        "vij": state["vij"],
        "mu": state["chemical_potentials"][0],
        "pressure": state["pressure"],
    })
```

The example writes eos_rho_data.json. Each state retains the density, integrated interaction matrix/vector, chemical potential, and pressure. This separation is important: the free-energy object is not rebuilt for every density; only the density-dependent state evaluation is repeated.

14.4 Binary EOS at fixed composition

```
species = a, b
interactions primary: aa: filename  = aa.txt
interactions primary: ab: filename  = ab.txt
interactions primary: bb: filename  = bb.txt
rdf closure aa = hnc
```

```
rdf closure bb = py
rdf closure ab = py
rdf alpha_max =  0.01
rdf r_max = 3.5
rdf max_iteration  = 200000
rdf tolerence = 0.000001
rdf n_points =  2000
rdf beta = 1.0
free_energy mode = standard
free_energy hs_functional = whitebeer
free_energy method = smf
free_energy integrated_strength_kernel = rdf
free_energy supplied_data = no
solution extrinsic_constraints: density: a = 0.15
solution extrinsic_constraints: density: b = 0.
```

The script scans total density rho at fixed mole fraction x_b=0.167 and computes rho_a=rho(1-x_b), rho_b=rho x_b before calling evaluate_canonical_state.

```
x_b = 0.167
for rho_total in rho_values:
    rho_a = rho_total * (1.0 - x_b)
    rho_b = rho_total * x_b
    density_block = system["system"]["solution"]["extrinsic_constraints"]["density"]
    density_block["a"] = float(rho_a)
    density_block["b"] = float(rho_b)
    state = evaluate_canonical_state(...)
```

14.5 Selecting the 12–6 or 24–12 one-component tabulated interaction

The current archive no longer contains separate Two_components_Eos directories. Instead, Equation_of_state/One_component_colloid_system ships two alternative tables, aa_colloids_12_6.txt and aa_colloids_24_12.txt, beside a single EOS driver. The associated input currently requests filename = aa_colloids.txt, which is not present in the folder. Before running the example, choose the desired dataset and either change the input filename to that exact file or create an aa_colloids.txt copy/alias. The thermodynamic scan code itself is unchanged by this choice.

> Current-folder consistency note — The binary EOS folder contains one tabulated aa/ab/bb data set and one driver. The 12–6 versus 24–12 choice now appears only in the one-component EOS folder through the two alternative aa tables described above. Do not look for the obsolete Two_components_Eos paths when using this archive.

15. Phase-Coexistence Examples — LJ Fluid and Binary Mixture

The coexistence folders reuse the thermodynamic construction used by the EOS examples, but the final state call is replaced by coexistence_densities_isocore or coexistence_densities_isochem. The shipped scripts also generate several potential JSON representations for inspection; these exports should not be confused with the minimal dependency chain leading to the coexistence solve.

15.1 One-component LJ coexistence in the isocore formulation

```
ensemble = isocore
species = a
interactions primary: aa: type = lj,   sigma = 1.0,    cutoff = 3.5,  epsilon = 0.8
rdf closure aa = py
rdf alpha_max =  0.01
rdf max_iteration  = 600000
rdf tolerence = 0.00001
rdf beta = 1.0
rdf r_max  =  20.0
rdf n_points =  2000
free_energy ensemble = isocore
```

```
free_energy mode = standard
free_energy method = smf
free_energy integrated_strength_kernel = rdf
free_energy hs_functional = whitebeer
free_energy supplied_data = no
solution intrinsic_constraints: species_fraction: a = 0.26
solution extrinsic_constraints: number_of_phases = 2
solution extrinsic_constraints: heterogeneous_pair = None
solution extrinsic_constraints: total_density_bound = 2.0
value = coexistence_densities_isocore(
    ctx=ctx,
    config_dict=system,
    fe_res=free_energy,
    supplied_data=None,
    max_outer_iters=10,
    tol_outer=1e-3,
    tol_solver=1e-8,
    verbose=True,
)
```

The result contains rhos_per_phase, fractions, and thermodynamic information associated with the converged phases. The isocore input constrains composition/density information and lets the coexistence solver determine the phase split consistent with equality conditions.

15.2 One-component LJ coexistence in the isochemical formulation

```
ensemble = isochem
species = a
interactions primary: aa: type = lj,   sigma = 1.0,    cutoff = 3.5,  epsilon =1.0
rdf closure aa = py
rdf alpha_max =  0.01
rdf max_iteration  = 200000
rdf tolerence = 0.00001
rdf beta = 1.0
rdf r_max  =  10.0
rdf n_points =  200
free_energy ensemble = isochem
free_energy mode = standard
free_energy method = smf
free_energy integrated_strength_kernel = delta_c
free_energy supplied_data = no
solution extrinsic_constraints: number_of_phases = 2
solution extrinsic_constraints: heterogeneous_pair = None
solution extrinsic_constraints: total_density_bound = 5.0
solution intrinsic_constraints: chemical_potential: None
```

This example changes the ensemble and intensive constraint to chemical potential while reusing the same free-energy construction. The shipped driver terminates immediately after printing the coexistence result. Its later rdf_radial call is placed after exit(0), is unreachable, and is therefore removed from the documented execution path; the rdf_radial import is likewise unnecessary for this coexistence run.

15.3 Binary isochemical binodal scan

```
species = a, b
interactions primary: aa: type = gs,  sigma = 1.0,   cutoff = 3.5,     epsilon = 2.0
interactions primary: ab: type = lj,  sigma = 1.0,   cutoff = 1.1224,    epsilon = 1.0
interactions primary: bb: type = mie, sigma = 1.0,  m = 24, n= 12,   cutoff =2.5,    epsilon = 0.1
rdf closure aa = py
rdf closure ab = py
rdf closure bb = py
rdf alpha_max =  0.1
rdf max_iteration  = 600000
rdf tolerence = 0.00001
rdf beta = 1.0
rdf r_max  =  10.0
rdf n_points =  2000
```

```
free_energy ensemble = isochem
free_energy mode = standard
free_energy method = smf
free_energy hs_functional = whitebeer
free_energy integrated_strength_kernel = uniform
free_energy supplied_data = no
solution extrinsic_constraints: number_of_phases = 2
solution extrinsic_constraints: heterogeneous_pair = None
solution extrinsic_constraints: total_density_bound = 10
solution intrinsic_constraints: chemical_potential: a = 14
```

The binary automation scans chemical potential mu_a from 7 to 10 and simultaneously increases total_density_bound. For each successful two-phase solution it extracts both phase densities, converts them to packing fractions using chosen effective diameters, and writes the paired binodal points. The imported rdf_radial module is not used by this driver and is omitted from the cleaned workflow.

```
for mu_a, rho_bound in zip(mu_values, density_bounds):
    trial = copy.deepcopy(system)
    trial["system"]["solution"]["intrinsic_constraints"]["chemical_potential"]["a"] = float(mu_a)
    trial["system"]["solution"]["extrinsic_constraints"]["total_density_bound"] = float(rho_bound)
    coexist = coexistence_densities_isochem(...)
```

The example writes binodal_data.txt and binodal_plot.png. Note that the row stored by the script contains eight numerical values (four packing fractions followed by four densities), whereas its np.savetxt header names only the first four packing-fraction columns. The manual treats the actual row layout as the source of truth and recommends extending the header if this example is used for production data.

15.4 Intrinsic equilibrium conditions: what makes phases to coexist

Two phases are in thermodynamic coexistence only when there is no driving force for transfer of any freely exchanging species and no mechanical driving force for one phase to expand at the expense of the other. For a mixture, this is expressed by equality of every relevant chemical potential and equality of pressure across phases. These Gibbs conditions are intrinsic to equilibrium: they characterize coexistence itself and are independent of how the sample was prepared. In a two-phase, N-species problem they provide N chemical-potential equalities plus one pressure equality [Theory: Varma & Scacchi, Phys. Rev. E 113, 065414 (2026).]. In the isochemical approach, the unknown variables are the densities of the different species in each coexisting phase. To determine these variables uniquely, the number of independent equations must equal the number of unknowns. However, the equilibrium equations obtained from the Gibbs coexistence criteria alone are fewer than the total number of unknown variables. Therefore, additional constraints must be introduced to close the system of equations. These constraints are specified externally, typically by prescribing or estimating the expected form of the phase distribution or phase composition. These are called intrinsic constraints since they closes the number of equations. They provide the additional equations required to make the problem well posed and allow the Python solver to determine the coexistence solution.

mu_i^(I) = mu_i^(II) for every exchanging species i; P^(I) = P^(II)

In case of Gibbs ensemble or the semi grand canonical ensemble which is implemented through the isocore input and workflow, the Gibbs equalities alone do not determine all phase densities in a multicomponent canonical problem. The missing information comes from global constraints imposed by the ensemble or sample preparation. For fixed total particle numbers, each species obeys a material balance. For two phases with phase-I volume fraction V_f, the imposed average density satisfies V_f rho_i^(I)+(1-V_f) rho_i^(II)=rho_bar_i. The phase fraction is therefore an additional unknown, but it also closes the species-wise conservation equations. These global balances are what this manual calls intrinsic constraints for the gibbs ensemble or ensemble constraints. [Theory: Varma & Scacchi, 2026.]

$V_f \rho_i^{(I)} + (1 - V_f) \rho_i^{(II)} = \bar{\rho}_i = N_i / V$

15.5 Extrinsic ensemble constraints: selecting a point on the coexistence manifold

Extrinsic constraints are provided to help the python solver to find the solution. Such, heterogeneous pairs and what are the total upper density bound of the system in which the solution finder should search for the solution etc. Some Extrinsic constraints must be applied cautiously and should be checked multiple times such as, number of phases present in the system.

15.6 Constraint counting and why the phase fraction appears

For two phases and N species, the phase compositions contain 2N density unknowns. Gibbs equilibrium supplies N+1 equations, leaving N-1 degrees of freedom. In the canonical formulation, introducing V_f adds one unknown while the N material-balance equations add N constraints, producing 2N+1 equations for 2N+1 unknowns. For q phases, the isocore solver generalizes this logic: it solves q density/composition blocks plus q-1 independent phase fractions, while enforcing N(q-1) chemical-potential equalities, q-1 pressure equalities, and N species-conservation equations. The counts match exactly: qN+(q-1) unknowns and qN+(q-1) equations.

15.7 Package namespaces versus thermodynamic terminology

The code uses the names `intrinsic_constraints` and `extrinsic_constraints`, but users should not assume that these keys are a literal transcription of the thermodynamic classification. The current implementation mixes thermodynamic targets, global composition data, and algorithmic controls across these namespaces. The safest interpretation is to read the key according to the selected solver (`isocore` or `isochem`) rather than according to its English label alone.

15.8 Isocore coexistence: canonical material balance in the actual solver

The `coexistence_densities_isocore` residual in the audited source implements the canonical logic explicitly. For each phase it reconstructs all species densities from a reduced total density and composition variables; it evaluates mu_i and P from the free energy; it enforces equality of chemical potentials and pressure between phases; and it adds one material-balance equation per species using the solved phase fractions. This is the package path that most directly mirrors the canonical coexistence construction in Varma & Scacchi (2026). The quantity called `species_fraction` in the input is used by the code as the fixed species-density vector pvec, so users should supply values with that operational meaning.

15.9 Isochemical coexistence: prescribed intensives and phase endpoints

The isochemical solver removes the canonical phase-fraction variables and instead uses intensive constraints. If a species has a prescribed chemical potential, each phase is required to match that target. If it is not prescribed, the solver imposes equality of that species chemical potential between adjacent phases. Pressure is treated in the same way: either every phase matches a supplied target pressure or the phase pressures are equated. This formulation is useful for tracing coexistence endpoints as an intensive variable is scanned, which is exactly what the binary binodal automation does when it changes mu_a. Because no overall composition is imposed, a phase fraction is not determined by this solver; it would be obtained later from a lever-rule/material-balance calculation if a specific global composition were supplied.

15.10 Reduced-density variables, positivity, and disjoint phases

Directly solving for many non-negative species densities can be numerically fragile. The phase-separation paper introduces a nested reduced-density representation in which each phase is described by a total density rho_t and composition variables x_i in (0,1). This parameterization guarantees non-negative component densities and transforms composition constraints into bounded variables. The package coexistence solvers use the same style of reduced representation, generate trial points in reduced space, convert them to physical densities, evaluate the thermodynamic residuals, and iterate until a disjoint multiphase solution is found. This explains why `total_density_bound` controls the search space without being a thermodynamic coexistence law. [Theory: Varma & Scacchi, 2026.]

15.11 Coexistence, criticality, and the limits of homogeneous structural input

Near a binodal or critical region, obtaining homogeneous structural correlations becomes harder and integral-equation closures may fail to converge. The structural-correlation paper therefore uses a virial/second-virial estimate as an initial coexistence guide and then updates the integrated strength using the thermodynamic-integration or DCF route where possible. It also notes that smooth density dependence of the integrated strength can be interpolated or extrapolated into regions where direct homogeneous structural data are unavailable. These steps are numerical/physical continuation strategies, not additional Gibbs conditions. The predicted binodal should also be interpreted within the fluid-fluid, homogeneous-fluid framework; at sufficiently high colloid density, competing fluid-solid transitions may lie outside the model used for that coexistence calculation. [Theory: Varma, Archer & Scacchi, 2026 manuscript.]

Solver/input item	**What the current code does**	**Thermodynamic interpretation**
isocore: `intrinsic_constraints: species_fraction`	Loads a per-species vector pvec and enforces phase-fraction-weighted mass balance	Fixed global/average densities; an extrinsic canonical constraint
isochem: `intrinsic_constraints: chemical_potential`	Pins selected mu_i to targets in every phase; unpinned mu_i are equated between phases	Prescribed intensive control variable plus Gibbs equality for the rest
isochem: optional `pressure` intrinsic constraint	Pins each phase pressure to a target; otherwise pressures are equated	Prescribed mechanical intensive variable
`extrinsic_constraints: number_of_phases`	Selects how many phase blocks are solved	Problem definition
`extrinsic_constraints: total_density_bound`	Sets random-search/initial-density range	Numerical control, not a physical coexistence condition
`extrinsic_constraints: heterogeneous_pair`	Used to reject/label solutions based on phase enrichment	Solution-selection aid

16. Ternary Phase-Response and Free-Energy-Landscape Examples

Folder consistency — The ternary coexistence catalogue is split first by the quantity scanned (Colloids_phase_response or Polymers_phase_response) and then by the free-energy model (EMF or SMF). Each leaf folder contains one example_input_isocore.in and one automation driver. The accompanying

simulation/comparison .txt file is not read by the main driver; it is reference data for comparison or plotting. Energy_landscape is a separate top-level example folder and uses its own ternary isocore input.

The ternary examples use species a, b, and c to study how a two-phase solution changes as either the colloid-like component c or the total polymer-like density a+b is varied. Separate EMF and SMF variants allow the mean-field treatment to be compared without changing the outer coexistence scan.

16.1 EMF versus SMF input variants

The EMF and SMF configurations are structurally identical. The main deliberate differences are free_energy method = emf versus smf and the secondary ac Gaussian strength (-0.25 in the EMF snapshot versus -0.33 in the SMF snapshot).

Variant	free_energy method	secondary ac epsilon	Scan script
EMF	emf	-0.25	colloid-response or polymer-response
SMF	smf	-0.33	colloid-response or polymer-response

16.2 Colloid-density response

The colloid-response scripts scan c from 0.03 to 0.4 in ten points. At each point they update solution.intrinsic_constraints.species_fraction.c, run coexistence_densities_isocore, and append the two phase compositions plus fractions to coexistence_density_scan.txt.

```
rho_c_values = np.linspace(0.03, 0.4, 10)
for rho_c in rho_c_values:
    system["system"]["solution"]["intrinsic_constraints"]["species_fraction"]["c"] = float(rho_c)
    coexist = coexistence_densities_isocore(...)
```

16.3 Polymer-density response

The polymer-response scripts scan total polymer density from 0.54 to 0.8. The total is split equally, rho_a=rho_b=0.5 rho_polymer,total, and both intrinsic species fractions are updated before solving coexistence. Output rows include the input total, the two input polymer densities, both three-component phase-density vectors, and the two phase fractions.

16.4 Free-energy landscape workflow

The Energy_landscape example first solves a ternary isocore coexistence problem, then uses the returned phase densities and phase fraction to construct constrained two-phase free-energy surfaces. For a trial composition in phase 1, the corresponding phase-2 composition is determined from the fixed overall composition and phase fraction. The total two-phase free-energy density is then p f(rho_1)+(1-p) f(rho_2).

```
species = a, b, c
interactions primary: aa: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.0
interactions primary: ab: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.5
interactions primary: ac: type = gs,  sigma = 1.000, cutoff = 3.5, epsilon = -0.599
interactions primary: bb: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.0
interactions primary: bc: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 0.0
interactions primary: cc: type = hc,  sigma = 1.0, cutoff = 5.0, epsilon = 2.0
interactions secondary: ac: type = ghc,  sigma = 1.0, cutoff = 3.2, epsilon=2
interactions secondary: bc: type = ghc,  sigma = 1.0, cutoff = 3.2, epsilon=2
rdf closure aa = hnc
rdf closure ab = hnc
rdf closure ac = py
rdf closure bb = hnc
rdf closure bc = py
```

```
rdf closure cc = py
rdf alpha_max =  0.1
rdf max_iteration  = 100000
rdf tolerence = 0.000001
rdf beta = 1.0
free_energy ensemble = isocore
free_energy mode = standard
free_energy method = smf
free_energy hs_functional = rosenfeld
free_energy integrated_strength_kernel = uniform
free_energy supplied_data = no
solution intrinsic_constraints: species_fraction: a = 0.18
solution intrinsic_constraints: species_fraction: b = 0.18
solution intrinsic_constraints: species_fraction: c = 0.09
solution extrinsic_constraints: number_of_phases = 2
solution extrinsic_constraints: heterogeneous_pair = ab
solution extrinsic_constraints: total_density_bound = 2.0
solution extrinsic_constraints: density: a = 0.18
solution extrinsic_constraints: density: b = 0.18
solution extrinsic_constraints: density: c = 0.09
```

The script constructs two planes: an XY plane varying species a and b while holding c at the coexistence value of each phase, and a YZ plane varying b and c. Each trial state is evaluated through evaluate_canonical_state; the script saves valid points to output_XY_data.txt and output_YZ_data.txt and produces contour/surface plots.

File	Columns
output_XY_data.txt	X1, Y1, X2, Y2, TotalEnergy
output_YZ_data.txt	Y1, Z1, Y2, Z2, TotalEnergy
Example-specific clipping — The energy-landscape script currently keeps XY values only when total_energy < 2.5. This is an example-specific visualization/data-selection rule, not a general cDFT requirement. Remove or change it when adapting the workflow to another system.	

17. Inhomogeneous One-Dimensional Profile Example with External Confinement

Folder contents — Inhomogeneous_profile/Ternary_colloid_polymer_mixture contains only the active input and the short example_workflow_isocore_planer.py driver. The reachable workflow is parse input -> one_d_profile_iterator_box. The Path import is unused; exit(0) is simply the final statement and does not hide any later computational stage.

The inhomogeneous example is the most configuration-rich workflow in the catalogue. It combines a ternary interaction model, RDF closure settings, a free-energy model, external species/positions, a one-dimensional confinement grid, a planar-RDF grid, and profile-iteration controls. The Python workflow itself is intentionally short because one_d_profile_iterator_box orchestrates the internal stages.

17.1 Complete active input used by the example

```
task = profile_planer
ensemble = isocore
species = a, b, c
interactions primary: aa: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.0
interactions primary: ab: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.5
interactions primary: ac: type = mie, m = 36, n = 18, sigma = 1.000, cutoff = 3.5, epsilon = 1.6358
interactions primary: bb: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.0
interactions primary: bc: type = lj,  sigma = 1.0, cutoff = 1.1224, epsilon = 1.0
interactions primary: cc: type = lj,  sigma = 1.0, cutoff = 1.1224, epsilon = 1.0
interactions secondary: ac: type = gs,  sigma = 1, cutoff = 3.2, epsilon=0.1
rdf closure aa = hnc
rdf closure ab = hnc
```

```
rdf closure ac = py
rdf closure bb = hnc
rdf closure bc = py
rdf closure cc = py
rdf alpha_max =  0.01
rdf max_iteration  = 100000
rdf tolerence = 0.00001
rdf beta = 1.0
rdf r_max  =  5.0
rdf n_points =  200
free_energy ensemble = isocore
free_energy mode = standard
free_energy method = emf
free_energy integrated_strength_kernel = meanfield
free_energy supplied_data = no
solution intrinsic_constraints: species_fraction: a = 0.27
solution intrinsic_constraints: species_fraction: b = 0.27
solution intrinsic_constraints: species_fraction: c = 0.09
solution extrinsic_constraints: number_of_phases = 2
solution extrinsic_constraints: heterogeneous_pair = None
solution extrinsic_constraints: total_density_bound = 2.0
external species = d
external interaction ad: type = custom_3,  sigma = 1.0, cutoff = 2.5, epsilon = 0.01
external interaction bd: type = custom_3,  sigma = 1.0, cutoff = 2.5, epsilon = 0.01
external interaction cd: type = custom_3,  sigma = 1.0, cutoff = 2.5, epsilon = 0.01
external d position = 0.0,  0.0,  0.0
external d position = 60.0, 0.0,  0.0
space_confinement_parameters space_properties: dimension = 1
space_confinement_parameters space_properties: space_confinement = abox
space_confinement_parameters box_properties: box_length = 60, 5, 0
space_confinement_parameters box_properties: box_points = 200, 50, 0
planer_rdf space_properties: dimension = 2
planer_rdf space_properties: space_confinement = abox
planer_rdf tolerance = 0.001
planer_rdf box_properties: box_length = 60, 5, 0
planer_rdf box_properties: box_points = 200, 50, 0
profile iteration_max  =   3000
profile vij_interval = 1000
profile tolerance  = 0.0000000001
profile alpha_mixing_max = 0.001
profile log_period = 10
```

17.2 Input blocks unique to profile calculations

Block	Role in the workflow
external species / interaction	defines fixed external objects/walls and how each mobile species couples to them
external <species> position	may be repeated; two d positions at x=0 and x=60 form the example confinement
space_confinement_parameters	defines the 1D box used for the density profile
planer_rdf	defines the planar correlation grid used by the inhomogeneous machinery
profile iteration_max	maximum profile iterations
profile vij_interval	frequency for updating interaction-strength information
profile tolerance	profile convergence threshold
profile alpha_mixing_max	maximum mixing factor for density updates
profile log_period	iteration reporting interval

17.3 Workflow call

```
one_d_profile_iterator_box(
    ctx=ctx,
    config=system,
    export_json=True,
    export_plots=True,
    filename="one_d_profiles",
)
```

Internally the iterator builds box and planar grids, constructs the external potential, prepares the free-energy contributions, iterates the spatial densities, and exports density distributions and thermodynamic diagnostics. The current implementation writes per-species files named data_density_distribution_r_<species>.txt, pressure/surface-tension style data, grand-potential data, and vis_rho_distribution.png in the configured output locations.

External potential registration — The example uses interaction type custom_3 for the external field. If a custom external potential is not already registered in the runtime potential registry, that name must be registered before the profile calculation. Part VI explains runtime-defined potential registration and reuse for external fields.

18. Phase Finder, Dispersion Relation, and Second-Virial Analysis Examples

18.1 Thermodynamic phase-finder surface

The Phase_finder folder contains one input, one automation driver, and three independent plotting helpers. The driver first performs a valid one-component evaluate_canonical_state calculation using the supplied one-species input. It then contains a second, binary-style two-dimensional packing-fraction grid section. Because that later section assumes species a and b while the input defines only a, it should be treated as a template that requires a compatible two-species input rather than as a directly runnable continuation of the first stage.

```
species = a
interactions primary: aa: type = lj,  sigma = 1.0,  cutoff = 3.5,  epsilon = 1.0
rdf closure aa = hnc
rdf alpha_max =  0.01
rdf max_iteration  = 100000
rdf tolerence = 0.000001
rdf beta = 1.0
free_energy mode = standard
free_energy method = smf
free_energy integrated_strength_kernel = uniform
free_energy supplied_data = no
solution extrinsic_constraints: density: a = 0.15
```

In the later grid template, packing fractions are converted to number densities with rho=phi/[(pi/6)d^3], density entries a and b are updated, and the intended output schema contains phi_a, phi_b, rho_a, rho_b, mu_a, mu_b, pressure, and free_energy_density in thermo_surface_phi_data.json. With the supplied one-species input, the attempt to read the second chemical potential is inconsistent and must be corrected before this section can complete.

Current snapshot caveat — The shipped Phase_finder input defines only species a, while the grid section of the Python script later tries to update both density["a"] and density["b"] and read two chemical potentials. The single-state evaluation earlier in the script is compatible with the one-species input, but the 2D surface section requires a binary input. The grand manual documents this mismatch explicitly rather than presenting the grid section as directly runnable without correction.

18.2 Dispersion relation from direct correlations

```
species = a, b
interactions primary: aa: type = nvrn,  sigma = 1.0, cutoff = 3.5, epsilon = 42, n = 3, m_i = 1.0, m_j = 1.0
interactions primary: bb: type = nvrn,  sigma = 1.0, cutoff = 3.5, epsilon = 42, n = 3, m_i = 0.025, m_j =
0.025
interactions primary: ab: type = nvrn,  sigma = 1.0, cutoff = 3.5, epsilon = 42, n = 3, m_i = 1.0, m_j =
0.025
rdf closure aa = hnc
rdf closure ab = hnc
rdf closure bb = hnc
rdf alpha_max =  0.001
rdf dimension  = 2d
rdf max_iteration  = 1000000
rdf tolerance = 0.00001
rdf beta = 1.0
rdf r_max  =  10.0
rdf n_points =  4000
```

The workflow supplies densities=[1.0,1.0] and diffusivity=[1.0,1.0] and calls dispersion_relation. The rdf.dimension field controls whether the routine obtains c_ij(r) from the isotropic 3D RDF solver or the pure-2D solver. It then Fourier/Hankel transforms c(r) to c(k), constructs the thermodynamic matrix C_ij(k)=delta_ij-rho_j c_ij(k), multiplies by k^2 D, and stores minus the real eigenvalues as dispersion branches omega_n(k).

```
result = dispersion_relation(
    ctx=ctx,
    rdf_config=system,
    densities=np.array([1.0, 1.0]),
    diffusivity=np.array([1.0, 1.0]),
    supplied_data=None,
    export=True,
    plot=True,
    filename_prefix="dispersion",
)
```

The return value has two top-level entries: rdf (the structural result used to build c(r)) and dispersion with k and omega. Exported JSON additionally records species, densities, dimension, and pair c_r arrays. One PNG is written per eigenmode.

18.2.1 Structural interpretation of the dispersion matrix

The dispersion calculation inherits its thermodynamic content from the DCF matrix. Because c_ij(k) is linked through the OZ equations to the partial structure factors, the eigenmodes of the constructed matrix describe collective combinations of species-density fluctuations rather than independent motion of each component. Small k corresponds to long wavelengths, precisely the regime where the supplied structural paper finds enhanced sensitivity of colloid-containing structure factors to attractive interactions and impending phase behavior. A positive growth branch in a dynamical convention should therefore be interpreted together with the low-k structural response and the chosen diffusivities, not as a property of the bare potential alone. [Theory: Varma, Archer & Scacchi, 2026 manuscript.]

Clean dispersion workflow — The driver calls process_supplied_data even though example_input_dispersion.in has no supplied_data block, and the returned object is ignored because dispersion_relation is called with supplied_data=None. The minimal executed calculation is parse input -> dispersion_relation. The imports Path, rdf_planer, rdf_radial, boltzmann_inversion_advance, and c_analysis are also unused by this driver; plotter_dispersion_relation.py and plotter_rdf_relation.py are separate post-processing helpers.

18.3 Second virial coefficient and thermodynamic integrated strength

The virial example uses two inputs. The first defines a reference interaction for which second_virial returns both B2 and an integrated strength; the second defines a target interaction whose amplitude is calibrated to a chosen B2 target.

```
species = a
interactions primary: aa: type = lj,  sigma = 1.0, cutoff = 6, epsilon = 0.15
virial beta = 1.0
B2, strength = second_virial(
    ctx=ctx_ref,
    virial_config=system_ref,
    on="raw",
    export=True,
    filename_prefix="second_virial_coefficient",
    r_max_factor=12.0,
    nr=8192,
    n_lambda=128,
)
```

For on="raw", B2 is calculated from the full raw potential through the Mayer-function integral. The thermodynamic integrated strength is evaluated with a lambda grid and g_lambda(r)=exp[-lambda beta u(r)] in the low-density limit. The exported JSON stores B2 and integrated_strength for each pair.

Driver cleanup note — example_workflow_virial_analysis.py calls process_supplied_data even though neither virial input contains a supplied_data block, so that step is unnecessary. NumPy and Path are also unused. After computing and printing the reference B2, the script explicitly overwrites b2t with [[-0.125]] before calibration; consequently the shipped calibration targets -0.125 rather than the B2 just calculated. A reusable workflow should make that target choice explicit instead of appearing to pass the first-stage result automatically.

18.4 Calibrating another potential to a target B2

```
species = a
interactions primary: aa: type = mie,  sigma = 1.0, n = 48, m = 24, cutoff = 6, epsilon = 1.0
virial beta = 1.0
B2_new, scale = second_virial_scale_calibration(
    ctx=ctx_target,
    virial_config=system_target,
    B2_target=B2_target,
    on="raw",
    export=True,
    filename_prefix="second_virial_scale_calibration",
    r_max_factor=12.0,
    nr=8192,
    beta_scale=1.0,
)
```

The calibration optimizes one multiplicative scale factor per unique pair. When a meaningful Barker-Henderson diameter is detected, the objective compares the reduced ratio B2/B2_HS with the supplied target; otherwise it compares B2 directly. The exported JSON stores B2, B2_target, and scale_factor per pair.

Example target override — The shipped workflow first computes b2t from the reference potential, prints it, and then replaces it with [[-0.125]] before calibration. Therefore the actual calibration target in the example is -0.125, not the B2 just computed. Delete that assignment if your intention is to calibrate directly to the reference result.

19. Example Selection, Portable Adaptation, and Physical-Quantity Cookbook

This chapter turns the example catalogue into a practical starting-point map. Choose the example whose output is closest to the quantity you need, then replace only the system-specific inputs and scan variables.

19.1 Which example should I start from?

Goal	Best starting example	Primary output
bulk g_ij(r)	Radial RDF	g, c, gamma
planar correlations	Planar RDF	g(z_i,z_j,r), c, gamma
infer u(r) from RDF	Potential inversion	effective potential
analyze DCF/structure for known u(r)	Structural correlation	c(r), gamma(r), structural diagnostics
pressure vs density	EOS	pressure, mu, vij
one coexistence point	LJ isocore/isochem	phase densities, fractions
binary binodal	Binary isochem automation	paired coexistence points
ternary response curve	Ternary phase response	coexistence vs polymer/colloid density
two-phase free-energy surface	Energy landscape	constrained total-energy surface
inhomogeneous density profile	Ternary profile	rho_i(x), interfacial diagnostics
thermodynamic surface	Phase finder	mu, P, f over density grid
linear growth/decay modes	Dispersion relation	omega_n(k)
B2 and equal-B2 mapping	Virial analysis	B2, strength, scale factors

19.2 Portable path pattern

```
run_dir = Path("my_calculation").resolve()
input_file = run_dir / "inputs" / "system.in"
scratch = run_dir / "results"
plots = scratch / "plots"
scratch.mkdir(parents=True, exist_ok=True)
plots.mkdir(parents=True, exist_ok=True)

ctx = ExecutionContext(
    input_file=str(input_file),
    scratch_dir=scratch,
    plots_dir=plots,
)
```

For tabulated interactions and supplied RDFs, keep their filenames relative to the input working directory expected by the relevant loader or pass/construct an ExecutionContext whose work_dir resolves those files. Do not reproduce the Examples folder structure unless you want to run the shipped example unchanged.

19.3 Reuse expensive preprocessing

13. Parse the input once.
14. Build only the potential/reference objects actually required by the downstream calculation; keep raw/total potential generation as an optional export/diagnostic branch when those files are useful.
15. Construct the symbolic total free energy once.
16. Inside density/composition scans, update only the relevant constraint entries.
17. Call evaluate_canonical_state or the coexistence solver for each scan point.
18. Export one compact dataset after the scan. Avoid regenerating intermediate potential representations at every point unless a density-dependent workflow explicitly requires them.

19.4 When to move to Part VI

Use the bundled examples as the first layer. Move to Part VI when you need a custom closure, runtime-defined potential, file-backed potential plus analytic contributions, supplied RDFs for only selected pairs, mixed computed/supplied structure, partial inversion, or one shared potential constrained by several state points.

PART IV — Audited Example Input Systems and Cleaned Reachable Workflows

20. Troubleshooting and Current-Implementation Notes

Symptom	Likely cause / action
FileNotFoundError for pair potential	Potential splitters use os.getcwd() with the filename. Run from the input/data directory, use an absolute filename, or adopt the portable os.chdir(input_path.parent) pattern.
Supplied RDF file not found	process_supplied_data also falls back to Path.cwd() because ExecutionContext has no work_dir field.
NameError: SuppliedDataError	process_supplied_data raises SuppliedDataError but the class is not defined/imported in that module. Valid inputs avoid the branch; malformed/missing supplied data can expose this source bug.
Unknown free energy mode advanced	Current total_free_energy accepts standard, hybrid, hybrid_lattice. Old comments mentioning advanced are stale.
Unknown method zdl	Current mean_field dispatch accepts smf, emf, void. Old comments calling the third method zdl do not match the current key.
tolerance seems ignored	Check whether the target module reads tolerance or the legacy misspelling tolerence.
Single species behaves strangely	Use a one-character label. Parser returns "a" rather than ["a"], but many modules rely on iterating characters and therefore only one-character scalar labels are safe.
Pair names with long species labels fail	Several modules split pair keys as key[0], key[1]. Current safe convention is a, b, c.
Unexpected “None” heterogeneous pair	The parser stores None as the string "None". Prefer omitting the line when no heterogeneous pair constraint is needed, or be aware the current solver ignores it as a pair because its string length is not 2.
EOS is very slow	rdf integrated strength calls an internal structural/coupling calculation for each state. uniform/meanfield kernels are cheaper but represent a different physical approximation.
OZ shared-library load error	The platform-specific liboz_radial binary may not match the OS/architecture; compile the accompanying C source and required numerical libraries.
Install succeeds but workflow import fails	Declared dependencies omit some active imports such as sympy, numba and tqdm. Install them explicitly.
executor_dft_main import error	The legacy executor references get_unique_dir and engine dependencies absent from the current module tree. Use direct module workflows.

Potential is not zero at stated Gaussian cutoff	The Gaussian/GEM callables do not enforce cutoff internally; cutoff can still influence grid extent. If strict truncation is required, provide a tabulated truncated potential or modify the potential implementation.
Mie exponent confusion	Follow the supplied convention m=larger repulsive exponent and n=smaller exponent, e.g. m=24,n=12.

21–22. Merged Source-Audited Example Inputs and Reachable Workflows

This merged chapter replaces the former generic input-design chapter and the separate input-dictionary catalogue. It is built directly from the supplied Examples.zip snapshot. The archive contains 20 example_workflow_*.py drivers, 21 .in files, and 19 unique input definitions. The purpose here is to make every example self-contained: the exact active input lines and the cleaned executable Python path appear together.

Uniform audit rule — “cleaned reachable workflow” means that the code block retains the package calls and scan logic required for the physical calculation, removes imports that are never referenced, removes process_supplied_data when no supplied_data block is present and the result is discarded, removes diagnostic potential/export branches whose returned objects are not consumed by the later physical calculation, and removes every statement after an unconditional top-level exit(). A reachable but inconsistent block is not silently fixed; it is identified explicitly in the audit note.

Input-system rule — The .in blocks below reproduce all nonblank, noncomment lines from the corresponding file. Values and key spellings are preserved exactly, including legacy spellings such as tolerence and any filename mismatch that exists in the supplied folder. Comments are omitted only to keep the presentation uniform.

21–22.1 Radial_distrubtion_function — 3D radial and 2D planar RDF

Folder(s)	Examples/Radial_distrubtion_function
Driver(s)	example_workflow_rdf_radial.py ; example_workflow_rdf_planer.py
Scientific purpose	Compute an isotropic radial RDF for a one-component fluid or a planar two-dimensional correlation field for a three-species density profile.
Local files	example_input_rdf_planer.in, example_input_rdf_radial.in, example_workflow_rdf_planer.py, example_workflow_rdf_radial.py
Support-data role	No external numerical input files. The folder contains only the two .in files and the two drivers.
Executed flow	radial: parse input -> rho=[0.3] -> rdf_radial; planar: parse input -> construct rho_z with shape (Ns,Nz) -> rdf_planer.

Active input system — Radial_distrubtion_function/example_input_rdf_radial.in

```
task = profile_planer
ensemble = isocore
species = a
interactions primary: aa: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.01
rdf closure aa = hnc
rdf alpha_max =  0.001
rdf max_iteration  = 100000
rdf tolerance = 0.00001
rdf beta = 1.0
rdf r_max  =  10.0
rdf n_points =  400
```

Active input system — Radial_distrubtion_function/example_input_rdf_planer.in

```
task = rdf_planer
species = a, b, c
interactions primary: aa: type = gs,   sigma = 1.414,  cutoff = 3.5,     epsilon = 2.0
interactions primary: ab: type = gs,   sigma = 1.414,  cutoff = 3.5,     epsilon = 2.5
interactions primary: ac: type = ma,  sigma = 1.0,    cutoff = 5.0,    epsilon = 0.3104, m = 24, n = 12
interactions primary: bb: type = gs,   sigma = 1.414,  cutoff = 3.5,     epsilon = 2.0
interactions primary: cc: type = lj,   sigma = 1.0,    cutoff = 1.1224,  epsilon = 1.0
interactions secondary: aa: type = ghc,  sigma = 1.02, cutoff = 3.2, epsilon = 1.0
interactions secondary: bb: type = ghc,  sigma = 1.0, cutoff = 3.2, epsilon = 1.0
rdf closure aa = hnc
rdf closure ab = hnc
rdf closure ac = py
rdf closure bb = hnc
rdf closure bc = py
rdf closure cc = py
rdf alpha_max =  0.1
rdf max_iteration  = 100000
rdf tolerance = 0.01
rdf beta = 1.0
planer_rdf space_properties: dimension = 2
planer_rdf space_properties: space_confinement = abox
planer_rdf box_properties: box_length = 5, 5, 0
planer_rdf box_properties: box_points = 50, 50, 0
```

Cleaned reachable workflow — radial driver

```
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.calculators.radial_distribution_function.rdf_radial import rdf_radial

scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)
ctx = ExecutionContext(
    input_file="example_input_rdf_radial.in",
    scratch_dir=scratch,
    plots_dir=plots,
)
system = super_dictionary_creator(
    ctx, export_json=True, filename="input_system.json", super_key_name="system"
)

rho = [0.3]
rdf_radial(
    ctx,
    rdf_config=system,
    densities=rho,
    supplied_data=None,
    export=True,
    plot=True,
    filename_prefix="rdf",
)
```

Cleaned reachable workflow — planar driver

```
import numpy as np
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.calculators.radial_distribution_function.rdf_planer import rdf_planer

scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)
ctx = ExecutionContext(
    input_file="example_input_rdf_planer.in",
    scratch_dir=scratch,
    plots_dir=plots,
)
system = super_dictionary_creator(
    ctx, export_json=True, filename="input_system.json", super_key_name="system"
)

Nz = 50
rho_bulk = np.array([0.4, 0.0001, 0.09])
rho_z = np.repeat(rho_bulk[:, None], Nz, axis=1)  # shape (Ns, Nz)

rdf_planer(
    ctx,
    rdf_config=system,
    densities=rho_z,
```

```
    supplied_data=None,
    export=True,
    plot=True,
    filename_prefix="rdf_2d",
)
```

Audit / rectification notes

- The literal archive spelling is Radial_distrubtion_function and is retained so users can find the folder.
- The radial driver imports Path, rdf_planer, and NumPy without using them; those imports are removed.
- The planar driver imports Path without using it; it is removed.

21–22.2 Potential_inversion/One_component_colloid_inversion

Folder(s)	Examples/Potential_inversion/One_component_colloid_inversion
Driver(s)	example_workflow_potential_invertor.py
Scientific purpose	Infer a one-component effective pair potential from a supplied RDF using the advanced IBI workflow.
Local files	example_input_inversion.in, example_workflow_potential_invertor.py, potential_extractor.py, rdf_colloid.txt
Support-data role	rdf_colloid.txt is the supplied target RDF. potential_extractor.py is a separate helper and is not called by the driver.
Executed flow	parse input -> process supplied RDF -> boltzmann_inversion_advance.

Active input system — Potential_inversion/One_component_colloid_inversion/example_input_inversion.in

```
species = a
rdf closure aa = py
rdf alpha_max =  0.1
rdf max_iteration  = 40000
rdf max_iteration_ibi = 10000
rdf alpha_ibi_max  = 0.1
rdf tolerance = 0.00001
rdf ibi_tolerance  = 0.0001
rdf beta = 1.0
rdf r_max  =  20.0
rdf n_points =  2000
supplied_data states 1 rdf aa: filename = rdf_colloid.txt
supplied_data states 1 densities a = 0.4
supplied_data states 1 beta = 1.0
```

Cleaned reachable workflow

```
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.generators.supplied_data.process_supplied_data import process_supplied_data
from cdft_solver.calculators.rdf_inversion.rdf_inversion_advance import boltzmann_inversion_advance

scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)
ctx = ExecutionContext(
    input_file="example_input_inversion.in",
    scratch_dir=scratch,
    plots_dir=plots,
)
system = super_dictionary_creator(
    ctx, export_json=True, filename="input_system.json", super_key_name="system"
)
supplied_data = process_supplied_data(
    ctx=ctx, config=system, export_json=True, export_plot=True
)

boltzmann_inversion_advance(
    ctx=ctx,
    rdf_config=system,
    supplied_data=supplied_data,
    filename_prefix="multistate",
    export_plot=True,
```

```
    export_json=True,
)
```

Audit / rectification notes

- Unused NumPy, Path, rdf_planer, and rdf_radial imports are removed.
- The supplied-data stage is essential here because its returned dictionary is passed directly to boltzmann_inversion_advance.

21–22.3 Potential_inversion/Binary_colloid_polymer_inversion

Folder(s)	Examples/Potential_inversion/Binary_colloid_polymer_inversion
Driver(s)	example_workflow_potential_invertor.py
Scientific purpose	Infer aa, ab, and bb effective interactions for a binary mixture from a complete set of partial RDFs at the supplied state.
Local files	average_g11.txt, average_g12.txt, average_g22.txt, example_input_inversion.in, example_workflow_potential_invertor.py, potential_extractor.py
Support-data role	average_g11.txt, average_g12.txt, and average_g22.txt supply the three unique pair RDFs. potential_extractor.py is independent post-processing.
Executed flow	parse input -> process three supplied partial RDFs -> boltzmann_inversion_advance.

Active input system — Potential_inversion/Binary_colloid_polymer_inversion/example_input_inversion.in

```
species = a, b
rdf closure aa = hnc
rdf closure ab = py
rdf closure bb = py
rdf alpha_max =  0.1
rdf max_iteration  = 40000
rdf max_iteration_ibi = 10000
rdf alpha_ibi_max  = 0.1
rdf tolerance = 0.00001
rdf ibi_tolerance  = 0.0001
rdf beta = 1.0
rdf r_max  =  20.0
rdf n_points =  2000
supplied_data states 1 rdf aa: filename = average_g11.txt
supplied_data states 1 rdf ab: filename = average_g12.txt
supplied_data states 1 rdf bb: filename = average_g22.txt
supplied_data states 1 densities a = 0.109860935
supplied_data states 1 densities b = 0.022041065
supplied_data states 1 beta = 1.0
```

Cleaned reachable workflow

```
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.generators.supplied_data.process_supplied_data import process_supplied_data
from cdft_solver.calculators.rdf_inversion.rdf_inversion_advance import boltzmann_inversion_advance

scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)
ctx = ExecutionContext(
    input_file="example_input_inversion.in",
    scratch_dir=scratch,
    plots_dir=plots,
)
system = super_dictionary_creator(
    ctx, export_json=True, filename="input_system.json", super_key_name="system"
)
supplied_data = process_supplied_data(
    ctx=ctx, config=system, export_json=True, export_plot=True
)

boltzmann_inversion_advance(
    ctx=ctx,
    rdf_config=system,
    supplied_data=supplied_data,
```

```
    filename_prefix="multistate",
    export_plot=True,
    export_json=True,
)
```

Audit / rectification notes

- Unused NumPy, Path, rdf_planer, and rdf_radial imports are removed.
- All three pair RDFs are constrained at the same state defined by the input densities and beta.

21–22.4 Structural_correlation_analysis/One_component

Folder(s)	Examples/Structural_correlation_analysis/One_component
Driver(s)	example_workflow_c_2_analysis.py
Scientific purpose	Compute the real/reference structural correlations and direct-correlation analysis for a one-component tabulated interaction at density 0.4.
Local files	aa_colloids.txt, example_input_structural_analysis.in, example_workflow_c_2_analysis.py, plotter_correlation_function.py, plotter_integrated_strength_kernel.py
Support-data role	aa_colloids.txt is a tabulated interaction potential referenced by the input. The two plotter scripts are separate post-processing utilities.
Executed flow	parse input -> c_analysis(densities=[0.4]).

Active input system — Structural_correlation_analysis/One_component/example_input_structural_analysis.in

```
species = a
interactions primary: aa: filename = aa_colloids.txt
rdf closure aa = py
rdf alpha_max =  0.4
rdf max_iteration  = 400000
rdf tolerance = 0.000001
rdf beta = 1.0
rdf r_max  =  20.0
rdf n_points =  2000
```

Cleaned reachable workflow

```
import numpy as np
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.calculators.c_2_analysis.c_analysis import c_analysis

scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)
ctx = ExecutionContext(
    input_file="example_input_structural_analysis.in",
    scratch_dir=scratch,
    plots_dir=plots,
)
system = super_dictionary_creator(
    ctx, export_json=True, filename="input_system.json", super_key_name="system"
)

c_analysis(
    ctx=ctx,
    rdf_config=system,
    supplied_data=None,
    densities=np.array([0.4]),
    filename_prefix="multistate",
    export_plot=True,
    export_json=True,
)
```

Audit / rectification notes

- process_supplied_data is removed because this input has no supplied_data block and its returned object is not passed to c_analysis.

- rdf_planer, rdf_radial, and boltzmann_inversion_advance are imported by the source driver but never used.

21–22.5 Structural_correlation_analysis/Binary_colloid_polymer_mixture

Folder(s)	Examples/Structural_correlation_analysis/Binary_colloid_polymer_mixture
Driver(s)	example_workflow_c_2_analysis.py
Scientific purpose	Compute pp/cp/cc structural and DCF diagnostics for a binary mixture at densities [0.6, 0.096].
Local files	aa.txt, ab.txt, bb.txt, colloid_colloid.png, example_input_structural_analysis.in, example_workflow_c_2_analysis.py, plotter_integrated_strength_binary.py, plotter_structural_correlation_binary.py, polymer_colloid.png, polymer_polymer.png
Support-data role	aa.txt, ab.txt, and bb.txt are tabulated pair potentials. Plotter scripts and the three PNGs are post-processing/reference artifacts.
Executed flow	parse input -> c_analysis(densities=[0.6,0.096]).

Active input system — Structural_correlation_analysis/Binary_colloid_polymer_mixture/example_input_structural_analysis.in

```
species = a, b
interactions primary: aa: filename = aa.txt
interactions primary: ab: filename = ab.txt
interactions primary: bb: filename = bb.txt
rdf closure aa = hnc
rdf closure bb = py
rdf closure ab = py
rdf alpha_max =  0.1
rdf max_iteration  = 400000
rdf tolerance = 0.00001
rdf beta = 1.0
rdf r_max  =  10.0
rdf n_points =  2000
```

Cleaned reachable workflow

```
import numpy as np
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.calculators.c_2_analysis.c_analysis import c_analysis

scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)
ctx = ExecutionContext(
    input_file="example_input_structural_analysis.in",
    scratch_dir=scratch,
    plots_dir=plots,
)
system = super_dictionary_creator(
    ctx, export_json=True, filename="input_system.json", super_key_name="system"
)

c_analysis(
    ctx=ctx,
    rdf_config=system,
    supplied_data=None,
    densities=np.array([0.6, 0.096]),
    filename_prefix="multistate",
    export_plot=True,
    export_json=True,
)
```

Audit / rectification notes

- process_supplied_data is a no-op for the supplied input and its result is discarded; it is removed.
- The tabulated files enter through interactions ... filename, not through supplied_data.

21–22.6 Equation_of_state/One_component_colloid_system

Folder(s)	Examples/Equation_of_state/One_component_colloid_system
Driver(s)	example_workflow_phase_finder_thermodynamic_quantities_automation.py
Scientific purpose	Evaluate pressure, chemical potential, and integrated strength along the supplied one-component density grid.
Local files	aa_colloids_12_6.txt, aa_colloids_24_12.txt, example_input_phase_finder_isocore.in, example_workflow_phase_finder_thermodynamic_quantities_automation.py
Support-data role	aa_colloids_12_6.txt and aa_colloids_24_12.txt are alternative interaction tables. The input as shipped asks for aa_colloids.txt, which is not present.
Executed flow	parse input -> hard-core preprocessing -> total_free_energy -> density scan -> evaluate_canonical_state -> eos_rho_data.json.

Active input system — Equation_of_state/One_component_colloid_system/example_input_phase_finder_isocore.in

```
species = a
interactions primary: aa: filename = aa_colloids.txt
rdf closure aa = py
rdf alpha_max =  0.01
rdf r_max = 5.0
rdf max_iteration  = 200000
rdf tolerence = 0.000001
rdf n_points =  2000
rdf beta = 1.0
free_energy mode = standard
free_energy hs_functional = whitebeer
free_energy method = smf
free_energy integrated_strength_kernel = rdf
free_energy supplied_data = no
solution extrinsic_constraints: density: a = 0.15
```

Cleaned computational workflow

```
import json
import numpy as np
from tqdm import tqdm
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.generators.potential_splitter.hc import hard_core_potentials
from cdft_solver.calculators.total_free_energy.total_free_energy import total_free_energy
from cdft_solver.calculators.phase_finder.thermodynamic_potentials_isocore import evaluate_canonical_state

scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)
ctx = ExecutionContext(
    input_file="example_input_phase_finder_isocore.in",
    scratch_dir=scratch,
    plots_dir=plots,
)
system = super_dictionary_creator(ctx, export_json=True,
    filename="input_system.json", super_key_name="system")
hc_data = hard_core_potentials(ctx=ctx, input_data=system, grid_points=5000,
    file_name_prefix="hc.json", export_files=True)
free_energy = total_free_energy(
    ctx=ctx, hc_data=hc_data, system_config=system,
    export_json=True,
    filenames={"hard_core":"fe_hc.json", "mean_field":"fe_mf.json",
               "ideal":"fe_id.json", "hybrid":"fe_hybrid.json"},
)

rho_vals = np.array([
    0.047829631, 0.092051503, 0.131009262, 0.162611908,
    0.196647877, 0.225118437, 0.326015639, 0.362218,
    0.396196, 0.426810, 0.453910, 0.478527,
    0.641778, 0.734988, 0.798874, 0.847508,
])
results = []
for rho in tqdm(rho_vals, desc="Computing EOS"):
    system["system"]["solution"]["extrinsic_constraints"]["density"]["a"] = float(rho)
    result = evaluate_canonical_state(
```

```
            ctx=ctx, config_dict=system, fe_res=free_energy,
            supplied_data=None, export=False, verbose=False,
        )
        results.append({
            "rho": float(rho),
            "vij": result["vij"],
            "mu": result["chemical_potentials"][0],
            "pressure": result["pressure"],
        })

    with open("eos_rho_data.json", "w") as f:
        json.dump(results, f, indent=4)
```

Audit / rectification notes

- Before execution, set the input filename to aa_colloids_12_6.txt or aa_colloids_24_12.txt (or provide an aa_colloids.txt alias). The manual does not silently change the shipped input.
- meanfield_potentials, raw_potentials, total_potentials, and free_energy_exporter are executed in the source mainly for exported diagnostics; their returned objects are not required by the EOS state loop, so they are omitted from the minimal computational workflow.
- Path is imported but unused.

21–22.7 Equation_of_state/Binary_colloid_polymer_mixture

Folder(s)	Examples/Equation_of_state/Binary_colloid_polymer_mixture
Driver(s)	example_workflow_phase_finder_thermodynamic_quantities_automation.py
Scientific purpose	Evaluate the EOS of a binary mixture along a total-density grid at fixed mole fraction x_b=0.167.
Local files	aa.txt, ab.txt, bb.txt, example_input_phase_finder_isocore.in, example_workflow_phase_finder_thermodynamic_quantities_automation.py
Support-data role	aa.txt, ab.txt, and bb.txt are the three tabulated pair interactions consumed through the input file.
Executed flow	parse input -> hard-core preprocessing -> total_free_energy -> fixed-composition density scan -> evaluate_canonical_state -> eos_rho_data_binary.json.

Active input system — Equation_of_state/Binary_colloid_polymer_mixture/example_input_phase_finder_isocore.in

```
species = a, b
interactions primary: aa: filename  = aa.txt
interactions primary: ab: filename  = ab.txt
interactions primary: bb: filename  = bb.txt
rdf closure aa = hnc
rdf closure bb = py
rdf closure ab = py
rdf alpha_max =  0.01
rdf r_max = 3.5
rdf max_iteration  = 200000
rdf tolerence = 0.000001
rdf n_points =  2000
rdf beta = 1.0
free_energy mode = standard
free_energy hs_functional = whitebeer
free_energy method = smf
free_energy integrated_strength_kernel = rdf
free_energy supplied_data = no
solution extrinsic_constraints: density: a = 0.15
solution extrinsic_constraints: density: b = 0.
```

Cleaned computational workflow

```
import json
import numpy as np
from tqdm import tqdm
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.generators.potential_splitter.hc import hard_core_potentials
from cdft_solver.calculators.total_free_energy.total_free_energy import total_free_energy
from cdft_solver.calculators.phase_finder.thermodynamic_potentials_isocore import evaluate_canonical_state
```

```
scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)
ctx = ExecutionContext(input_file="example_input_phase_finder_isocore.in",
    scratch_dir=scratch, plots_dir=plots)
system = super_dictionary_creator(ctx, export_json=True,
    filename="input_system.json", super_key_name="system")
hc_data = hard_core_potentials(ctx=ctx, input_data=system, grid_points=5000,
    file_name_prefix="hc.json", export_files=True)
free_energy = total_free_energy(
    ctx=ctx, hc_data=hc_data, system_config=system, export_json=True,
    filenames={"hard_core":"fe_hc.json", "mean_field":"fe_mf.json",
               "ideal":"fe_id.json", "hybrid":"fe_hybrid.json"},
)

rho_vals = np.array([
    0.043148, 0.077346, 0.105969, 0.131388, 0.154042,
    0.174898, 0.193891, 0.211916, 0.228800, 0.244669,
    0.274328, 0.301519, 0.326719, 0.350384, 0.372804,
    0.551128, 0.685570, 0.797709, 0.895470,
])
x = 0.167
results = []
for rho in tqdm(rho_vals, desc="Computing EOS (binary)"):
    rho_a, rho_b = rho * (1.0 - x), rho * x
    system["system"]["solution"]["extrinsic_constraints"]["density"]["a"] = float(rho_a)
    system["system"]["solution"]["extrinsic_constraints"]["density"]["b"] = float(rho_b)
    result = evaluate_canonical_state(
        ctx=ctx, config_dict=system, fe_res=free_energy,
        supplied_data=None, export=False, verbose=False,
    )
    results.append({
        "rho_total": float(rho), "rho_a": float(rho_a), "rho_b": float(rho_b),
        "x_a": float(1.0-x), "x_b": float(x), "vij": result["vij"],
        "mu_a": result["chemical_potentials"][0],
        "mu_b": result["chemical_potentials"][1],
        "pressure": result["pressure"],
    })

with open("eos_rho_data_binary.json", "w") as f:
    json.dump(results, f, indent=4)
```

Audit / rectification notes

- raw_data and total_data are generated in the source but are not consumed by the state loop; they are omitted from the minimal physical workflow.
- Path is imported but unused.

21–22.8 Phase_coexistence/LJ_fluid — isocore and isochem

Folder(s)	Examples/Phase_coexistence/LJ_fluid
Driver(s)	example_workflow_isocore_bulk.py ; example_workflow_isochem_bulk.py
Scientific purpose	Demonstrate one-component two-phase coexistence in canonical/isocore and isochemical forms.
Local files	example_input_isochem.in, example_input_isocore.in, example_workflow_isochem_bulk.py, example_workflow_isocore_bulk.py, fig_4.png, plotter.py
Support-data role	plotter.py and fig_4.png are not required by either coexistence driver.
Executed flow	parse input -> hard-core preprocessing -> total_free_energy -> coexistence_densities_isocore OR coexistence_densities_isochem.

Active input system — Phase_coexistence/LJ_fluid/example_input_isocore.in

```
ensemble = isocore
species = a
interactions primary: aa: type = lj,   sigma = 1.0,    cutoff = 3.5,  epsilon = 0.8
rdf closure aa = py
rdf alpha_max =  0.01
rdf max_iteration  = 600000
```

```
rdf tolerence = 0.00001
rdf beta = 1.0
rdf r_max  =  20.0
rdf n_points =  2000
free_energy ensemble = isocore
free_energy mode = standard
free_energy method = smf
free_energy integrated_strength_kernel = rdf
free_energy hs_functional = whitebeer
free_energy supplied_data = no
solution intrinsic_constraints: species_fraction: a = 0.26
solution extrinsic_constraints: number_of_phases = 2
solution extrinsic_constraints: heterogeneous_pair = None
solution extrinsic_constraints: total_density_bound = 2.0
```

Active input system — Phase_coexistence/LJ_fluid/example_input_isochem.in

```
ensemble = isochem
species = a
interactions primary: aa: type = lj,   sigma = 1.0,    cutoff = 3.5,  epsilon =1.0
rdf closure aa = py
rdf alpha_max =  0.01
rdf max_iteration  = 200000
rdf tolerence = 0.00001
rdf beta = 1.0
rdf r_max  =  10.0
rdf n_points =  200
free_energy ensemble = isochem
free_energy mode = standard
free_energy method = smf
free_energy integrated_strength_kernel = delta_c
free_energy supplied_data = no
solution extrinsic_constraints: number_of_phases = 2
solution extrinsic_constraints: heterogeneous_pair = None
solution extrinsic_constraints: total_density_bound = 5.0
solution intrinsic_constraints: chemical_potential: None
```

Cleaned reachable workflow — isocore

```
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.generators.potential_splitter.hc import hard_core_potentials
from cdft_solver.calculators.total_free_energy.total_free_energy import total_free_energy
from cdft_solver.calculators.coexistence_densities.calculator_coexistence_densities_isocore import
coexistence_densities_isocore

scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)
ctx = ExecutionContext(input_file="example_input_isocore.in",
    scratch_dir=scratch, plots_dir=plots)
system = super_dictionary_creator(ctx, export_json=True,
    filename="input_system.json", super_key_name="system")
hc_data = hard_core_potentials(ctx=ctx, input_data=system, grid_points=5000,
    file_name_prefix="supplied_data_potential_hc.json", export_files=True)
free_energy = total_free_energy(
    ctx=ctx, hc_data=hc_data, system_config=system, export_json=True,
    filenames={"hard_core":"supplied_data_free_energy_hard_core.json",
               "mean_field":"supplied_data_free_energy_mean_field.json",
               "ideal":"supplied_data_free_energy_ideal.json",
               "hybrid":"supplied_data_free_energy_hybrid.json"},
)
value = coexistence_densities_isocore(
    ctx=ctx, config_dict=system, fe_res=free_energy, supplied_data=None,
    max_outer_iters=10, tol_outer=1e-3, tol_solver=1e-8, verbose=True,
)
print(value)
```

Cleaned reachable workflow — isochem

```
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.generators.potential_splitter.hc import hard_core_potentials
from cdft_solver.calculators.total_free_energy.total_free_energy import total_free_energy
from cdft_solver.calculators.coexistence_densities.calculator_coexistence_densities_isochem import
coexistence_densities_isochem

scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)
ctx = ExecutionContext(input_file="example_input_isochem.in",
```

```
    scratch_dir=scratch, plots_dir=plots)
system = super_dictionary_creator(ctx, export_json=True,
    filename="input_system.json", super_key_name="system")
hc_data = hard_core_potentials(ctx=ctx, input_data=system, grid_points=5000,
    file_name_prefix="supplied_data_potential_hc.json", export_files=True)
free_energy = total_free_energy(
    ctx=ctx, hc_data=hc_data, system_config=system, export_json=True,
    filenames={"hard_core":"supplied_data_free_energy_hard_core.json",
               "mean_field":"supplied_data_free_energy_mean_field.json",
               "ideal":"supplied_data_free_energy_ideal.json",
               "hybrid":"supplied_data_free_energy_hybrid.json"},
)
value = coexistence_densities_isochem(
    ctx=ctx, config_dict=system, fe_res=free_energy, supplied_data=None,
    max_outer_iters=10, tol_outer=1e-3, tol_solver=1e-8, verbose=True,
)
print(value)
```

Audit / rectification notes

- Both source drivers import rdf_radial without needing it for the coexistence result.
- In example_workflow_isochem_bulk.py, the rdf_radial call occurs after an unconditional exit(0) and is therefore unreachable. The cleaned workflow stops at coexistence, exactly as the executed source does.
- The grid and densities variables that exist only to support the unreachable RDF call are also omitted.

21–22.9 Phase_coexistence/Binary_colloid_polymer_mixture

Folder(s)	Examples/Phase_coexistence/Binary_colloid_polymer_mixture
Driver(s)	example_workflow_isochem_bulk_automation.py
Scientific purpose	Trace an isochemical binodal by scanning the imposed chemical potential of species a and the allowed total-density bound.
Local files	example_input_isochem.in, example_workflow_isochem_bulk_automation.py
Support-data role	No additional input data files are required by the driver.
Executed flow	parse input -> hard-core preprocessing -> total_free_energy -> mu_a/bound scan -> coexistence_densities_isochem -> binodal_data.txt.

Active input system — Phase_coexistence/Binary_colloid_polymer_mixture/example_input_isochem.in

```
species = a, b
interactions primary: aa: type = gs,  sigma = 1.0,   cutoff = 3.5,     epsilon = 2.0
interactions primary: ab: type = lj,  sigma = 1.0,   cutoff = 1.1224,    epsilon = 1.0
interactions primary: bb: type = mie, sigma = 1.0,  m = 24, n= 12,   cutoff =2.5,    epsilon = 0.1
rdf closure aa = py
rdf closure ab = py
rdf closure bb = py
rdf alpha_max =  0.1
rdf max_iteration  = 600000
rdf tolerence = 0.00001
rdf beta = 1.0
rdf r_max  =  10.0
rdf n_points =  2000
free_energy ensemble = isochem
free_energy mode = standard
free_energy method = smf
free_energy hs_functional = whitebeer
free_energy integrated_strength_kernel = uniform
free_energy supplied_data = no
solution extrinsic_constraints: number_of_phases = 2
solution extrinsic_constraints: heterogeneous_pair = None
solution extrinsic_constraints: total_density_bound = 10
solution intrinsic_constraints: chemical_potential: a = 14
```

Cleaned computational workflow

```
import copy
import numpy as np
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.generators.potential_splitter.hc import hard_core_potentials
```

```
from cdft_solver.calculators.total_free_energy.total_free_energy import total_free_energy
from cdft_solver.calculators.coexistence_densities.calculator_coexistence_densities_isochem import
coexistence_densities_isochem

scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)
ctx = ExecutionContext(input_file="example_input_isochem.in",
    scratch_dir=scratch, plots_dir=plots)
system = super_dictionary_creator(ctx, export_json=True,
    filename="input_system.json", super_key_name="system")
hc_data = hard_core_potentials(ctx=ctx, input_data=system, grid_points=5000,
    file_name_prefix="supplied_data_potential_hc.json", export_files=True)
free_energy = total_free_energy(
    ctx=ctx, hc_data=hc_data, system_config=system, export_json=True,
    filenames={"hard_core":"supplied_data_free_energy_hard_core.json",
               "mean_field":"supplied_data_free_energy_mean_field.json",
               "ideal":"supplied_data_free_energy_ideal.json",
               "hybrid":"supplied_data_free_energy_hybrid.json"},
)

d_a, d_b = 1.0271066034594893, 0.9528886708198382
mu_values = np.linspace(7.0, 10.0, 10)
density_bounds = np.linspace(5.0, 10.0, 10)
binodal_data = []
for mu, bound in zip(mu_values, density_bounds):
    cfg = copy.deepcopy(system)
    cfg["system"]["solution"]["intrinsic_constraints"]["chemical_potential"]["a"] = float(mu)
    cfg["system"]["solution"]["extrinsic_constraints"]["total_density_bound"] = float(bound)
    value = coexistence_densities_isochem(
        ctx=ctx, config_dict=cfg, fe_res=free_energy, supplied_data=None,
        max_outer_iters=10, tol_outer=1e-3, tol_solver=1e-8, verbose=False,
    )
    if value is None or np.allclose(value["rhos_per_phase"][0], value["rhos_per_phase"][1], atol=1e-6):
        continue
    (rho_a1, rho_b1), (rho_a2, rho_b2) = value["rhos_per_phase"][:2]
    phi_a1 = rho_a1 * (np.pi/6.0) * d_a**3
    phi_b1 = rho_b1 * (np.pi/6.0) * d_b**3
    phi_a2 = rho_a2 * (np.pi/6.0) * d_a**3
    phi_b2 = rho_b2 * (np.pi/6.0) * d_b**3
    binodal_data.append([phi_a1, phi_b1, phi_a2, phi_b2,
                         rho_a1, rho_b1, rho_a2, rho_b2])

np.savetxt(
    "binodal_data.txt", np.asarray(binodal_data), fmt="%.10f",
    header=("phi_a_phase1 phi_b_phase1 phi_a_phase2 phi_b_phase2 "
            "rho_a_phase1 rho_b_phase1 rho_a_phase2 rho_b_phase2"),
)
```

Audit / rectification notes

- rdf_radial and Path are unused in the source and are removed.
- raw_data and total_data are diagnostic exports, not inputs to the coexistence solver.
- The source writes eight numerical columns but labels only four in the header. The cleaned reference snippet corrects the header to match all eight saved values.

21–22.10 Ternary colloid phase response — EMF and SMF folders

Folder(s)	Examples/Phase_coexistence/Ternary_colloid_polymer_mixture/Colloids_phase_response/EMF ; Examples/Phase_coexistence/Ternary_colloid_polymer_mixture/Colloids_phase_response/SMF
Driver(s)	EMF/example_workflow_automation_phase_response_colloids.py ; SMF/example_workflow_automation_phase_response_colloids.py
Scientific purpose	Sweep the globally imposed colloid density from 0.03 to 0.4 while solving two-phase canonical coexistence. EMF/SMF are selected entirely by their respective input files.
Local files	colloids_delta_rho_vs_density_12_24_simulation.txt, example_input_isocore.in,

	example_workflow_automation_phase_response_colloids.py | colloids_delta_rho_vs_density_12_24_simulation.txt, example_input_isocore.in, example_workflow_automation_phase_response_colloids.py
Support-data role	Each folder also ships colloids_delta_rho_vs_density_12_24_simulation.txt as comparison/reference data; the driver does not read it.
Executed flow	parse input -> hard-core preprocessing -> total_free_energy -> update species_fraction[c] -> coexistence_densities_isocore -> coexistence_density_scan.txt.

Active input system — Phase_coexistence/Ternary_colloid_polymer_mixture/Colloids_phase_response/EMF/example_input_isocore.in

```
species = a, b, c
interactions primary: aa: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.0
interactions primary: ab: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.5
interactions primary: ac: type = mie, m = 24, n = 12, sigma = 1.000, cutoff = 3.5, epsilon = 0.3104
interactions primary: bb: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.0
interactions primary: bc: type = lj,  sigma = 1.0, cutoff = 1.1224, epsilon = 1.0
interactions primary: cc: type = lj,  sigma = 1.0, cutoff = 1.1224, epsilon = 1.0
interactions secondary: ac: type = gs,  sigma = 1, cutoff = 3.2, epsilon=-0.25
rdf closure aa = hnc
rdf closure ab = hnc
rdf closure ac = py
rdf closure bb = hnc
rdf closure bc = py
rdf closure cc = py
rdf alpha_max =  0.1
rdf max_iteration  = 100000
rdf tolerence = 0.000001
rdf beta = 1.0
free_energy ensemble = isocore
free_energy mode = standard
free_energy method = emf
free_energy hs_functional = rosenfeld
free_energy integrated_strength_kernel = uniform
free_energy supplied_data = no
solution intrinsic_constraints: species_fraction: a = 0.27
solution intrinsic_constraints: species_fraction: b = 0.27
solution intrinsic_constraints: species_fraction: c = 0.09
solution extrinsic_constraints: number_of_phases = 2
solution extrinsic_constraints: heterogeneous_pair = ab
solution extrinsic_constraints: total_density_bound = 2.0
```

Active input system — Phase_coexistence/Ternary_colloid_polymer_mixture/Colloids_phase_response/SMF/example_input_isocore.in

```
species = a, b, c
interactions primary: aa: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.0
interactions primary: ab: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.5
interactions primary: ac: type = mie, m = 24, n = 12, sigma = 1.000, cutoff = 3.5, epsilon = 0.3104
interactions primary: bb: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.0
interactions primary: bc: type = lj,  sigma = 1.0, cutoff = 1.1224, epsilon = 1.0
interactions primary: cc: type = lj,  sigma = 1.0, cutoff = 1.1224, epsilon = 1.0
interactions secondary: ac: type = gs,  sigma = 1, cutoff = 3.2, epsilon=-0.33
rdf closure aa = hnc
rdf closure ab = hnc
rdf closure ac = py
rdf closure bb = hnc
rdf closure bc = py
rdf closure cc = py
rdf alpha_max =  0.1
rdf max_iteration  = 100000
rdf tolerence = 0.000001
rdf beta = 1.0
free_energy ensemble = isocore
free_energy mode = standard
free_energy method = smf
free_energy hs_functional = rosenfeld
free_energy integrated_strength_kernel = uniform
free_energy supplied_data = no
solution intrinsic_constraints: species_fraction: a = 0.27
```

```
solution intrinsic_constraints: species_fraction: b = 0.27
solution intrinsic_constraints: species_fraction: c = 0.09
solution extrinsic_constraints: number_of_phases = 2
solution extrinsic_constraints: heterogeneous_pair = ab
solution extrinsic_constraints: total_density_bound = 2.0
```

Cleaned workflow shared by the EMF and SMF drivers

```
import numpy as np
from tqdm import tqdm
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.generators.potential_splitter.hc import hard_core_potentials
from cdft_solver.calculators.total_free_energy.total_free_energy import total_free_energy
from cdft_solver.calculators.coexistence_densities.calculator_coexistence_densities_isocore import
coexistence_densities_isocore

scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)
ctx = ExecutionContext(input_file="example_input_isocore.in",
    scratch_dir=scratch, plots_dir=plots)
system = super_dictionary_creator(ctx, export_json=True,
    filename="input_system.json", super_key_name="system")
hc_data = hard_core_potentials(ctx=ctx, input_data=system, grid_points=5000,
    file_name_prefix="supplied_data_potential_hc.json", export_files=True)
free_energy = total_free_energy(
    ctx=ctx, hc_data=hc_data, system_config=system, export_json=True,
    filenames={"hard_core":"supplied_data_free_energy_hard_core.json",
               "mean_field":"supplied_data_free_energy_mean_field.json",
               "ideal":"supplied_data_free_energy_ideal.json",
               "hybrid":"supplied_data_free_energy_hybrid.json"},
)

rho_vals = np.linspace(0.03, 0.4, 10)
output_file = scratch / "coexistence_density_scan.txt"
with open(output_file, "w") as f:
    f.write("# rho_colloid phase1_rho_a phase1_rho_b phase1_rho_c "
            "phase2_rho_a phase2_rho_b phase2_rho_c fraction1 fraction2\n")

for rho in tqdm(rho_vals, desc="Scanning densities"):
    system["system"]["solution"]["intrinsic_constraints"]["species_fraction"]["c"] = float(rho)
    value = coexistence_densities_isocore(
        ctx=ctx, config_dict=system, fe_res=free_energy, supplied_data=None,
        max_outer_iters=10, tol_outer=1e-3, tol_solver=1e-8, verbose=False,
    )
    phase1, phase2 = value["rhos_per_phase"][:2]
    frac1 = value["fractions"][0]
    frac2 = 1.0 - frac1
    with open(output_file, "a") as f:
        f.write(f"{rho:.8f} " + " ".join(f"{x:.8f}" for x in [*phase1, *phase2, frac1, frac2]) + "\n")
```

Audit / rectification notes

- Path is unused. raw_data and total_data are not consumed by the coexistence loop.
- The EMF and SMF drivers are operationally the same; the model choice and ac secondary interaction strength differ in the .in files.

21–22.11 Ternary polymer phase response — EMF and SMF folders

Folder(s)	Examples/Phase_coexistence/Ternary_colloid_polymer_mixture/ Polymers_phase_response/EMF ; Examples/Phase_coexistence/Ternary_colloid_polymer_mixture/Polymers_phase_response/SMF
Driver(s)	EMF/example_workflow_automation_phase_response_polymers.py ; SMF/example_workflow_automation_phase_response_polymers.py
Scientific purpose	Sweep the total polymer density from 0.54 to 0.8, split it equally between a and b, and solve the two-phase canonical response.
Local files	delta_rho_vs_density_12_24.txt, example_input_isocore.in, example_workflow_automation_phase_response_polymers.py |

	delta_rho_vs_density_12_24.txt, example_input_isocore.in, example_workflow_automation_phase_response_polymers.py
Support-data role	Each folder ships delta_rho_vs_density_12_24.txt as comparison/reference data; it is not read by the main driver.
Executed flow	parse input -> hard-core preprocessing -> total_free_energy -> update species_fraction[a,b] -> coexistence_densities_isocore -> polymer_coexistence_scan.txt.

Active input system — Phase_coexistence/Ternary_colloid_polymer_mixture/Polymers_phase_response/EMF/example_input_isocore.in

```
species = a, b, c
interactions primary: aa: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.0
interactions primary: ab: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.5
interactions primary: ac: type = mie, m = 24, n = 12, sigma = 1.000, cutoff = 3.5, epsilon = 0.3104
interactions primary: bb: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.0
interactions primary: bc: type = lj,  sigma = 1.0, cutoff = 1.1224, epsilon = 1.0
interactions primary: cc: type = lj,  sigma = 1.0, cutoff = 1.1224, epsilon = 1.0
interactions secondary: ac: type = gs,  sigma = 1, cutoff = 3.2, epsilon=-0.25
rdf closure aa = hnc
rdf closure ab = hnc
rdf closure ac = py
rdf closure bb = hnc
rdf closure bc = py
rdf closure cc = py
rdf alpha_max =  0.1
rdf max_iteration  = 100000
rdf tolerence = 0.000001
rdf beta = 1.0
free_energy ensemble = isocore
free_energy mode = standard
free_energy method = emf
free_energy hs_functional = rosenfeld
free_energy integrated_strength_kernel = uniform
free_energy supplied_data = no
solution intrinsic_constraints: species_fraction: a = 0.27
solution intrinsic_constraints: species_fraction: b = 0.27
solution intrinsic_constraints: species_fraction: c = 0.09
solution extrinsic_constraints: number_of_phases = 2
solution extrinsic_constraints: heterogeneous_pair = ab
solution extrinsic_constraints: total_density_bound = 2.0
```

Active input system — Phase_coexistence/Ternary_colloid_polymer_mixture/Polymers_phase_response/SMF/example_input_isocore.in

```
species = a, b, c
interactions primary: aa: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.0
interactions primary: ab: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.5
interactions primary: ac: type = mie, m = 24, n = 12, sigma = 1.000, cutoff = 3.5, epsilon = 0.3104
interactions primary: bb: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.0
interactions primary: bc: type = lj,  sigma = 1.0, cutoff = 1.1224, epsilon = 1.0
interactions primary: cc: type = lj,  sigma = 1.0, cutoff = 1.1224, epsilon = 1.0
interactions secondary: ac: type = gs,  sigma = 1, cutoff = 3.2, epsilon=-0.33
rdf closure aa = hnc
rdf closure ab = hnc
rdf closure ac = py
rdf closure bb = hnc
rdf closure bc = py
rdf closure cc = py
rdf alpha_max =  0.1
rdf max_iteration  = 100000
rdf tolerence = 0.000001
rdf beta = 1.0
free_energy ensemble = isocore
free_energy mode = standard
free_energy method = smf
free_energy hs_functional = rosenfeld
free_energy integrated_strength_kernel = uniform
free_energy supplied_data = no
solution intrinsic_constraints: species_fraction: a = 0.27
```

```
solution intrinsic_constraints: species_fraction: b = 0.27
solution intrinsic_constraints: species_fraction: c = 0.09
solution extrinsic_constraints: number_of_phases = 2
solution extrinsic_constraints: heterogeneous_pair = ab
solution extrinsic_constraints: total_density_bound = 2.0
```

Cleaned workflow shared by the EMF and SMF drivers

```
import numpy as np
from tqdm import tqdm
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.generators.potential_splitter.hc import hard_core_potentials
from cdft_solver.calculators.total_free_energy.total_free_energy import total_free_energy
from cdft_solver.calculators.coexistence_densities.calculator_coexistence_densities_isocore import
coexistence_densities_isocore

scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)
ctx = ExecutionContext(input_file="example_input_isocore.in",
    scratch_dir=scratch, plots_dir=plots)
system = super_dictionary_creator(ctx, export_json=True,
    filename="input_system.json", super_key_name="system")
hc_data = hard_core_potentials(ctx=ctx, input_data=system, grid_points=5000,
    file_name_prefix="supplied_data_potential_hc.json", export_files=True)
free_energy = total_free_energy(
    ctx=ctx, hc_data=hc_data, system_config=system, export_json=True,
    filenames={"hard_core":"supplied_data_free_energy_hard_core.json",
               "mean_field":"supplied_data_free_energy_mean_field.json",
               "ideal":"supplied_data_free_energy_ideal.json",
               "hybrid":"supplied_data_free_energy_hybrid.json"},
)

rho_totals = np.linspace(0.54, 0.8, 10)
output_file = scratch / "polymer_coexistence_scan.txt"
with open(output_file, "w") as f:
    f.write("# rho_polymer_total rho_a_input rho_b_input "
            "phase1_rho_a phase1_rho_b phase1_rho_c "
            "phase2_rho_a phase2_rho_b phase2_rho_c fraction1 fraction2\n")

for rho_total in tqdm(rho_totals, desc="Scanning polymer densities"):
    rho_a = rho_b = 0.5 * rho_total
    sf = system["system"]["solution"]["intrinsic_constraints"]["species_fraction"]
    sf["a"], sf["b"] = float(rho_a), float(rho_b)
    value = coexistence_densities_isocore(
        ctx=ctx, config_dict=system, fe_res=free_energy, supplied_data=None,
        max_outer_iters=10, tol_outer=1e-3, tol_solver=1e-8, verbose=False,
    )
    phase1, phase2 = value["rhos_per_phase"][:2]
    frac1 = value["fractions"][0]
    frac2 = 1.0 - frac1
    with open(output_file, "a") as f:
        vals = [rho_total, rho_a, rho_b, *phase1, *phase2, frac1, frac2]
        f.write(" ".join(f"{x:.8f}" for x in vals) + "\n")
```

Audit / rectification notes

- Path is unused. raw_data and total_data are not consumed by the coexistence loop.
- The EMF and SMF input definitions are shared with the corresponding colloid-response folders: there are 21 .in files in the archive but 19 unique input definitions.

21–22.12 Inhomogeneous_profile/Ternary_colloid_polymer_mixture

Folder(s)	Examples/Inhomogeneous_profile/Ternary_colloid_polymer_mixture
Driver(s)	example_workflow_isocore_planer.py
Scientific purpose	Compute a one-dimensional confined ternary density profile in the configured box with external wall species d.
Local files	example_input_isocore.in, example_workflow_isocore_planer.py
Support-data role	No external data file is required. The complete confinement geometry, external species, wall interactions, planar grid, and profile iteration controls are all defined in the input.

Executed flow	parse input -> one_d_profile_iterator_box(export_json=True, export_plots=True).

Active input system — Inhomogeneous_profile/Ternary_colloid_polymer_mixture/example_input_isocore.in

```
task = profile_planer
ensemble = isocore
species = a, b, c
interactions primary: aa: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.0
interactions primary: ab: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.5
interactions primary: ac: type = mie, m = 36, n = 18, sigma = 1.000, cutoff = 3.5, epsilon = 1.6358
interactions primary: bb: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.0
interactions primary: bc: type = lj,  sigma = 1.0, cutoff = 1.1224, epsilon = 1.0
interactions primary: cc: type = lj,  sigma = 1.0, cutoff = 1.1224, epsilon = 1.0
interactions secondary: ac: type = gs,  sigma = 1, cutoff = 3.2, epsilon=0.1
rdf closure aa = hnc
rdf closure ab = hnc
rdf closure ac = py
rdf closure bb = hnc
rdf closure bc = py
rdf closure cc = py
rdf alpha_max =  0.01
rdf max_iteration  = 100000
rdf tolerence = 0.00001
rdf beta = 1.0
rdf r_max  =  5.0
rdf n_points =  200
free_energy ensemble = isocore
free_energy mode = standard
free_energy method = emf
free_energy integrated_strength_kernel = meanfield
free_energy supplied_data = no
solution intrinsic_constraints: species_fraction: a = 0.27
solution intrinsic_constraints: species_fraction: b = 0.27
solution intrinsic_constraints: species_fraction: c = 0.09
solution extrinsic_constraints: number_of_phases = 2
solution extrinsic_constraints: heterogeneous_pair = None
solution extrinsic_constraints: total_density_bound = 2.0
external species = d
external interaction ad: type = custom_3,  sigma = 1.0, cutoff = 2.5, epsilon = 0.01
external interaction bd: type = custom_3,  sigma = 1.0, cutoff = 2.5, epsilon = 0.01
external interaction cd: type = custom_3,  sigma = 1.0, cutoff = 2.5, epsilon = 0.01
external d position = 0.0,  0.0,  0.0
external d position = 60.0, 0.0,  0.0
space_confinement_parameters space_properties: dimension = 1
space_confinement_parameters space_properties: space_confinement = abox
space_confinement_parameters box_properties: box_length = 60, 5, 0
space_confinement_parameters box_properties: box_points = 200, 50, 0
planer_rdf space_properties: dimension = 2
planer_rdf space_properties: space_confinement = abox
planer_rdf tolerance = 0.001
planer_rdf box_properties: box_length = 60, 5, 0
planer_rdf box_properties: box_points = 200, 50, 0
profile iteration_max  =   3000
profile vij_interval = 1000
profile tolerance  = 0.0000000001
profile alpha_mixing_max = 0.001
profile log_period = 10
```

Cleaned reachable workflow

```
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.calculators.one_d_profile_iterator.one_d_profile_iterator_box import
one_d_profile_iterator_box

scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)
ctx = ExecutionContext(input_file="example_input_isocore.in",
    scratch_dir=scratch, plots_dir=plots)
system = super_dictionary_creator(
    ctx, export_json=True, filename="input_system.json", super_key_name="system"
)
one_d_profile_iterator_box(
    ctx=ctx,
    config_dict=system,
    export_json=True,
```

```
    export_plots=True,
)
```

Audit / rectification notes

- Path is imported but unused.
- The source terminates with exit(0) immediately after the profile call. No physical computation exists after the exit, so the exit statement itself is unnecessary in a reference workflow and is omitted.

21–22.13 Phase_finder

Folder(s)	Examples/Phase_finder
Driver(s)	example_workflow_phase_finder_thermodynamic_quantities_automation.py
Scientific purpose	Evaluate a canonical one-component thermodynamic state from the supplied density input. The source also contains a later binary phi_a/phi_b surface template.
Local files	example_input_phase_finder_isocore.in, example_workflow_phase_finder_thermodynamic_quantities_automation.py, plotter_3d_surface.py, plotter_3d_surface_coexistence.py, plotter_3d_surface_in_phi.py
Support-data role	plotter_3d_surface.py, plotter_3d_surface_coexistence.py, and plotter_3d_surface_in_phi.py are separate plotting helpers.
Executed flow	valid supplied workflow: parse input -> hard-core preprocessing -> total_free_energy -> evaluate_canonical_state -> state_from_density.json.

Active input system — Phase_finder/example_input_phase_finder_isocore.in

```
species = a
interactions primary: aa: type = lj,  sigma = 1.0,  cutoff = 3.5,  epsilon = 1.0
rdf closure aa = hnc
rdf alpha_max =  0.01
rdf max_iteration  = 100000
rdf tolerence = 0.000001
rdf beta = 1.0
free_energy mode = standard
free_energy method = smf
free_energy integrated_strength_kernel = uniform
free_energy supplied_data = no
solution extrinsic_constraints: density: a = 0.15
```

Cleaned runnable workflow for the supplied one-species input

```
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.generators.potential_splitter.hc import hard_core_potentials
from cdft_solver.calculators.total_free_energy.total_free_energy import total_free_energy
from cdft_solver.calculators.phase_finder.thermodynamic_potentials_isocore import evaluate_canonical_state

scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)
ctx = ExecutionContext(input_file="example_input_phase_finder_isocore.in",
    scratch_dir=scratch, plots_dir=plots)
system = super_dictionary_creator(ctx, export_json=True,
    filename="input_system.json", super_key_name="system")
hc_data = hard_core_potentials(ctx=ctx, input_data=system, grid_points=5000,
    file_name_prefix="supplied_data_potential_hc.json", export_files=True)
free_energy = total_free_energy(
    ctx=ctx, hc_data=hc_data, system_config=system, export_json=True,
    filenames={"hard_core":"supplied_data_free_energy_hard_core.json",
               "mean_field":"supplied_data_free_energy_mean_field.json",
               "ideal":"supplied_data_free_energy_ideal.json",
               "hybrid":"supplied_data_free_energy_hybrid.json"},
)

result = evaluate_canonical_state(
    ctx=ctx,
    config_dict=system,
    fe_res=free_energy,
    supplied_data=None,
    export=True,
```

```
    output_file="state_from_density.json",
    verbose=True,
)
print(result)
```

Audit / rectification notes

- The source input defines only species a. The later reachable grid code writes densities for a and b and accesses two chemical potentials; it is therefore incompatible with the supplied one-component input and is not presented as a runnable example here.
- That binary grid occurs before exit(0), so it is not “unreachable”; it is excluded because of an input/driver mismatch, not because of control flow.
- The final exit(0) has no useful computation after it and is omitted. Duplicate NumPy/JSON imports and Path are also omitted.

21–22.14 Energy_landscape

Folder(s)	Examples/Energy_landscape
Driver(s)	example_workflow_automation_free_energy_landscape.py
Scientific purpose	First solve ternary two-phase coexistence, then evaluate constrained two-phase free-energy surfaces while preserving the imposed global material balances.
Local files	converter.py, example_input_isocore.in, example_workflow_automation_free_energy_landscape.py, output_XY_data.txt, output_YZ_data.txt
Support-data role	converter.py is an independent helper. output_XY_data.txt and output_YZ_data.txt are shipped reference/generated data, not input configuration.
Executed flow	parse input -> hard-core preprocessing -> total_free_energy -> coexistence_densities_isocore -> repeated evaluate_canonical_state over paired XY/YZ density states -> output_XY_data.txt/output_YZ_data.txt -> plotting.

Active input system — Energy_landscape/example_input_isocore.in

```
species = a, b, c
interactions primary: aa: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.0
interactions primary: ab: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.5
interactions primary: ac: type = gs,  sigma = 1.000, cutoff = 3.5, epsilon = -0.599
interactions primary: bb: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 2.0
interactions primary: bc: type = gs,  sigma = 1.414, cutoff = 3.5, epsilon = 0.0
interactions primary: cc: type = hc,  sigma = 1.0, cutoff = 5.0, epsilon = 2.0
interactions secondary: ac: type = ghc,  sigma = 1.0, cutoff = 3.2, epsilon=2
interactions secondary: bc: type = ghc,  sigma = 1.0, cutoff = 3.2, epsilon=2
rdf closure aa = hnc
rdf closure ab = hnc
rdf closure ac = py
rdf closure bb = hnc
rdf closure bc = py
rdf closure cc = py
rdf alpha_max =  0.1
rdf max_iteration  = 100000
rdf tolerence = 0.000001
rdf beta = 1.0
free_energy ensemble = isocore
free_energy mode = standard
free_energy method = smf
free_energy hs_functional = rosenfeld
free_energy integrated_strength_kernel = uniform
free_energy supplied_data = no
solution intrinsic_constraints: species_fraction: a = 0.18
solution intrinsic_constraints: species_fraction: b = 0.18
solution intrinsic_constraints: species_fraction: c = 0.09
solution extrinsic_constraints: number_of_phases = 2
solution extrinsic_constraints: heterogeneous_pair = ab
solution extrinsic_constraints: total_density_bound = 2.0
solution extrinsic_constraints: density: a = 0.18
solution extrinsic_constraints: density: b = 0.18
solution extrinsic_constraints: density: c = 0.09
```

Cleaned computational core (plotting-only tail omitted)

```
import numpy as np
from tqdm import tqdm
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.generators.potential_splitter.hc import hard_core_potentials
from cdft_solver.calculators.total_free_energy.total_free_energy import total_free_energy
from cdft_solver.calculators.phase_finder.thermodynamic_potentials_isocore import evaluate_canonical_state
from cdft_solver.calculators.coexistence_densities.calculator_coexistence_densities_isocore import
coexistence_densities_isocore

scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)
ctx = ExecutionContext(input_file="example_input_isocore.in",
    scratch_dir=scratch, plots_dir=plots)
system = super_dictionary_creator(ctx, export_json=True,
    filename="input_system.json", super_key_name="system")
hc_data = hard_core_potentials(ctx=ctx, input_data=system, grid_points=5000,
    file_name_prefix="supplied_data_potential_hc.json", export_files=True)
free_energy = total_free_energy(
    ctx=ctx, hc_data=hc_data, system_config=system, export_json=True,
    filenames={"hard_core":"supplied_data_free_energy_hard_core.json",
               "mean_field":"supplied_data_free_energy_mean_field.json",
               "ideal":"supplied_data_free_energy_ideal.json",
               "hybrid":"supplied_data_free_energy_hybrid.json"},
)
coex = coexistence_densities_isocore(
    ctx=ctx, config_dict=system, fe_res=free_energy, supplied_data=None,
    max_outer_iters=10, tol_outer=1e-3, tol_solver=1e-8, verbose=True,
)
phase1, phase2 = coex["rhos_per_phase"][:2]
p = coex["fractions"][0]

def energy_free(rho_a, rho_b, rho_c):
    density = system["system"]["solution"]["extrinsic_constraints"]["density"]
    density["a"], density["b"], density["c"] = float(rho_a), float(rho_b), float(rho_c)
    state = evaluate_canonical_state(
        ctx=ctx, config_dict=system, fe_res=free_energy,
        supplied_data=None, export=False, verbose=False,
    )
    return state["free_energy_density"]

# XY plane: phase-pair densities are linked by the global material balance.
x_sum = p*phase1[0] + (1-p)*phase2[0]
y_sum = p*phase1[1] + (1-p)*phase2[1]
x1_vals = np.linspace(1e-4, max(phase1[0], phase2[0], 2.005*x_sum), 100)
y1_vals = np.linspace(1e-4, max(phase1[1], phase2[1], 2.005*y_sum), 100)
export_xy = []
for x1 in tqdm(x1_vals, desc="Computing XY surface"):
    x2 = (x_sum - p*x1)/(1-p)
    if x2 < 0: continue
    for y1 in y1_vals:
        y2 = (y_sum - p*y1)/(1-p)
        if y2 < 0: continue
        f = p*energy_free(x1, y1, phase1[2]) + (1-p)*energy_free(x2, y2, phase2[2])
        if f < 2.5:
            export_xy.append([x1, y1, x2, y2, f])
np.savetxt("output_XY_data.txt", export_xy,
    fmt="%.8f", header="X1 Y1 X2 Y2 TotalEnergy")

# The source repeats the same construction in the YZ plane and then plots both surfaces.
# That plotting-only block is intentionally omitted from this cleaned computational core.
```

Audit / rectification notes

- Path is unused. raw_data and total_data are produced but not consumed by the free-energy landscape calculation.
- The source contains a long Matplotlib presentation block. It is intentionally omitted from the cleaned computational core because it does not alter the physical state calculation.
- converter.py is not imported or called by the driver.

21–22.15 Dispersion_relation

Folder(s)	Examples/Dispersion_relation
Driver(s)	example_workflow_dispersion_relation.py
Scientific purpose	Evaluate the dispersion relation of the configured two-component 2D fluid for densities [1,1] and diffusivities [1,1].
Local files	example_input_dispersion.in, example_workflow_dispersion_relation.py, plotter_dispersion_relation.py, plotter_rdf_relation.py
Support-data role	plotter_dispersion_relation.py and plotter_rdf_relation.py are separate plotting helpers.
Executed flow	parse input -> dispersion_relation.

Active input system — Dispersion_relation/example_input_dispersion.in

```
species = a, b
interactions primary: aa: type = nvrn,  sigma = 1.0, cutoff = 3.5, epsilon = 42, n = 3, m_i = 1.0, m_j = 1.0
interactions primary: bb: type = nvrn,  sigma = 1.0, cutoff = 3.5, epsilon = 42, n = 3, m_i = 0.025, m_j =
0.025
interactions primary: ab: type = nvrn,  sigma = 1.0, cutoff = 3.5, epsilon = 42, n = 3, m_i = 1.0, m_j =
0.025
rdf closure aa = hnc
rdf closure ab = hnc
rdf closure bb = hnc
rdf alpha_max =  0.001
rdf dimension  = 2d
rdf max_iteration  = 1000000
rdf tolerance = 0.00001
rdf beta = 1.0
rdf r_max  =  10.0
rdf n_points =  4000
```

Cleaned reachable workflow

```
import numpy as np
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.calculators.dispersion_relation.dispersion_relation import dispersion_relation

scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)
ctx = ExecutionContext(input_file="example_input_dispersion.in",
    scratch_dir=scratch, plots_dir=plots)
system = super_dictionary_creator(
    ctx, export_json=True, filename="input_system.json", super_key_name="system"
)

densities = np.array([1.0, 1.0])
diffusivity = np.array([1.0, 1.0])
dispersion_relation(
    ctx=ctx,
    rdf_config=system,
    densities=densities,
    diffusivity=diffusivity,
    supplied_data=None,
    export=True,
    plot=True,
    filename_prefix="dispersion",
)
```

Audit / rectification notes

- process_supplied_data is called in the source even though the input has no supplied_data block; the returned value is ignored and dispersion_relation is called with supplied_data=None. That stage is removed.
- Path, rdf_planer, rdf_radial, boltzmann_inversion_advance, and c_analysis are imported but unused.

21–22.16 Second_virial_coefficient_analysis

Folder(s)	Examples/Second_virial_coefficient_analysis

Driver(s)	example_workflow_virial_analysis.py
Scientific purpose	Compute a reference second virial coefficient, then calibrate the strength of a second interaction model to a chosen target B2.
Local files	example_input_virial_analysis.in, example_input_virial_analysis_calibration.in, example_workflow_virial_analysis.py
Support-data role	No external numerical data files are required.
Executed flow	parse reference -> second_virial -> parse calibration input -> choose B2_target -> second_virial_scale_calibration.

Active input system — Second_virial_coefficient_analysis/example_input_virial_analysis.in

```
species = a
interactions primary: aa: type = lj,  sigma = 1.0, cutoff = 6, epsilon = 0.15
virial beta = 1.0
```

Active input system — Second_virial_coefficient_analysis/example_input_virial_analysis_calibration.in

```
species = a
interactions primary: aa: type = mie,  sigma = 1.0, n = 48, m = 24, cutoff = 6, epsilon = 1.0
virial beta = 1.0
```

Cleaned reachable workflow

```
from cdft_solver.utils import ExecutionContext, create_unique_scratch_dir
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator
from cdft_solver.calculators.virial_analysis.second_virial import second_virial
from cdft_solver.calculators.virial_analysis.second_virial_scale_calibration import
second_virial_scale_calibration

scratch = create_unique_scratch_dir()
plots = scratch / "plots"
plots.mkdir(exist_ok=True)

ctx_ref = ExecutionContext(input_file="example_input_virial_analysis.in",
    scratch_dir=scratch, plots_dir=plots)
system_ref = super_dictionary_creator(ctx=ctx_ref, export_json=True,
    filename="input_system.json", super_key_name="system")
B2_computed, strength = second_virial(
    ctx=ctx_ref, virial_config=system_ref, on="raw", export=True,
    filename_prefix="second_virial_coefficient",
    r_max_factor=12.0, nr=8192, n_lambda=128,
)

ctx_target = ExecutionContext(input_file="example_input_virial_analysis_calibration.in",
    scratch_dir=scratch, plots_dir=plots)
system_target = super_dictionary_creator(ctx=ctx_target, export_json=True,
    filename="input_system_target.json", super_key_name="system")

# The shipped example explicitly chooses this target instead of B2_computed.
B2_target = [[-0.125]]
result = second_virial_scale_calibration(
    ctx=ctx_target, virial_config=system_target, B2_target=B2_target,
    on="raw", export=True, filename_prefix="second_virial_coefficient",
    r_max_factor=12.0, nr=8192, beta_scale=1.0,
)
print(result)
```

Audit / rectification notes

- process_supplied_data is a no-op for both supplied inputs and its returned value is unused, so it is removed.
- The source prints the computed B2 and then overwrites b2t with [[-0.125]]. The cleaned snippet makes this explicit by naming the computed result B2_computed and the chosen calibration value B2_target.
- NumPy and Path are unused in the original driver.

21–22.17 What was deliberately removed from the reference workflows

Across the archive, the most common sources of apparent complexity are imports left over from earlier experiments, optional potential exports, and data-processing calls whose results are not actually passed forward. Removing these from the reference snippets does not change the package source; it clarifies the minimum path a reader needs to understand and reproduce the physical calculation.

- Unused imports such as pathlib.Path, rdf_planer/rdf_radial in unrelated drivers, and inversion/analysis modules that are never called.
- raw_potentials and total_potentials branches when their returned dictionaries are not consumed by the subsequent EOS/coexistence/free-energy calculation. These remain useful optional diagnostics and are documented elsewhere in the manual.
- process_supplied_data calls in structural, dispersion, and virial examples when the input contains no supplied_data block and the returned object is discarded.
- free_energy_exporter calls from the minimal computational snippets. They are optional persistence steps; total_free_energy already supplies the in-memory object used by the state/coexistence routines.
- All statements after an unconditional top-level exit(). The LJ isochemical RDF block is the concrete example in this archive.
- Standalone plotter_*.py, converter.py, and potential_extractor.py files. They are independent helpers and are not imported by the main drivers.

PART V — Source Reference, Outputs, and Package Maintenance

23. Module-by-Module Reference

The following catalogue is generated from the Python source tree. “Demonstrated” means the module appears directly in a bundled example_workflow_*.py driver or in its immediate data path; other modules are included for developer orientation. The example-workflow descriptions in Part III additionally distinguish imports from functions that are actually reached and used.

cdft_solver/calculators/c_2_analysis/c_analysis.py

Structural/reference c(r), sigma, B2, G(r), Delta-c analysis.

Callable	Source description / role
solve_oz_kspace(h_k, densities, eps)	Solve multi-component OZ equation in k-space:
solve_oz_matrix(c_r, r, densities)	
multi_component_oz_solver_alpha(r, pair_closures, densities, u_matrix, sigma_matrix, c_update_flag, c_initial, n_iter, tol, alpha_rdf_max, gamma_initial)	Multi-component OZ solver with adaptive alpha-mixing
wca_split(r, u)	Returns repulsive part of u(r) using WCA splitting.
detect_sigma_from_gr(r, g, g_tol, min_width)	Detect hard-core diameter ONLY if g(r) is strictly zero
boltzmann_potential_from_gr(g, beta, g_min)	u(r) = -ln g(r) with numerical protection
detect_first_minimum_near_core(r, u_ij, sigma)	Detect the first local minimum of u(r) after the hard core.
find_key_recursive(d, key)	

plot_u_matrix(r, u_matrix, species, outdir, filename)	Plot and export u_ij(r) for all species pairs.
c_analysis(ctx, rdf_config, supplied_data, densities, filename_prefix, export_plot, export_json)	Multistate Isotropic Boltzmann / IBI inversion

cdft_solver/calculators/coexistence_densities/calculator_coexistence_densities_isochem.py

Multiphase isochemical coexistence solver.

Callable	Source description / role
deep_get(data, key, default)	Recursively search for key in nested dicts/lists.
deep_get_all(data, key)	Return all matches for key in nested structure.
build_thermodynamics_from_fe_res(fe_res)	Given a free-energy result dictionary (from JSON),
coexistence_densities_isochem(ctx, config_dict, fe_res, supplied_data, max_outer_iters, tol_outer, tol_solver, verbose)	Iso-chemical coexistence solver using ONLY a supplied dictionary.

cdft_solver/calculators/coexistence_densities/calculator_coexistence_densities_isocore.py

Canonical/isocore coexistence solver.

Callable	Source description / role
deep_get(data, key, default)	Recursively search for key in nested dicts/lists.
deep_get_all(data, key)	Return all matches for key in nested structure.
build_thermodynamics_from_fe_res(fe_res)	Given a free-energy result dictionary (from JSON),
coexistence_densities_isocore(ctx, config_dict, fe_res, supplied_data, max_outer_iters, tol_outer, tol_solver, verbose)	Iso-chemical coexistence solver using ONLY a supplied dictionary.

cdft_solver/calculators/coexistence_densities/kernel_generator.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
build_strength_kernel(ctx, config, supplied_data, densities, kernel_type)	Dispatcher for integrated strength kernel.

cdft_solver/calculators/dispersion_relation/dispersion_relation.py

Dispersion-relation calculation from structural input.

Callable	Source description / role
dispersion_relation(ctx, rdf_config, densities, diffusivity, supplied_data, export, plot, filename_prefix)	Compute dispersion relation ω(k) using RDF input.

cdft_solver/calculators/free_energy_hard_core/hard_core_bulk_rosenfeld.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
hard_core_bulk_rosenfeld(ctx, hc_data, export_json, filename)	Computes the hard-core contribution to the free energy for a multi-species system.

cdft_solver/calculators/free_energy_hard_core/hard_core_bulk_whitebeer.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
hard_core_bulk_whitebeer(ctx, hc_data, export_json, filename)	White Bear FMT (true form) for colloids + Free Volume Theory for polymers.

cdft_solver/calculators/free_energy_hard_core/hard_core_planer.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
hard_core_planer(ctx, hc_data, export_json, filename)	Computes the hard-core free energy (FMT-like) for multi-species system

cdft_solver/calculators/free_energy_hybrid/hybrid.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
hybrid(ctx, hc_data, export_json, filename)	Computes hybrid free energy:

cdft_solver/calculators/free_energy_hybrid/hybrid_planer.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
hybrid_planer(ctx)	Computes the hard-core contribution to the free energy for a multi-species system.

cdft_solver/calculators/free_energy_hybrid_lattice/free_energy_lattice.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
free_energy_lattice(ctx, hc_data, export_json, filename)	Computes lattice free energy:

cdft_solver/calculators/free_energy_ideal/ideal.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
ideal(ctx, hc_data, export_json, filename)	Computes the symbolic ideal (entropic) part of the free energy for a multi-species system.

cdft_solver/calculators/free_energy_mean_field/EMF.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
free_energy_EMF(ctx, hc_data, export_json, filename)	

cdft_solver/calculators/free_energy_mean_field/EMF_z.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
free_energy_EMF_z(ctx, hc_data, export_json, filename)	Enhanced Mean-Field (EMF) two-point free-energy kernel:

cdft_solver/calculators/free_energy_mean_field/SMF.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
free_energy_SMF(ctx, hc_data, export_json, filename)	Computes the symbolic mean-field (SMF) free energy for a multi-species system

cdft_solver/calculators/free_energy_mean_field/SMF_z.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
free_energy_SMF_z(ctx, hc_data, export_json, filename)	Standard Mean-Field (SMF) two-point free-energy kernel:

cdft_solver/calculators/free_energy_mean_field/mean_field.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
mean_field(ctx, hc_data, system_config, export_json, filename)	Unified wrapper for symbolic free energy models (EMF, SMF, VOID).

cdft_solver/calculators/free_energy_mean_field/mean_field_planer.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
mean_field_planer(ctx, hc_data, system_config, export_json, filename)	Unified wrapper for symbolic free energy models (EMF, SMF, VOID).

cdft_solver/calculators/free_energy_mean_field/void.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
free_energy_void(ctx, hc_data, export_json, filename)	Computes the symbolic cavity mean-field (CMF) free energy for a multi-species system

cdft_solver/calculators/free_energy_mean_field/void_z.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
free_energy_void_z(ctx, hc_data, export_json, filename)	Cavity Mean-Field (CMF) two-point free-energy kernel:

cdft_solver/calculators/g_alpha_r/delta_c.py

Direct-correlation-difference kernel route.

Callable	Source description / role
solve_oz_kspace(h_k, densities, eps)	Solve multi-component OZ equation in k-space:
plot_u_matrix(r, u_matrix, species, outdir, filename)	Plot and export u_ij(r) for all species pairs.
process_supplied_rdf(supplied_data, species, r_grid)	Uses canonical pair keys like 'AA', 'AB', ...
hankel_forward_dst(f_r, r)	
hankel_inverse_dst(k, Fk, r)	
hankel_transform_matrix_fast(f_r_matrix, r)	
inverse_hankel_transform_matrix_fast(f_k_matrix, k, r)	
solve_oz_matrix(c_r, r, densities)	
multi_component_oz_solver_alpha(r, pair_closures, densities, u_matrix, sigma_matrix, c_update_flag, c_initial, n_iter, tol, alpha_rdf_max, gamma_initial)	Multi-component OZ solver with adaptive alpha-mixing
apply_adaptive_ylim(ax, ydata, limit, clip)	
plot_matrix_quantity(r, quantity, u_matrix, species, title_prefix, filename, plots_dir)	
wca_split(r, u)	Returns repulsive part of u(r) using WCA splitting.
detect_sigma_from_gr(r, g, g_tol, min_width)	Detect hard-core diameter ONLY if g(r) is strictly zero
boltzmann_potential_from_gr(g, beta, g_min)	u(r) = -ln g(r) with numerical protection

detect_first_minimum_near_core(r, u_ij, sigma)	
find_key_recursive(d, key)	
delta_c_alpha(ctx, rdf_config, densities, supplied_data, export, plot, filename_prefix)	Fully dictionary-driven isotropic RDF solver

cdft_solver/calculators/g_alpha_r/rdf_alpha_r.py

Coupling-parameter RDF/interaction kernel construction.

Callable	Source description / role
solve_oz_kspace(h_k, densities, eps)	Solve multi-component OZ equation in k-space:
plot_u_matrix(r, u_matrix, species, outdir, filename)	Plot and export u_ij(r) for all species pairs.
process_supplied_rdf(supplied_data, species, r_grid)	Uses canonical pair keys like 'AA', 'AB', ...
hankel_forward_dst(f_r, r)	
hankel_inverse_dst(k, Fk, r)	
hankel_transform_matrix_fast(f_r_matrix, r)	
inverse_hankel_transform_matrix_fast(f_k_matrix, k, r)	
solve_oz_matrix(c_r, r, densities)	
multi_component_oz_solver_alpha(r, pair_closures, densities, u_matrix, sigma_matrix, c_update_flag, c_initial, n_iter, tol, alpha_rdf_max, gamma_initial)	Multi-component OZ solver with adaptive alpha-mixing
apply_adaptive_ylim(ax, ydata, limit, clip)	
plot_matrix_quantity(r, quantity, u_matrix, species, title_prefix, filename, plots_dir)	
wca_split(r, u)	Returns repulsive part of u(r) using WCA splitting.
detect_sigma_from_gr(r, g, g_tol, min_width)	Detect hard-core diameter ONLY if g(r) is strictly zero
boltzmann_potential_from_gr(g, beta, g_min)	u(r) = -ln g(r) with numerical protection
detect_first_minimum_near_core(r, u_ij, sigma)	Detect the first local minimum of u(r) after the hard core.
find_key_recursive(d, key)	
rdf_alpha_r(ctx, rdf_config, densities, supplied_data, export, plot, filename_prefix)	Fully dictionary-driven isotropic RDF solver

cdft_solver/calculators/integrated_strength/integrated_strength_planer_kernal.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
vij_planer_kernel(ctx, config, kernel_data, u_data, export_json, filename, plot)	Compute planar MF coupling:

cdft_solver/calculators/integrated_strength/integrated_strength_radial_kernal.py

Pairwise integrated strength v_ij from radial kernels.

Callable	Source description / role
vij_radial_kernel(ctx, config, kernel, supplied_data, export_json, filename)	Compute:

cdft_solver/calculators/one_d_profile_iterator/kernel_generator_planer.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
build_strength_kernel_planer(ctx, config, densities, supplied_data, kernel_type)	Build planar strength kernel.

cdft_solver/calculators/one_d_profile_iterator/one_d_profile_iterator_box.py

One-dimensional density-profile iteration.

Callable	Source description / role
deep_get(data, key, default)	Recursively search for key in nested dicts/lists.
deep_get_all(data, key)	Return all matches for key in nested structure.
build_thermodynamics_from_fe_res(fe_res)	Given a free-energy result dictionary (from JSON),
one_d_profile_iterator_box(ctx, config, export_json, export_plots, filename)	Required solvers (install via pip if needed).

cdft_solver/calculators/phase_finder/thermodynamic_potentials_isochem.py

Grand/isochemical thermodynamic state evaluation.

Callable	Source description / role
deep_get(data, key, default)	Recursively search for key in nested dicts/lists.
deep_get_all(data, key)	Return all matches for key in nested structure.
build_thermodynamics_from_fe_res(fe_res)	Given a free-energy result dictionary (from JSON),
find_key_recursive(d, key)	
evaluate_grand_canonical_state(ctx, config_dict, fe_res, supplied_data, export, output_file, verbose)	Grand canonical evaluation at given fixed densities.

cdft_solver/calculators/phase_finder/thermodynamic_potentials_isocore.py

Canonical state evaluation from densities.

Callable	Source description / role

deep_get(data, key, default)	Recursively search for key in nested dicts/lists.
deep_get_all(data, key)	Return all matches for key in nested structure.
find_key_recursive(d, key)	
build_thermodynamics_from_fe_res(fe_res)	Given a free-energy result dictionary (from JSON),
evaluate_canonical_state(ctx, config_dict, fe_res, supplied_data, export, output_file, verbose)	Canonical evaluation:

cdft_solver/calculators/radial_distribution_function/builtin.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
py_closure(r, gamma, u, sigma_ab)	Percus-Yevick (PY) closure.
hnc_closure(r, gamma, u, sigma_ab)	
hybrid_closure(r, gamma, u, sigma_ab)	

cdft_solver/calculators/radial_distribution_function/closure.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
closure_update_c_matrix(gamma_r_matrix, r, pair_closures, u_matrix, sigma_matrix, closure_registry)	pair_closures[a,b] can be:

cdft_solver/calculators/radial_distribution_function/rdf_planer.py

Planar correlation solver.

Callable	Source description / role
plot_correlation_functions(gamma_r, c_r, r_grid, z_grid, Ns, Nz, Nr, ctx, plot)	Plot and save RDF results.
hankel_transform_2d(f_r, r, k_grid, J0)	Apply Hankel transform for cylindrical coordinates along r.
inverse_hankel_transform_2d(F_k, r, k_grid, J0)	
solve_oz_matrix_2d(c_r, RHO, r_grid, k_grid, J0, Ns, Nz, Nr)	
solve_oz_matrix_2d(c_r_matrix, RHO, r, k_grid, J0, Ns, Nz, Nr)	2D OZ solver along r for each plane z_i, z_j using the C solver.
rdf_planer(ctx, rdf_config, densities, supplied_data, export, plot, filename_prefix)	

cdft_solver/calculators/radial_distribution_function/rdf_radial.py

3D isotropic multicomponent OZ/RDF solver.

Callable	Source description / role
solve_oz_kspace(h_k, densities, eps)	Solve multi-component OZ equation in k-space:
plot_u_matrix(r, u_matrix, species, outdir, filename)	Plot and export u_ij(r) for all species pairs.
process_supplied_rdf(supplied_data, species, r_grid)	Uses canonical pair keys like 'AA', 'AB', ...
hankel_forward_dst(f_r, r)	
hankel_inverse_dst(k, Fk, r)	
hankel_transform_matrix_fast(f_r_matrix, r)	
inverse_hankel_transform_matrix_fast(f_k_matrix, k, r)	
solve_oz_matrix(c_r, r, densities)	
multi_component_oz_solver_alpha(r, pair_closures, densities, u_matrix, sigma_matrix, c_update_flag, c_initial, n_iter, tol, alpha_rdf_max)	Multi-component OZ solver with adaptive alpha-mixing
apply_adaptive_ylim(ax, ydata, limit, clip)	
plot_matrix_quantity(r, quantity, u_matrix, species, title_prefix, filename, plots_dir)	
find_key_recursive(d, key)	
rdf_radial(ctx, rdf_config, densities, supplied_data, export, plot, filename_prefix)	Fully dictionary-driven isotropic RDF solver

cdft_solver/calculators/radial_distribution_function/rdf_two_d.py

2D isotropic OZ/RDF solver.

Callable	Source description / role
solve_oz_kspace(h_k, densities, eps)	Solve multi-component OZ equation in k-space:
plot_u_matrix(r, u_matrix, species, outdir, filename)	Plot and export u_ij(r) for all species pairs.
process_supplied_rdf(supplied_data, species, r_grid)	Uses canonical pair keys like 'AA', 'AB', ...
build_k(r)	
build_bessel_matrix(k, r)	
hankel_forward_core(f_r, r, J0, Nk, Nr)	
hankel_forward_2d(f_r, r, k, J0)	
hankel_inverse_2d(Fk, k, r, J0)	
hankel_transform_matrix(f_r_matrix, r, k, J0)	
inverse_hankel_transform_matrix(f_k_matrix, r, k, J0)	
solve_oz_kspace_numba(c_k, densities)	
multi_component_oz_solver_alpha(r, pair_closures, densities, u_matrix,	

sigma_matrix, n_iter, tol, alpha_rdf_max)	
apply_adaptive_ylim(ax, ydata, limit, clip)	
plot_matrix_quantity(r, quantity, u_matrix, species, title_prefix, filename, plots_dir)	
find_key_recursive(d, key)	
rdf_2d(ctx, rdf_config, densities, supplied_data, export, plot, filename_prefix)	Fully dictionary-driven isotropic RDF solver

cdft_solver/calculators/radial_distribution_function/utils.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
safe_exp(x, xmin, xmax)	Numerically safe exponential.

cdft_solver/calculators/rdf_inversion/data_analyser_examples/plotter_integrated_strength.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
find_common_crossing(curves, r, r_guess)	

cdft_solver/calculators/rdf_inversion/data_analyser_examples/plotter_structure_factor.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
structure_factor_ij(k, r, g_ij, rho_i, rho_j)	Partial structure factor S_ij(k)

cdft_solver/calculators/rdf_inversion/rdf_inversion_advance.py

Multistate IBI/Boltzmann inversion plus reference diagnostics.

Callable	Source description / role
solve_oz_kspace(h_k, densities, eps)	Solve multi-component OZ equation in k-space:
solve_oz_matrix(c_r, r, densities)	
multi_component_oz_solver_alpha(r, pair_closures, densities, u_matrix, sigma_matrix, c_update_flag, c_initial, n_iter, tol, alpha_rdf_max, gamma_initial)	Multi-component OZ solver with adaptive alpha-mixing
wca_split(r, u)	Returns repulsive part of u(r) using WCA splitting.
detect_sigma_from_gr(r, g, g_tol, min_width)	Detect hard-core diameter ONLY if g(r) is strictly zero
boltzmann_potential_from_gr(g, beta, g_min)	u(r) = -ln g(r) with numerical protection

detect_first_minimum_near_core(r, u_ij, sigma)	Detect the first local minimum of u(r) after the hard core.
clean_gr_tail(r, g, prominence, alpha, noise_tol, min_persistence)	Physically motivated RDF tail cleaning.
process_supplied_rdf_multistate(supplied_data, species, r_grid)	
find_key_recursive(d, key)	
plot_u_matrix(r, u_matrix, species, outdir, filename)	Plot and export u_ij(r) for all species pairs.
boltzmann_inversion_advance(ctx, rdf_config, supplied_data, filename_prefix, export_plot, export_json)	Multistate Isotropic Boltzmann / IBI inversion

cdft_solver/calculators/total_free_energy/free_energy_exporter.py

Serialize symbolic total free energy.

Callable	Source description / role
_serialize_symbol(sym)	Serialize a SymPy symbol safely.
_serialize_expression(expr)	Serialize a SymPy expression (lossless).
free_energy_exporter(ctx, total_fe, filename, system_config, indent)	Export merged total free energy to a JSON file.

cdft_solver/calculators/total_free_energy/total_free_energy.py

Free-energy dispatcher and symbolic component merger.

Callable	Source description / role
unify_symbols(components)	Unify SymPy symbols across multiple free-energy components
extract_sigma_matrix(hc_data)	
sigma_is_zero(sigma_matrix, tol)	
merge_free_energies(components)	Merge multiple free-energy components into one symbolic object,
find_key_recursive(d, key)	
total_free_energy(ctx, hc_data, system_config, export_json, filenames)	Unified free-energy dispatcher.

cdft_solver/calculators/virial_analysis/second_virial.py

Second-virial analysis.

Callable	Source description / role
find_key_recursive(d, key)	
second_virial(ctx, virial_config, on, export, filename_prefix, r_max_factor, nr, n_lambda, beta_scale)	Research-grade computation of:

cdft_solver/calculators/virial_analysis/second_virial_scale_calibration.py

Calibrate potential scales against target B2.

Callable	Source description / role
find_zero_crossing(r, u)	
compute_sigma_BH(scale, r, u, tol, hc_threshold)	Compute Barker–Henderson diameter with robust hard-core detection.
find_key_recursive(d, key)	
compute_B2_scaled(scale, r, u_total)	Second virial coefficient using a globally scaled total potential:
unpack_scale_vector(scale_vec, pair_list, n)	
second_virial_scale_calibration(ctx, virial_config, B2_target, on, export, filename_prefix, r_max_factor, nr, beta_scale)	Calibrate pairwise scalar factors f_ij such that

cdft_solver/engines/bulk.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
bulk_executor(ctx)	Bulk executor for Density Functional Theory simulations.

cdft_solver/engines/one_d_profile.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
one_d_profile_executor(ctx)	Bulk executor for Density Functional Theory simulations.

cdft_solver/engines/rdf_bulk.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
rdf_bulk_executor(ctx)	Bulk executor for Density Functional Theory simulations.

cdft_solver/executor_dft_main.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
parse_input_keywords(input_text)	Parse ensemble, method, task, and profile from an input file.
main()	

cdft_solver/generators/density_weights/fmt_weights_planer.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
fmt_weights_planer(ctx, data_dict, grid_properties, export_json, filename, plot)	Generate 1D FMT weight functions for cylindrical geometry.

cdft_solver/generators/density_weights/mf_weights_planer.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
mf_weights_planer(ctx, data_dict, grid_properties, export_json, filename, plot)	Dictionary-driven MF cylindrical kernel generator.

cdft_solver/generators/grids_properties/bulk_rho_mue_planer.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
bulk_rho_mue_planer(ctx, thermodynamic_parameter, r_space_coordinates, export_json, filename, plot)	Assign bulk density and chemical potential values to an r-space grid

cdft_solver/generators/grids_properties/external_potential_grid.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
external_potential_grid(ctx, data_dict, grid_properties, export_json, filename, plot)	Compute external potentials from a dictionary and a spatial grid.

cdft_solver/generators/grids_properties/k_and_r_space_box.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
r_k_space_box(ctx, data_dict, export_json, filename)	Generate r-space and k-space grids from a dictionary.

cdft_solver/generators/grids_properties/k_and_r_space_cylindrical.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
r_k_space_cylindrical(ctx, data_dict, export_json, filename)	Generate cylindrical r-space and k-space grids from a dictionary.

cdft_solver/generators/grids_properties/k_and_r_space_spherical.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
r_k_space_spherical(ctx, data_dict, export_json, filename)	Generate spherical r-space and k-space grids from a dictionary.

cdft_solver/generators/parameters/advance_dictionary.py

Hierarchical .in parser and JSON exporter.

Callable	Source description / role
super_dictionary_creator(ctx, input_file, base_dict, export_json, filename, super_key_name)	Universal dictionary builder from hierarchical input.

cdft_solver/generators/potential/pair_potential_isotropic.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
pair_potential_isotropic(specific_pair_potential)	

cdft_solver/generators/potential/pair_potential_isotropic_default.py

Built-in analytic isotropic potential implementations.

Callable	Source description / role
zero_potential(p)	
hard_core(p)	
lj(p)	
gaussian(p)	
generalized_gaussian(p)	Generalized Gaussian / GEM-n potential
mie(p)	
custom_1(p)	
custom_3(p)	
wca(p)	
salj(p)	
nvrn(p)	
ma(p)	

cdft_solver/generators/potential/pair_potential_isotropic_registry.py

Potential registry and factory lookup.

Callable	Source description / role
pair_potential_isotropic(specific_pair_potential)	
register_isotropic_pair_potential(potential_type, factory_fn, overwrite)	Register an isotropic pair potential factory.
get_isotropic_pair_potential_factory(potential)	Resolve factory from registry.

cdft_solver/generators/potential_splitter/hc.py

Hard-core detection, effective sigma matrix, HC potential generation.

Callable	Source description / role
hard_core_potentials(ctx, input_data, grid_points, file_name_prefix, export_files)	Hard-core detection module using a dictionary input instead of a JSON file.

cdft_solver/generators/potential_splitter/mf.py

Mean-field interaction conversion and potential generation.

Callable	Source description / role
meanfield_potentials(ctx, input_data, grid_points, file_name_prefix, export_files)	Mean-field potential generator using a dictionary input.

cdft_solver/generators/potential_splitter/mf_registry.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
register_potential_converter(potential_type, converter_fn, overwrite)	Register a potential conversion function.
convert_potential_via_registry(potential)	Convert a potential dictionary using the registry.
_hc_to_zero(pot)	
_lj_to_salj(pot)	
_mie_to_ma(pot)	

cdft_solver/generators/potential_splitter/raw.py

Unsplitted sum of primary/secondary/tertiary pair interactions.

Callable	Source description / role

raw_potentials(ctx, input_data, grid_points, file_name_prefix, export_files)	

cdft_solver/generators/potential_splitter/total.py

Combine hard-core and mean-field potential representations.

Callable	Source description / role
load_json_or_dict(obj)	Accepts either:
reconstruct_potential(pot_dict, u_key)	Convert JSON potential entry to NumPy arrays.
find_key_recursive(obj, key)	Recursively find ALL occurrences of `key` in nested dict/list structures.
build_hc_flag_map(species, flag_matrix)	Build map: pair -> 0/1
interpolate_to_grid(r_src, u_src, r_target)	Interpolate u_src(r_src) onto r_target.
total_potentials(ctx, hc_source, mf_source, file_name_prefix, export_files)	Assemble hard-core + mean-field potentials with:

cdft_solver/generators/supplied_data/process_supplied_data.py

Materializes filename-backed numerical supplied data.

Callable	Source description / role
find_key_recursive(d, key)	
_load_xy_file(path)	
materialize_supplied_data(obj, base_dir)	Recursively walk supplied_data and replace
_to_serializable(obj)	
_plot_materialized(obj, plots_dir, prefix)	
process_supplied_data(ctx, config, export_json, export_plot)	

cdft_solver/generators/supplied_data/process_supplied_potential.py

Registers/replaces supplied potential data.

Callable	Source description / role
supplied_potential_factory(r_data, U_data)	Return callable interpolated potential V(r).
wca_split(r, U)	Split a potential into repulsive (soft) and attractive parts (WCA).
has_hard_core(U, threshold)	Heuristic to detect a hard-core in a potential.
find_key_recursive(d, key)	Recursively find the first occurrence of a key in a nested dict.
replace_primary_potential_recursive(config, pair, new_name)	Recursively search for a 'primary' section inside 'potentials' and
register_supplied_potentials(ctx, config, supplied_data, export_json, export_plot)	Register supplied potentials as callable functions, split if needed,

cdft_solver/utils.py

Supporting module; inspect function signature and source before using in a new public workflow.

Callable	Source description / role
create_unique_scratch_dir(base_name)	Create a unique scratch directory in the current working directory.
class ExecutionContext	Class defined in this module.

cdft_solver/calculators/radial_distribution_function/registry.py

Runtime closure registry used to associate closure names with Python callables. This module supports the manual-closure workflow documented in Part VI.

cdft_solver/calculators/rdf_inversion/data_analyser_examples/plotter_g_gamma_c.py

Auxiliary plotting example for inspecting g(r), gamma(r), and c(r) produced by inversion/structural-analysis data.

24. Output File and Array Shape Reference

Workflow	Primary return	Primary exported files	Key shapes
Potential preprocessing	hc_data / mf_data / raw_data / total_data	hc.json, mf.json, raw.json, total.json	pair r,U vectors; sigma/flag N×N
Free energy	dict with SymPy expression/function	fe_id/mf/hc/hybrid + fe_total	variables length $N+N^2$ (for MF symbols)
Canonical state	state dict	state_from_density.json optional	densities N; vij N×N; mu N
RDF radial	pair structural output	prefix JSON/plots depending function	g,c,gamma N×N×Nr
Inversion	(u_matrix, sigma_matrix)	prefix_potential.json + plots/diagnostics	u N×N×Nr; sigma N×N
c_analysis	(u_matrix, sigma_matrix)	result_sigma_analysis, delta_B2, result_G_of_r, delta_c, prefix_potential	many N×N×Nr arrays
Coexistence	dict	workflow-controlled text/JSON/plot	rhos nphases×N; vij nphases×N×N

24.1 Pair indexing convention

```
species = [a, b]
index 0 -> a
index 1 -> b

matrix[0,0] -> aa
matrix[0,1] -> ab
matrix[1,0] -> ba  (normally symmetric with ab)
matrix[1,1] -> bb
```

24.2 Radial grid convention in RDF/inversion modules

For r_max and n_points, the standard 3D radial grid is constructed with dr = r_max/(n_points+1) and r = dr * [1,2,...,n_points]. Therefore r=0 is not an explicit structural grid point, although some integration helpers insert a zero anchor internally when calculating effective diameters.

25. Quick Reference: Which Workflow Do I Use?

Scientific question	Primary workflow / function	Minimum input blocks
What is P(rho) for one component?	preprocessing + evaluate_canonical_state density loop	species, interaction, rdf, free_energy, solution density
What is P(rho_t) at fixed binary composition?	binary canonical state loop	species, all pair interactions, closures, free_energy, both densities
What potential reproduces a supplied RDF?	process_supplied_data + boltzmann_inversion_advance	species, rdf settings, supplied_data states
What are g(r), c(r), reference sigma, Delta B2 and Delta c?	c_analysis	species, interactions, closures, density array
What is v_ij by uniform/B2/RDF/delta-c route?	build_strength_kernel + vij_radial_kernel, often called indirectly	interactions, rdf, free_energy integrated_strength_kernel
What are coexisting densities?	preprocessing + coexistence_densities_isochem	species, interactions, free_energy, phase and intrinsic constraints
What is one state's mu and pressure?	evaluate_canonical_state	full free energy plus density for every species

26. Recommended Improvements Before Public Release

These recommendations follow directly from the source audit and are included because they materially improve the reliability of the manual/data contract for other users.

19. Synchronize package version between setup.py and pyproject.toml.
20. Add sympy, numba and tqdm to declared dependencies if these workflows are public API; document pyfftw only for any retained legacy engine path.
21. Add work_dir to ExecutionContext and resolve all filename-backed data relative to the input file directory rather than Path.cwd().
22. Define/import SuppliedDataError in process_supplied_data.
23. Standardize tolerance spelling and keep a backward-compatible alias for tolerence if old inputs must continue to run.
24. Standardize free-energy comments/documentation to the actual accepted keys: mode standard/hybrid/hybrid_lattice and method smf/emf/void.
25. Normalize species to a list immediately after parsing and define an explicit pair-key convention that supports multi-character species names.
26. Add schema validation after super_dictionary_creator so missing pair closures/densities and invalid values are caught before expensive solvers start.
27. Write all workflow-created output files into ctx.scratch_dir rather than occasionally into the current working directory.
28. Add small regression tests that parse every example input, run a low-resolution potential/free-energy setup, and check output keys/shapes.

29. Replace or repair the legacy executor/engine imports, or clearly mark them deprecated so users start from the direct API documented here.

27. Closing Summary

The supplied cDFT_solver snapshot already contains the pieces for a broad cDFT/integral-equation workflow: hierarchical input parsing, analytic/tabulated potentials, hard-core and mean-field decomposition, OZ closures, effective-potential inversion, symbolic free energies, integrated-strength routes, canonical thermodynamics, and coexistence. The most important practical requirement is to preserve the data contract between these pieces. For that reason, this manual treats every input line, parsed dictionary, numerical array, and exported JSON object as part of the scientific method rather than as incidental software plumbing.

For new calculations, the recommended sequence is: validate the .in file by exporting input_system.json; inspect potential decomposition and sigma/flag matrices; build the free energy once; then run a clearly scoped state, structural, inversion, or coexistence workflow while saving the relevant kernel/interaction quantities alongside the final observable.

PART VI — Advanced Extensibility, Supplied Data, and Multistate Inversion

This extended part supplements the preceding source-audited manual. It focuses on the package features that are easiest to miss when reading only the bundled workflows: user-defined closures, runtime-registered potentials, file-backed potentials, mixed supplied/computed structural data, and effective-potential inversion constrained by several thermodynamic state points.

Source basis — The descriptions below are based on the active modules closure.py, registry.py, builtin.py, rdf_radial.py, pair_potential_isotropic_registry.py, pair_potential_isotropic.py, pair_potential_isotropic_default.py, raw.py, hc.py, mf.py, mf_registry.py, process_supplied_data.py, process_supplied_potential.py, rdf_inversion_advance.py, c_analysis.py, external_potential_grid.py, and the bundled Example_workflow inputs/scripts.

28. Advanced Extension and Data-Injection Overview

The package supports several independent mechanisms for introducing data or physics from outside the built-in definitions. The correct mechanism depends on what is being supplied and at what stage it should enter the computation.

Mechanism	Where it enters	Internal object	Best use
Built-in analytical potential	interactions ... type = ...	potential factory -> V(r)	standard LJ/Mie/Gaussian/HC/GEM-style interactions
Tabulated pair potential	interactions ... filename = ...	interpolated r,U table	simulation-derived, inverted, fitted, or externally generated pair potential
Runtime custom potential	Python potential registry	user factory: dict -> callable V(r)	new functional form without editing package source
Manual closure	rdf.closure after parsing or closure registry	string name or Python callable	test a new OZ closure on selected pairs
Generic supplied file data	supplied_data + process_supplied_data	nested x/y arrays	RDF state data and other numerical series
Constrained structural data	rdf_radial supplied_data	pair r/g arrays + fixed mask	hold selected RDFs fixed while solving the rest
Multistate inversion data	boltzmann_inversion_advance	states -> densities, beta, g_target, fixed_mask	infer a shared effective potential from one or many states

Important terminology — “Supplied data” is not a single universal schema in this source snapshot. The generic loader produces x/y; advanced inversion accepts x/y or r/g; rdf_radial constrained projection expects r/g; supplied-potential registration expects r/U. The later compatibility table makes these differences explicit.

29. Adding a Manual Closure Through the Workflow

The closure dispatcher was written to accept either a built-in closure name or a Python callable on a pair-by-pair basis. This is the cleanest extension point for testing a new liquid-state closure without modifying the package source files.

29.1 Built-in closure path

Theory connection: the supplied structural paper uses PY for hard-core / excluded-volume pairs and HNC for soft polymer-like pairs in its OZ/IBI workflow. This provides a physical example of why the package supports a pair-specific closure matrix. A manual closure should be validated against the same limiting requirements (core condition, asymptotic g→1, consistent h/c/g definitions) before it is used for thermodynamic inference. [Theory: Varma, Archer & Scacchi, 2026 manuscript.]

The text input normally creates a nested closure dictionary such as the following:

```
species = a, b
rdf closure aa = hnc
rdf closure ab = py
rdf closure bb = hybrid
```

After super_dictionary_creator, this becomes system["system"]["rdf"]["closure"] = {"aa":"hnc", "ab":"py", "bb":"hybrid"}. rdf_radial, c_analysis, and boltzmann_inversion_advance convert that dictionary into an N x N object matrix and mirror each i<=j selection to j,i.

29.2 Exact callable signature required by closure_update_c_matrix

Required custom-closure interface

```
def user_closure(*, r, gamma, u, sigma_ab=None):
    # r        : ndarray, shape (Nr,)
    # gamma    : ndarray, shape (Nr,)
    # u        : ndarray, shape (Nr,)
    # sigma_ab : float or None
    # Return c(r) on the same grid and with the same shape.
    c_r = ...
    return c_r
```

Argument	Meaning	Critical detail
r	radial grid	same 1D grid used by the OZ iteration
gamma	current indirect correlation gamma=h-c	one value per radial point
u	pair potential passed to the closure	in active radial workflows it is already scaled as beta*u for the current state
sigma_ab	effective hard-core/reference diameter	can be None or zero; do not assume a hard core
return	new direct correlation c(r)	must be a numerical 1D array matching gamma.shape

Keyword calling convention — The dispatcher invokes a callable as closure(r=r, gamma=gamma, u=u, sigma_ab=...). A function with incompatible keyword names will fail even if the numerical formula itself is valid.

29.3 Recommended method: replace one closure after parsing

Because a Python function cannot be represented in the plain-text .in file, the most transparent research workflow is: keep a readable placeholder closure in the input file, parse the file, then replace the desired pair entry with the callable before the structural calculation begins.

Manual closure injected into the ordinary workflow

```
import numpy as np
from cdft_solver.generators.parameters.advance_dictionary import super_dictionary_creator

# HNC-equivalent formula used here only to demonstrate the software interface.
def my_hnc(*, r, gamma, u, sigma_ab=None):
    return np.exp(-u + gamma) - gamma - 1.0

system = super_dictionary_creator(
    ctx=ctx,
    export_json=True,
    filename='input_system_before_python_override.json',
)

# .in file may say: rdf closure ab = py
system['system']['rdf']['closure']['ab'] = my_hnc

# The normal workflow can now be called.
# rdf_radial(...), c_analysis(...), or boltzmann_inversion_advance(...)
```

The callable survives because the solver stores pair_closures in a NumPy object array. closure_update_c_matrix checks callable(closure) before trying a string registry lookup.

29.4 Alternative method: register a new closure name globally

The built-in registry is the dictionary CLOSURE_REGISTRY. It contains PY, HNC, and HYBRID. A user can add another entry before the solver builds its pair-closure matrix.

Register and select a named custom closure

```
from cdft_solver.calculators.radial_distribution_function.registry import CLOSURE_REGISTRY

def my_closure(*, r, gamma, u, sigma_ab=None):
    return np.exp(-u + gamma) - gamma - 1.0

CLOSURE_REGISTRY['MYCLOSURE'] = my_closure
system['system']['rdf']['closure']['aa'] = 'MYCLOSURE'
```

String entries are converted to uppercase by the dispatcher, so a registry key MYCLOSURE can be selected by "myclosure" or "MYCLOSURE" in the runtime dictionary.

> **No dedicated helper in the current snapshot** — Unlike the potential registry, the closure package does not expose a register_closure(...) helper. The current extension mechanism is direct assignment into CLOSURE_REGISTRY or direct use of a callable.

29.5 Mixed closure matrix in a multicomponent system

Closures can be mixed by pair. A binary mixture can therefore use one analytical closure for aa, another for bb, and a manually defined callable for ab.

```
closure = system['system']['rdf']['closure']
closure['aa'] = 'HNC'
closure['ab'] = my_hnc      # callable
closure['bb'] = 'PY'
```

The same pair mapping is used in rdf_radial, c_analysis, and each thermodynamic state solved during boltzmann_inversion_advance. Thus, if a custom ab closure is injected before a multistate inversion, it is applied consistently to ab at every supplied state point.

29.6 Using sigma_ab inside a manual closure

The dispatcher also passes the effective pair diameter. A new closure can therefore distinguish core and non-core regions. The following is only an interface example, not a recommended physical closure:

```
def sigma_aware_example(*, r, gamma, u, sigma_ab=None):
    c = np.exp(-u + gamma) - gamma - 1.0
    if sigma_ab is not None and sigma_ab > 0.0:
        core = r < sigma_ab
        # Example numerical replacement inside the core only.
        c[core] = -(1.0 + gamma[core])
    return c
```

29.7 Validation checklist for a newly implemented closure

- Return a finite array with exactly the same length as r and gamma.
- Protect exponentials/logarithms if the closure can overflow for a strongly repulsive potential.
- Check the dilute or weak-interaction limit against the expected theory.
- Use one physical pair entry and allow the driver to mirror it to the symmetric matrix element.
- Begin with a low-resolution test and inspect OZ convergence before using a large n_points/max_iteration.
- When comparing closures, keep potential, beta, density, grid, and tolerances unchanged.

30. Supplying a Pair Potential from a File

The most mature external-potential path in the current source is to define an interaction with filename rather than type. This path is used directly by several bundled EOS and structural examples.

30.1 Minimal input syntax

```
species = a, b
interactions primary: aa: filename = pair_aa.txt
interactions primary: ab: filename = pair_ab.txt
interactions primary: bb: filename = pair_bb.txt
```

30.2 Required numerical table

Typical two-column potential table

```
# r             U(r)
0.50000000      100.00000000
0.51000000       75.00000000
0.52000000       54.00000000
...
5.00000000        0.00000000
```

Property	Behavior in raw/hc/mf splitters
Minimum columns	at least two; first is r, second is U(r)
Additional columns	ignored by the pair-potential splitter code
Interpolation	linear
r below table	uses the first tabulated U value in the active file-backed interpolators
r above table	filled with 0.0

Units	not converted; must be consistent with beta and the chosen length/density convention
File location	resolved from the process current working directory, not automatically from ctx.input_file

30.3 Portable workflow for relative files

```
from pathlib import Path
import os

input_path = Path('system.in').expanduser().resolve()
os.chdir(input_path.parent)

# Now any filename = pair_aa.txt in system.in is resolved relative to
# the directory containing system.in.
```

The directory name is arbitrary. The important point is to establish a predictable current working directory before hard_core_potentials, meanfield_potentials, or raw_potentials read file-backed interactions.

30.4 Combining a file potential with analytical contributions

The splitters explicitly recognize primary, secondary, and tertiary levels. These levels are summed for the same physical pair, which allows a file-backed base interaction to be supplemented by one or two analytical terms.

File + analytic interaction composition

```
interactions primary: aa: filename = base_aa.txt
interactions secondary: aa: type = gs, sigma = 1.4, epsilon = 0.25, cutoff = 5.0
interactions tertiary: aa: type = zero

interactions primary: ab: type = mie, sigma = 1.0, epsilon = 0.5, m = 24, n = 12, cutoff = 3.5
interactions primary: bb: filename = measured_bb.txt
```

Use interaction levels for additive terms — Repeating the same primary/pair line is not a safe substitute for multiple contributions because the hierarchical parser merges dictionaries. Use primary, secondary, and tertiary for additive pieces.

30.5 What happens to a supplied file in each potential generator

Module	Role	File-backed behavior
raw_potentials	physical unsplit pair potential	interpolates and sums every interaction level
hard_core_potentials	effective core/reference analysis	loads the table, probes the repulsive region, and may infer an effective sigma through the current hard-core/BH logic
meanfield_potentials	mean-field representation	loads/interpolates file data and applies the module's hard-core/WCA heuristic when a very large short-range repulsion is detected
total_potentials	reference + MF reconstruction	combines the HC representation where flag==1 with the mean-field representation, interpolating grid

		mismatches

31. Defining a New Analytical Potential Outside the Package

A user does not have to edit pair_potential_isotropic_default.py to add a new analytical interaction. The potential registry maps a string type to a factory that receives the parsed interaction dictionary and returns V(r).

31.1 Exact potential-factory contract

```
def my_potential_factory(parameter_dict):
    # Extract any parameters defined by the user.
    ...
    def V(r):
        r = np.asarray(r, dtype=float)
        return ...
    return V
```

31.2 Runtime registration example

Register a custom potential before preprocessing

```
import numpy as np
from cdft_solver.generators.potential import register_isotropic_pair_potential

def screened_exponential_factory(p):
    amplitude = float(p.get('amplitude', 1.0))
    xi = float(p.get('xi', 1.0))
    cutoff = float(p.get('cutoff', 10.0))

    def V(r):
        r = np.asarray(r, dtype=float)
        out = amplitude * np.exp(-r / xi)
        out[r > cutoff] = 0.0
        return out

    return V

register_isotropic_pair_potential(
    'screened_exponential',
    screened_exponential_factory,
)
```

The type name is lowercased by the registry. The parser preserves arbitrary attributes, so an input line can use parameter names such as amplitude and xi even though the built-in package does not know them.

Input line for the registered potential

```
interactions primary: aa: type = screened_exponential, amplitude = 2.0, xi = 1.5, cutoff = 8.0
```

31.3 Complete workflow position

30. Import the registry and register the custom factory.
31. Parse the .in file.
32. Run hard_core_potentials / meanfield_potentials / raw_potentials only after registration.
33. Proceed normally to total_potentials, RDF, free energy, EOS, coexistence, or external-potential generation.

31.4 Mean-field converter registry for a custom interaction

meanfield_potentials first passes each analytical interaction through POTENTIAL_CONVERTER_REGISTRY. If no converter exists, the dictionary is copied unchanged and the

same potential type is used. If a model needs a distinct perturbative/attractive representation, register a converter that rewrites the type to another registered factory.

```
from cdft_solver.generators.potential_splitter.mf_registry import register_potential_converter

def screened_to_mf(pot):
    pot['type'] = 'screened_exponential_attr'
    return pot

register_potential_converter('screened_exponential', screened_to_mf)
```

Modeling implication — Changing the mean-field converter changes which part/form of the interaction enters the perturbative free-energy and integrated-strength machinery. This must be justified physically and documented, not treated only as code plumbing.

31.5 Reusing a registered potential for an external field

external_potential_grid also calls pair_potential_isotropic. Consequently, a custom registered potential can be reused for the interaction between mobile system species and fixed external particles. The module sums V(|r-r_ext|) over the positions of the external particles on the supplied spatial grid.

32. In-Memory Supplied Potential Registration

The package also contains register_supplied_potentials in process_supplied_potential.py. This is conceptually different from the simpler filename route: it receives numerical arrays already loaded into memory, creates runtime potential names, and mutates the configuration to use them.

32.1 Expected in-memory schema

Schema expected by register_supplied_potentials

```
supplied_potential_data = {
    'potentials': {
        'primary': {
            'aa': {
                'r': r_values,
                'U': u_values,
            },
            'ab': {
                'r': r_ab,
                'U': u_ab,
            },
        }
    }
}
```

The function searches recursively for the first potentials block. It then loops over family and pair, registers a factory named supplied_<pair>, and recursively changes the matching primary interaction type in config to that new name.

32.2 Example runtime use

```
from cdft_solver.generators.supplied_data import register_supplied_potentials

# A matching primary entry must exist so it can be replaced.
system['system']['interactions']['primary']['aa'] = {'type': 'zero'}

register_supplied_potentials(
    ctx=ctx,
    config=system,
    supplied_data=supplied_potential_data,
    export_json=True,
    export_plot=True,
```

```
)

# The primary aa type is now 'supplied_aa'.
```

32.3 Interpolation contract of supplied_potential_factory

Region	Value
inside tabulated interval	linear interpolation
below first r	first U value
above last r	0.0

32.4 WCA splitting intention for strongly repulsive supplied potentials

The module checks whether any supplied U exceeds 1e3. If so, it is designed to generate repulsive and attractive WCA components, register supplied_<pair>_soft and supplied_<pair>_attr, and configure the mean-field conversion to use the attractive part.

Current source bug in the hard-core branch — process_supplied_potential.py imports convert_potential_via_registry under the alias register_potential_converter, but then calls that alias with the two-argument registration signature. Therefore the hard-core supplied-potential branch is not reliable in the supplied snapshot. For a strongly repulsive tabulated potential, use interactions ... filename until this import is corrected.

The intended repair is to import register_potential_converter from mf_registry. After that correction, the design of the module is clear from the remaining code.

33. Supplied Numerical Data — Rigorous Generic Contract

process_supplied_data is a generic recursive file materializer. It does not assign physical meaning to a file; it simply replaces any nested {filename: ...} leaf by numerical x/y arrays.

33.1 Transformation performed by process_supplied_data

```
# .in file leaf
supplied_data states 1 rdf aa: filename = g_aa.txt

# Parsed dictionary leaf
{'filename': 'g_aa.txt'}

# After process_supplied_data
{
    'x': np.ndarray([...]),
    'y': np.ndarray([...]),
}
```

Any additional keys in that leaf are retained. The recursion also descends through lists.

33.2 Multi-column numerical files

Input table	x	y
2 columns	first column, shape (M,)	second column, shape (M,)
3+ columns	first column, shape (M,)	all remaining columns, shape (M,K)
1 column	invalid	invalid

Current exception issue — The module intends to raise SuppliedDataError for malformed/missing data, but SuppliedDataError is not defined or imported in process_supplied_data.py. Valid files work; malformed input can reveal a NameError rather than the intended custom exception.

33.3 Relative-path behavior

The loader uses ctx.work_dir only if such an attribute exists; the current ExecutionContext dataclass does not define work_dir. It therefore normally falls back to Path.cwd(). This is another reason the portable workflows change into the input-file directory before loading supplied data.

33.4 Consumer compatibility matrix

Consumer	Pair leaf expected	State metadata	Directly compatible with process_supplied_data?
boltzmann_inversion_advance	x/y or r/g	densities required; beta or temperature	YES
rdf_radial projection	r/g	densities supplied separately to rdf_radial	NO: convert x/y -> r/g
register_supplied_potentials	r/U inside potentials/family/pair	not state-based	NO: convert x/y -> r/U
c_analysis	supplied_data argument exists	densities argument required	argument is not used in the main audited source path
evaluate_canonical_state / strength kernel	supplied_data is accepted at outer level	state densities from configuration	current rdf/delta_c kernel branches explicitly invoke their structural functions with supplied_data=None

33.5 Adapter from generic x/y to rdf_radial r/g

```
def materialized_rdf_to_rg(materialized, state='1'):
    state_data = materialized['states'][state]
    result = {'rdf': {}}
    for pair, entry in state_data['rdf'].items():
        result['rdf'][pair] = {
            'r': np.asarray(entry['x'], dtype=float),
            'g': np.asarray(entry['y'], dtype=float),
        }
    return result

materialized = process_supplied_data(ctx=ctx, config=system)
rdf_constraints = materialized_rdf_to_rg(materialized, state='1')
```

33.6 Adapter from generic x/y to register_supplied_potentials r/U

```
leaf = materialized['potentials']['primary']['aa']
supplied_for_registry = {
    'potentials': {
        'primary': {
            'aa': {
                'r': np.asarray(leaf['x'], dtype=float),
                'U': np.asarray(leaf['y'], dtype=float),
            }
        }
    }
}
```

For ordinary tabulated pair potentials, the direct filename interaction is usually simpler and more robust than this conversion route.

34. Combining Supplied and Computed RDF Data in rdf_radial

rdf_radial contains an advanced projection mode that is separate from potential inversion. It allows selected g_ij(r) to be treated as externally fixed structural information while the remaining pair correlations are determined self-consistently from OZ and the chosen closures.

34.1 Internal projection sequence

34. Run an ordinary unconstrained OZ calculation for all pairs.
35. Interpolate each supplied RDF pair onto the internal radial grid and mark that matrix element fixed.
36. Replace h_ij=g_ij-1 for the fixed pairs by the supplied values.
37. Transform the complete h matrix to k space.
38. Use C(k)=H(k)[I+rho H(k)]^{-1} to obtain a projected direct-correlation matrix.
39. Inverse-transform c(k) back to c(r).
40. Freeze c for the supplied pairs and allow closure updates only for the free pairs.
41. Resolve OZ and iterate the projection until the free-pair h functions stop changing within tolerance.

34.2 Example: supply aa, compute ab and bb

```
rdf_constraints = {
    'rdf': {
        'aa': {
            'r': r_sim,
            'g': g_aa_sim,
        }
    }
}

rdf_out = rdf_radial(
    ctx=ctx,
    rdf_config=system,
    densities=np.array([rho_a, rho_b], dtype=float),
    supplied_data=rdf_constraints,
    export=True,
    plot=True,
    filename_prefix='hybrid_structure',
)
```

The potentials are still the interactions defined in the configuration. This operation constrains structure; it does not update or infer a potential.

34.3 Supplying more than one pair

Any subset can in principle be fixed: aa only, ab only, aa+ab, or all pair RDFs. The free pairs are those for which fixed_mask is False.

Pair-key asymmetry in this consumer — rdf_radial process_supplied_rdf checks exact keys as it loops over ordered matrix positions and does not perform the AB/BA fallback and symmetric assignment used by the advanced inversion processor. For robust usage, supply canonical pair keys consistent with species order; if both matrix directions must be fixed explicitly, provide both.

35. Multistate Effective-Potential Inversion in Detail

boltzmann_inversion_advance accepts one or many target RDF state points and updates one common pair-potential matrix. It is therefore capable of both ordinary single-state inversion and a more restrictive multistate coarse-graining problem.

35.1 Accepted state container names

process_supplied_rdf_multistate looks for a top-level states mapping, then state, then falls back to treating the supplied_data object itself as the mapping of state names to state dictionaries.

35.2 State density formats

Input	Example	Result for species [a,b]
dict by species	{"a":0.10,"b":0.02}	[0.10, 0.02]
dict missing b	{"a":0.10}	[0.10, 0.00]
scalar	0.10	[0.10, 0.10]
length-1 array	[0.10]	[0.10, 0.10]
length-2 array	[0.10,0.02]	[0.10,0.02]

A multidimensional density array or an array with the wrong length raises a ValueError.

35.3 Beta and temperature metadata

If beta is present, it is used directly. Otherwise, if temperature is present, beta=1/temperature. If neither appears, beta defaults to 1.0. The first state beta becomes beta_ref for the common internal potential representation.

35.4 RDF leaves accepted by inversion

For each pair the inversion processor accepts either x/y or r/g. This is why process_supplied_data output can be passed directly to boltzmann_inversion_advance.

The supplied RDF is linearly interpolated onto the inversion grid. Below the first r point the first g value is used; above the last supplied r it approaches 1.0. The interpolated curve is then passed through clean_gr_tail before being stored in g_target.

35.5 Missing pair RDFs are allowed

The processor allocates g_target as zeros and fixed_mask as False, then sets only those pairs that are actually found in the supplied state. Missing pair data are therefore not invented. This is what permits known model interactions and unknown/inverted interactions to coexist.

35.6 Partial inversion: known aa and bb, infer ab only

Mixed known + inverted pair workflow

```
species = a, b

# Known pair interactions
interactions primary: aa: filename = known_aa.txt
interactions primary: bb: type = gs, sigma = 1.414, epsilon = 2.0, cutoff = 5.0
# No ab model interaction is required if ab is to be inferred.
```

```
rdf closure aa = hnc
rdf closure ab = py
rdf closure bb = hnc
rdf alpha_max = 0.1
rdf max_iteration = 50000
rdf max_iteration_ibi = 5000
rdf alpha_ibi_max = 0.1
rdf tolerance = 0.00001
rdf ibi_tolerance = 0.0001
rdf r_max = 15.0
rdf n_points = 1500

supplied_data states 1 rdf ab: filename = g_ab_state1.txt
supplied_data states 1 densities a = 0.10
supplied_data states 1 densities b = 0.02
supplied_data states 1 beta = 1.0
```

35.7 How the inversion mask is actually created

42. u_matrix starts at zero and invert_mask starts True for every pair.
43. Any preprocessed model pair potential is interpolated into u_matrix. A nonzero pair is initially marked invert_mask=False.
44. For each supplied target RDF pair, the first-state g is converted to -ln(max(g,g_floor)) and written into u_matrix, sigma is estimated from the RDF core, and invert_mask is set True.
45. During IBI, a correction is accumulated only where fixed_mask is True. Therefore an unsupplied known model pair receives no IBI update.
46. A pair with neither a model potential nor supplied RDF remains effectively zero because no correction is accumulated.

If model potential and RDF are both supplied for the same pair — The current initialization replaces that pair by -ln(g) and marks it for inversion. The model potential is not retained as the IBI starting guess for a supplied pair.

35.8 Multiple thermodynamic state points constraining one potential

Three-state inversion constraint

```
supplied_data states 1 rdf ab: filename = g_ab_rho1_T1.txt
supplied_data states 1 densities a = 0.10
supplied_data states 1 densities b = 0.02
supplied_data states 1 beta = 1.0

supplied_data states 2 rdf ab: filename = g_ab_rho2_T1.txt
supplied_data states 2 densities a = 0.18
supplied_data states 2 densities b = 0.04
supplied_data states 2 beta = 1.0

supplied_data states 3 rdf ab: filename = g_ab_rho2_T2.txt
supplied_data states 3 densities a = 0.18
supplied_data states 3 densities b = 0.04
supplied_data states 3 temperature = 1.25
```

The routine currently assigns equal state weights, w_state=1/N_states. A separate OZ problem is solved at each state on every IBI iteration, but all states update the same common u_matrix.

35.9 Numerical IBI correction

For state s and supplied pair ij, the implemented correction is proportional to (beta_ref/beta_s) ln[g_pred,s(r)/g_target,s(r)]. Only radii with g_target>1e-4 enter the logarithmic update. State corrections are averaged and then multiplied by adaptive alpha_ibi.

35.10 Adaptive alpha_ibi

Quantity	Current behavior
initial alpha_ibi	0.01
minimum	1e-4
maximum	rdf alpha_ibi_max
adaptation interval	every 10 IBI iterations after iteration 1
adaptation signal	ratio of previous maximum RDF error to current maximum RDF error, clipped before applying a power 0.5
convergence metric	maximum absolute |g_pred-g_target| over all target pairs and states
convergence target	rdf ibi_tolerance

35.11 Post-inversion hard-core/reference analysis

After IBI convergence, the routine continues with substantial reference analysis. It detects near-zero RDF core intervals, identifies hard-core pairs, constructs repulsive references, optimizes sigma collectively over all supplied states, evaluates Barker-Henderson diameters, generates attractive/reference decompositions, and computes several state-resolved structural diagnostics.

35.12 Main exported files from an advanced inversion

Export	Meaning
<prefix>_potential.json	compact final pair potentials: r, u_r, sigma for every matrix pair
result_sigma_analysis.json	sigma_opt, sigma_bh, reference/real potentials and state-resolved g/c/gamma packages
result_G_of_r.json	coupling-integrated G(r), G_u(r), and attractive potential information
delta_c_by_state.json	real/reference direct correlations and several Delta-c combinations
plots/	potential and RDF/reference diagnostics when enabled

The function finally returns (u_matrix / beta_ref, sigma_matrix), with shapes (N,N,Nr) and (N,N), respectively.

36. Advanced Data-Combination Recipes

36.1 Recipe A — tabulated potential + manual closure

```
# system.in
species = a
interactions primary: aa: filename = measured_u.txt
rdf closure aa = py
rdf alpha_max = 0.05
rdf max_iteration = 100000
rdf tolerance = 1e-6
```

```
rdf beta = 1.0
rdf r_max = 12.0
rdf n_points = 1200
# workflow.py
system = super_dictionary_creator(ctx=ctx, export_json=True)

def my_closure(*, r, gamma, u, sigma_ab=None):
    return np.exp(-u + gamma) - gamma - 1.0

system['system']['rdf']['closure']['aa'] = my_closure
rdf_out = rdf_radial(
    ctx=ctx,
    rdf_config=system,
    densities=np.array([0.2]),
    export=True,
)
```

Here the potential is external numerical data, while the closure is external Python logic. They enter the solver independently.

36.2 Recipe B — analytical self interactions + inverted cross interaction

47. Define aa and bb under interactions using built-in, registered, or file-backed potentials.
48. Supply g_ab only.
49. Materialize the RDF files.
50. Call boltzmann_inversion_advance.
51. Reuse the exported ab potential in a later thermodynamic calculation if desired.

36.3 Recipe C — inversion at many states, then reuse as a potential file

A transparent two-stage scientific workflow is often preferable: first infer a potential from several states and save <prefix>_potential.json; next export the desired pair as a two-column r,U text file and reference that file from a new EOS/coexistence input. The inferred potential then becomes an explicit, archivable input to the thermodynamic stage.

36.4 Recipe D — supplied RDF for one pair, closure solution for the others

When the physical potential is already known and only part of the structure is measured, use rdf_radial constrained projection rather than inversion. The supplied pair is imposed structurally; free pairs are reconstructed from OZ plus their closures.

36.5 Recipe E — file + analytic + custom potential in one mixture

```
# aa is read from file
interactions primary: aa: filename = aa.txt

# ab is a runtime registered functional form
interactions primary: ab: type = screened_exponential, amplitude = 1.2, xi = 0.8, cutoff = 5.0

# bb is a sum of two built-in contributions
interactions primary: bb: type = gs, sigma = 1.4, epsilon = 2.0
interactions secondary: bb: type = mie, sigma = 1.0, epsilon = 0.2, m = 24, n = 12, cutoff = 3.0
```

The potential generators do not require all pairs to be represented by the same mechanism.

36.6 Recipe F — manual closure during multistate inversion

```
system = super_dictionary_creator(ctx=ctx)
supplied = process_supplied_data(ctx=ctx, config=system)

# Use a new closure only for the cross pair.
system['system']['rdf']['closure']['ab'] = my_closure

u, sigma = boltzmann_inversion_advance(
    ctx=ctx,
```

```
    rdf_config=system,
    supplied_data=supplied,
    filename_prefix='effective_multistate',
)
```

Because the pair_closures matrix is built once from rdf_config and reused at every state, the same manual ab closure enters every state-specific OZ solve consistently.

37. Compatibility and Implementation Notes for Advanced Users

Feature	Observed source behavior	Recommended practice
Custom closure in .in file	plain text cannot encode a Python callable	parse first, then inject callable; keep workflow script as provenance
Custom closure registry	CLOSURE_REGISTRY is directly mutable	register before solver call; use a unique uppercase name
Custom potential	formal register_isotropic_pair_potential API exists	register before any potential preprocessing
Tabulated potential path	read relative to os.getcwd()	chdir to input directory before loading
Generic supplied data	creates x/y	use directly for inversion; adapt for other consumers
rdf_radial supplied RDF	expects r/g	convert materialized x/y explicitly
in-memory supplied potential	expects r/U	adapt generic data or use direct filename route
hard-core supplied-potential registrar	contains wrong converter-function alias	prefer filename route until source is repaired
c_analysis supplied_data	signature accepts it but main path ignores it	treat c_analysis as potential+density driven in this snapshot
EOS strength-kernel supplied_data	outer function accepts it, internal rdf/delta_c branches pass None	do not assume EOS reads external RDF data without source modification
multistate weights	uniform and hard-coded	document that all states have equal weight
initial guess for supplied inversion pair	set from -ln(g)	a simultaneously supplied model potential is overwritten as starting guess

37.1 Source improvements that would strengthen the public API

52. Add register_closure(name, fn, overwrite=False) to mirror the potential registry.
53. Add input_dir/work_dir to ExecutionContext and resolve all external files relative to ctx.input_file by default.
54. Standardize supplied numerical data to one canonical schema and centralize x/y <-> r/g <-> r/U adapters.
55. Fix process_supplied_potential.py to import register_potential_converter instead of aliasing convert_potential_via_registry.
56. Allow state weights to be supplied explicitly in multistate inversion.
57. Allow a supplied RDF pair to optionally retain a model potential as its IBI starting guess.
58. Mirror supplied AB/BA automatically in rdf_radial as the inversion processor already does.
59. Add preflight validation for closure callable signatures, pair completeness, state densities, and supplied data dimensions.

38. Advanced Quick Reference

Goal	Use	Input/data form
Use potential from simulation file	interactions pair filename	2-column r,U text
Add new analytical potential	register_isotropic_pair_potential	factory(dict)->V(r), then type=<registered name>
Add new closure	callable in rdf.closure or CLOSURE_REGISTRY	function(r=,gamma=,u=,sigma_ab=)->c(r)
Invert one pair only	known model pairs + supplied RDF for unknown pair	multistate supplied_data with only target pair present
Invert across several states	boltzmann_inversion_advance	states with densities, beta/temperature, pair RDFs
Fix measured RDF and compute other RDFs	rdf_radial projection	supplied_data {rdf:{pair:{r,g}}}
Reuse inverted potential later	export to r,U table and use filename	explicit staged workflow
Use custom potential for fixed external particles	external_potential_grid + same potential registry	external interaction dict uses registered type

39. Final Summary of the Extended Manual

39.1 Primary theoretical sources cited in this theory-enriched edition

1. V. A. Varma and A. Scacchi, “General approach for partitioning and phase separation in macromolecular coexisting phases,” Physical Review E 113, 065414 (2026). Used here for the bulk free-energy framework, Gibbs coexistence criteria, canonical material balances, phase-volume fractions, reduced-density variables, and multicomponent phase-partitioning interpretation.

2. V. A. Varma, A. J. Archer, and A. Scacchi, “A route to the thermodynamics of colloid-polymer mixtures from structural information,” manuscript dated 16 August 2026, supplied with this manual revision. Used here for RDF/structure-factor/DCF relationships, OZ theory, closure choices, structural reference mapping, DCF-based free-energy corrections, long-wavelength correlations, and structural routes to EOS/coexistence.

Citation convention in this manual: short parenthetical theory citations identify which of the two supplied papers supports a theoretical statement. Package-specific behavior, function signatures, field names, and data contracts remain source-audited from the Python package itself.

The package is more extensible than its bundled examples alone suggest. A user can mix analytical and file-backed potentials across different species pairs, decompose one pair into primary/secondary/tertiary contributions, introduce a completely new potential factory at runtime, replace one closure by a Python callable, constrain selected RDFs while calculating the remaining structure, or infer only selected unknown interactions from RDF data collected at several state points.

The most important practical rule is to track the data contract at every boundary: .in text -> parsed dictionary; filename -> numerical interpolation; generic supplied file -> x/y; structural projection -> r/g; supplied-potential registration -> r/U; RDF state -> g_target/fixed_mask; and inversion -> a common u_matrix plus sigma_matrix. When those boundaries are documented explicitly, the same cDFT_solver modules can be combined in many different but reproducible scientific workflows.